

Optimal Constrained Control Allocation for USVs

Sigurd von Brandis

Supervisor: Professor Thor I. Fossen

2025-06-03

Abstract

In environments with small margins for positional and heading errors, strict requirements for precise control of marine vessels can pose significant challenges. In this thesis, the controllability of a dual-azimuth pod catamaran is investigated for such requirements by focusing on the development and comparison of multiple control allocation strategies informed by established methods (pseudo-inverse and nonlinear optimization). A physics-based vessel model is constructed, and appropriate paths, guidance laws, and motion controls are implemented to generate results for comparison. A Nomoto-heading proportional-integral-derivative (PID) controller is used for motion control in path-following (PF) tests, and a multiple-input-multiple-output (MIMO) PID is used as a motion controller to evaluate the potential for dynamic positioning (DP) and automatic berthing. The work includes simulations with the vessel model following both straight-line and curved paths (Fermat's Spiral), looking at how desired states affects the control allocation and vessel response. Different scenarios include ideal environmental conditions and constant water currents, with additional tests performed with pod failure to assess fault tolerance. Results highlight how each allocation method handles actuator limits, environmental disturbances, and pod failures, demonstrating their performance and robustness under the different scenarios.

Sammen drag

I omgivelser med små marginer for posisjon og kursfeil kan krav om presis kontroll av marine fartøy by på betydelige utfordringer. I denne oppgaven undersøkes kontrollerbarheten til en katamaran med to azimuth-podder for slike krav ved å fokusere på utvikling og sammenligning av ulike kontrollallokeringsstrategier basert på etablerte metoder (pseudo-invers og ikke-lineær optimalisering). En fysikkbasert fartøysmodell er konstruert, og egnede baner, veilednings lover og bevegelseskontrollere er implementert for å generere resultater for sammenligning. En Nomoto-heading proporsjonal-integral-derivat (PID)-kontroller brukes for bevegelses kontroll under bane følgning, og en flerinngangs-flerutgangs (MIMO) PID brukes som bevegelseskontroller for å evaluere potensialet for dynamisk posisjonering og automatisk fortøyning. Banefølgningsevnen til fartøyet testes for både rette og kurvede baner (Fermats spiral), der det undersøkes hvordan ønskede tilstander påvirker kontrollallokeringen og fartøyets respons. Testene blir gjort med både ideelle forhold og konstante vannstrømmer. Kontrollerbarheten testes også ved svikt i en av podene. Resultatene viser hvordan hver allokeringsmetode håndterer aktuator begrensninger, ekstern påvirkning og pod feil, og demonstrerer ytelse og robusthet under de ulike senarioene.

Contents

Abstract	i
Sammendrag	iii
Contents	v
Figures	vii
Tables	xi
Acronyms	xiii
Glossary	xv
1 Introduction	1
1.1 Literature review	2
1.2 Problem definition	3
1.3 Main Ambitions	3
1.4 Report Structure	4
2 Background Theory and Control Allocation Methodologies	5
2.1 Background Theory	5
2.1.1 Mathematical concepts	5
2.1.2 Pseudo-Inverse Fundamentals	6
2.1.3 Optimization Fundamentals	7
2.1.4 Machine Learning Fundamentals	9
2.2 Review of Control Allocation Methodologies	10
2.2.1 A Survey on Control Allocation	11
2.2.2 Model predictive Control (MPC)	15
2.2.3 Mixed Integer Programming (MIP)	16
2.2.4 Neural Networks (NN)	17
3 Vessel Model	19
3.1 Model Specifications	20
3.2 Coordinate Frames	20
3.3 States	21
3.4 Ocean current and relative velocity	21
3.5 Inertia and trim	22
3.6 System Mass and Coriolis Matrices	22
3.7 Hydrodynamic added mass and Coriolis	23
3.8 Hydrostatics	24
3.9 Cross flow drag	25
3.10 Payload forces and moments	26

3.11	Trim condition	27
3.12	Azimuth Pod modeling	28
3.13	Kinematic transformation for position and state derivative update	29
3.14	Vessel State Propagation using RK4 Integration	30
3.15	Input matrix (B)	31
3.16	Pod State Propagation using Euler Integration	32
4	Control Allocation Methods and Implementation	33
4.1	Deciding on Control Allocation	33
4.2	Motion Control	34
4.2.1	Nomoto Heading PID	34
4.2.2	Nonlinear MIMO PID	35
4.3	Control Allocation: Pseudo Inverse	37
4.4	Control Allocation: Nonlinear Optimization	38
4.5	Control Allocation: MPC	40
4.6	Control System: MPC for DP and Berthing	40
5	Path Planning and Guidance	49
5.1	Path Planning	49
5.1.1	Straight Line Path	49
5.1.2	Continuous-Curvature Path Using Fermat's Spiral	51
5.2	Guidance laws	59
5.2.1	Line-of-Sight (LOS)	59
5.2.2	Dynamic Positioning (DP)	60
6	Results and Discussion	61
6.1	Path following	61
6.1.1	PF in Ideal Conditions	62
6.1.2	PF with Ocean Currents	66
6.1.3	PF with Pod Failure	67
6.2	Four-square test: Dynamic Positioning and Berthing	70
6.2.1	DP in Ideal conditions	70
6.2.2	Four-square test: with Ocean Currents	77
6.2.3	Four-square test: with Pod Failure	81
6.3	Points of consideration	82
7	Conclusion	85
	Bibliography	87
A	Simplified model parameters used by the MPC control system	91
B	Repository Structure	93
C	Project Description	95
D	Standard Agreement Contract	97

Figures

1.1	Overview of the custom Guidance, Navigation and Control system developed for this project. The main contributions are highlighted in red to show their place in the system architecture. These contributions include a path implementation guide, control system designs and a model of RAN. The figure is adapted from [1].	4
2.1	Overview of pseudo-inverse based control-allocation methods, placed under the pseudo-inverse, redistributed pseudo-inverse, daisy chaining or direct allocation topic.	10
2.2	Overview of optimization-based control-allocation methods, placed under the Adaptive Control, Model Predictive Control (MPC), Dynamic Allocation, Error Minimization, Mixed-Integer Programming (MIP), Nonlinear Programming (NLP) or Post-Processing topic. . . .	11
2.3	Overview of control-allocation methods utilizing machine-learning techniques, categorized into Deep Neural Network (DNN) based allocation and hybrids of Model Predictive control (MPC) and Reinforcement learning (RL) based allocation.	11
3.1	Coordinate Frames. Payload location, CG and CF relative to CO. Picture made with <i>DALL-E</i>	19
3.2	Illustrative picture of the azimuth pod setup on RAN, seen from behind in a 2D plane. $T(n)$ is the thrust given from propeller speed n . α is the azimuth angle of the pod, deciding thrust direction. α is physically constrained to an interval of $\pm 90^\circ$. The picture was made using <i>DALL-E</i>	28
4.1	Illustration of the soft exponential penalty function f_1 in Eq. 4.28 and f_2 Eq. 4.28 . Penalties for directing thrust into another propellers slipstream	40
4.2	Comparing $\sqrt{a^2}$ to $\sqrt{a^2 + \varepsilon}$ shows how adding a small $\varepsilon > 0$ removes the nondifferentiable corner at $a = 0$. The smoothed curve has continuous derivatives, which enables gradient-based solvers like IPOPT to find a solution reliably.	43

5.1	Showcase of how a Fermat's Spiral path (in blue) can look like when connecting waypoints (in red).	52
6.1	Path Following (PF) system 1 performance under ideal conditions. The red line is the desired path, the red dot's are the waypoints, the blue line is the path taken by the vessel and the black arrows shows the vessels heading along the path.	63
6.2	PF system 1. Actual vs. commanded propeller speed under ideal conditions using the straight line path.	63
6.3	PF system 1. Actual vs. commanded propeller speed under ideal conditions using the Fermat's spiral path.	64
6.4	PF system 2. Actual vs. commanded propeller speeds when following the straight line path under ideal conditions.	64
6.5	PF system 2. Actual vs. commanded Azimuth Angles when following the straight line path under ideal conditions.	65
6.6	PF system 3. Actual vs. commanded propeller speeds under ideal conditions.	65
6.7	PF system 3. Actual vs. commanded Azimuth Angles under ideal conditions.	66
6.8	Cross-track error when using PF system 1 and following a Fermat's Spiral path (FS path) with ocean currents. Along each straight-line segment of the FS path, the cross-track error eventually stabilizes when the heading controller has adjusted the vessel's heading sufficiently.	66
6.9	Vessel response using PF system 1, with ocean currents on a FS path. The red line is the desired path, the red dot's are the waypoints, the blue line is the path taken by the vessel and the black arrows shows the vessels heading along the path.	67
6.10	PF system 2 with pod (1) failure: Vessel response when following the FS path in ideal environmental conditions.	68
6.11	PF system 2 with pod (1) failure: Commanded vs Actual azimuth angles.	68
6.12	PF system 2 with pod (1) failure: Commanded vs Actual propeller speeds	69
6.13	The four-square test performance of the vessel under each control system under ideal conditions.	71
6.14	Dynamic Positioning (DP) system 1: X, Y and heading error over time.	72
6.15	DP system 1: Actual vs. commanded propeller speeds.	72
6.16	DP system 1: Actual vs. commanded azimuth angles.	72

6.17 The virtual control inputs generated by the Multiple Input Multiple Output (MIMO) Proportional–Integral–Derivative (PID) motion controller. The plot is made using DP system 1 and it is almost identical for system 2 and 3. The signal is far from realizable because of its high frequency, which explains the rapid changes in propeller speed and azimuth angles seen from the allocation methods used by DP system 1, 2, and 3 in Table 6.4	73
6.18 DP system 2: X, Y and heading error over time.	73
6.19 DP system 2: Actual vs. commanded propeller speeds.	74
6.20 DP system 2: Actual vs. commanded azimuth angles.	74
6.21 DP system 3: X, Y and heading error over time.	75
6.22 DP system 3: Actual vs. commanded propeller speeds.	75
6.23 DP system 3: Actual vs. commanded azimuth angles.	75
6.24 DP system 4: X, Y and heading error over time.	76
6.25 DP system 4: Actual vs. commanded propeller speeds.	76
6.26 DP system 4: Actual vs. commanded azimuth angles.	76
6.27 Vessel response in the four-square test under each control scheme, with ocean currents of 0.3 [m/s] with a 30° direction.	77
6.28 DP system 2 under ocean currents: X, Y and heading error over time.	78
6.29 DP system 2 under ocean currents: Actual vs. commanded propeller speeds.	78
6.30 DP system 2 under ocean currents: Actual vs. commanded azimuth angles.	79
6.31 DP system 3 under ocean currents: X, Y and heading error over time.	79
6.32 DP system 4 under ocean currents: X, Y and heading error over time.	80
6.33 DP system 4 under ocean currents: Actual vs. commanded propeller speeds.	80
6.34 DP system 4 under ocean currents: Actual vs. commanded azimuth angles.	80
6.35 DP system 4 with pod failure ($n_1 = 0$): Vessel response.	81
6.36 DP system 4 with pod failure ($n_1 = 0$): X, Y and heading error over time.	82
6.37 DP system 4 with pod failure ($n_1 = 0$): Actual vs. commanded propeller speeds.	82
6.38 DP system 4 with pod failure ($n_1 = 0$): Actual vs. commanded azimuth angles.	82

Tables

3.1	Key parameters for the RAN Unmanned Surface Vehicle (USV) model, listing geometric dimensions, mass and inertia properties, hydrodynamic coefficients and propulsion limits.	20
6.1	Control systems used for path following. The motion controller generate virtual control signals that the allocators try to replicate by commanding the propeller speed and azimuth angles of each pod. .	62
6.2	Description of PF tests. Under Ideal conditions pods are full operational and there are no wind or water currents affecting the vessel. Test 1 and 2 compares system performance for different paths under ideal conditions. In test 3, only a small ocean current is used to demonstrate how the vessel responds to ocean currents when there are no explicit ocean current compensation in the control loop. To evaluate system performance in the event of a pod failure, the propeller speed of pod 1 is set to zero under ideal environmental conditions in test 4.	62
6.3	Technical details of the four-square test used to evaluate DP and berthing capabilities.	70
6.4	Control system configurations for DP and berthing.	70
A.1	Simplified vessel-model parameters used by the MPC control system to predict vessel movement over a horizon, using propeller speed, azimuth angle and ocean current information.	91

Acronyms

ADCP Acoustic Doppler Current Profiler. 21

ASVs Autonomous Surface Vehicles. 1

CG Center of Gravity. 20, 22

CO Coordinate Origin. 20–23, 29

DNN Deep Neural Network. 17

DOF Degrees of Freedom. 2, 17, 27, 33, 35, 40, 41, 81

DP Dynamic Positioning. viii, ix, 15, 16, 33–35, 71–82, 84, 85

FS path Fermat’s Spiral path. viii, 65–68

GA Generic Algorithm. 16

GPU Graphics Processing Unit. 17

LCF Longitudinal Center of Flotation. 21

LP Linear Programming. 12, 13, 16

LSTD Least-Squares Temporal-Difference. 17

MILP Mixed Integer Linear Programming. 14–16

MIMO Multiple Input Multiple Output. ix, 34, 35, 40, 71, 73, 84–86

MIP Mixed Integer Programming. 14, 16, 17, 86

MIQP Mixed Integer Quadratic Programming. 16

MPC Model Predictive Control. 9, 13, 15–17, 33, 34, 41, 81, 86

MPQP Multiparametric Quadratic Programming. 13

NN Neural Network. 9, 10, 17, 86

PF Path Following. viii, xi, 61–69, 83, 85

PID Proportional–Integral–Derivative. ix, 34, 35, 40, 66, 71, 73, 84–86

PWL Piecewise linear. 13

QP Quadratic Programming. 12, 13, 16

RAM Random Access Memory. 16

RL Reinforcement Learning. 10, 17, 86

SQP Sequential Quadratic Programming. 14

SVD Singular Value Decomposition. 7

USV Unmanned Surface Vehicle. xi, 1, 3, 20, 40, 83, 85

Glossary

aft toward the stern (rear) of a vessel. 2

CasADi An open-source software framework for algorithmic differentiation and numerical optimization, tailored for modeling and solving control and dynamic system problems. 38

hull The main watertight body of a vessel, providing buoyancy and structural integrity. 2

IPOPT An open-source nonlinear optimizer that solves large-scale continuous optimization problems using an interior-point method. vii, 34, 38, 42, 43, 46–48

Opti CasADi’s high-level, user-friendly interface for formulating and solving optimization problems. 38

RAN A research vessel from Kongsberg Discovery. vii, xi, 1–5, 19–21, 28, 33, 40, 41, 81, 83–85

slipstream The high-speed wake of water accelerated aft by a propeller’s rotation. 3, 29, 33

Chapter 1

Introduction

There is a growing interest in deploying Autonomous Surface Vehicles (ASVs) in coastal regions, ports, and sheltered waterways. These are areas where precision maneuvers and tight navigation tolerances are essential. This report aims to assess the potential of smaller catamaran USVs, equipped with dual-azimuth pods, for autonomous execution of low-speed operations in confined waters by analyzing the control-allocation problem.

Sources and tools The main information sources used to find relevant literature are NTNU Open and Google Scholar. Parts of the report have been refined using ChatGPT. Two illustrative drawings have been generated using *DALL-E*. A system overview figure was made with PowerPoint. To generate results and test methods, a program was written from scratch in *C++* using Visual Studio Code, and version control was managed with GitHub.

Collaboration with Industry Partner The report was produced in collaboration with Kongsberg Discovery. They have provided approximate model specifications for their USV, named RAN. RAN is designed for ecosystem monitoring and maritime autonomy research. Kongsberg Discovery has granted permission to include these model specifications in this report to support open research and knowledge sharing.

Relevance to UN Sustainable Development Goals This work supports the development of autonomous transport and monitoring vessels. These vessels can be much smaller because they do not have to carry humans. Smaller vessels generally use less energy. By reducing energy demand, smaller vessels lower fuel costs and enable the integration of renewable power, supporting SDG 7 (Affordable and Clean Energy). Smaller vessels also cause less disturbance in sensitive coastal zones which contributes to SDG 14 (Life Below Water). Lower energy usage and reduced environmental impact further support SDG 13 (Climate Action). In addition, the development of maritime technologies aligns with SDG 9 (Industry Innovation and Infrastructure).

1.1 Literature review

Motion control systems are used to control marine craft motion by means of actuators. Actuators are electromechanical devices that are used to control magnitude and/or direction of forces generated by individual effectors. Effectors are mechanical devices that can generate time-varying forces and moments to control a marine craft. [1]

The main goal of a control allocation algorithm is to determine a control input that allows the effectors to collectively generate the desired virtual control $\tau_c = h(u, x, t)$ (forces and moments) at all times, where u are the actuator commands, x is the current system state and t is time. However, this objective may not always be achievable if the commanded virtual control τ_c demands forces or moments exceeding the effectors' capabilities due to saturation or other physical constraints. When no feasible control input exists, the control allocation algorithm usually have to allow performance degradation and try to find a control input that can minimize the allocation error, $\tau_c - h(u, x, t)$, in some way. [2]

For a marine craft in n Degrees of Freedom (DOF) it is necessary to distribute the generalized control forces and moments $\tau \in \mathbb{R}^n$ to the actuators in terms of control inputs $u \in \mathbb{R}^r$. If $r > n$, this becomes an overactuated control problem, meaning that there are more effectors than strictly needed to meet the motion control objectives of a given application [1]. Most control allocation methods for marine surface vessels assumes the marine vessels are overactuated ([2], [3], [4], [5]). This is favorable due to effector redundancy allowing for fault tolerance and control reconfiguration, and usually involves an optimization problem to find the optimal actuator setpoints [1].

RAN is a catamaran with one azimuth pod mounted beneath the aft section of each hull. A catamaran is a watercraft with two parallel hulls [6]. Azimuth pods are podded propellers, which can be rotated an angle α about the z axis and produce two force components (F_x, F_y) in the horizontal plane, allowing for control of thrust direction as well as force [1]. With a control input for thrust magnitude and angle for each azimuth pod, a marine vessel with two azimuth pods has 4 control inputs. Marine surface crafts are often only controlled in the horizontal plane corresponding to the DOF surge, sway and yaw as described in SNAME 1950 [7] ([1], [2], [8]). Given that RAN operates in 3 DOF while having 4 control inputs, the system is overactuated.

Azimuth pod's might offer enhanced maneuvering capabilities compared to non-rotatable actuator solutions. They do, however, present challenges in the form of constraints. Azimuth pod's have a maximum turning rate, which can cause singularities and have significance for maneuverability [5]. In a singular configuration there are commanded surge and sway forces, and yaw moment that cannot be obtained by applying finite thrusts T from the thruster devices [9]. It's worth considering that if the maximum turning rate is too slow an azimuth pod cannot be considered to have two DOF to the same extent as two independent, perpendicular, fixed thruster devices [9]. Azimuth pod's can also have mechanical limits to

the angle of rotation α , which can lead to singular configurations [9]. Singularity avoidance might be necessary to avoid thrust loss due to slowly varying angles, mechanical limits, but also interactions such as directing a pod's slipstream into another pod's propeller disc [2] [5].

For marine vessels, control allocation is generally explicit for fixed-direction actuators, meaning that there is a direct (often linear) mapping from desired forces and moments to actuator commands. However, with rotatable thrusters, the problem becomes implicit, as the actuator commands are linked by coupled equations rather than a single invertible formula. In addition, if the thruster angles are constrained to specific ranges, the problem becomes non-convex instead of convex. A set is convex if any straight line between points in the feasible region lies entirely within that region. This property ensures that any local minimum of a convex objective is guaranteed to be a global minimum. If this does not hold true the set is non-convex, meaning that local minima may not be global, which makes finding the best solution more challenging. [10]

1.2 Problem definition

The control allocation problem that needs to be solved for the catamaran vessel RAN, is to compute the thrust magnitudes and azimuth angles of its dual pods so that the desired forces and moments are achieved within their operational limits. Given the large scope of the problem, this report seeks to answer the following research questions:

- Q1: How can a control allocation method, for a catamaran equipped with dual azimuth pod's, be implemented for optimal performance at low speeds in confined waters?
- Q2: How can physical factors and operational constraints, such as actuator limits and environmental disturbances, be incorporated to enhance performance and safety?
- Q3: To what extent can such a control allocation method ensure fault tolerance during critical azimuth pod failures, solver issues (e.g., non-convergence, infeasible solutions), or numerical instability, while maintaining operational safety?

1.3 Main Ambitions

The primary objective of this research is to investigate the control allocation problem for the USV RAN, in the context of performing marine operations at low speeds in confined waters. This project aims to develop control allocation strategies tailored to this specific vessel, leveraging established methods and adapting them to the particular needs and constraints of RAN.

For this purpose, a comprehensive literature study of control allocation methodologies is done with the goal of giving the best possible background and under-

standing for developing control schemes suited for catamaran vessels with one azimuth pod mounted beneath the aft section of each hull.

A model of the vessel needs to be constructed and used together with appropriate paths and guidance, in combination with uncertainties in sensor data and ocean current disturbances, to evaluate the developed control methods. The methods should also be evaluated under actuator-failure conditions.

The focus areas and main contributions of this report is highlighted in **Figure 1.1**, which shows where these contributions fit into a larger system for vessel autonomy.

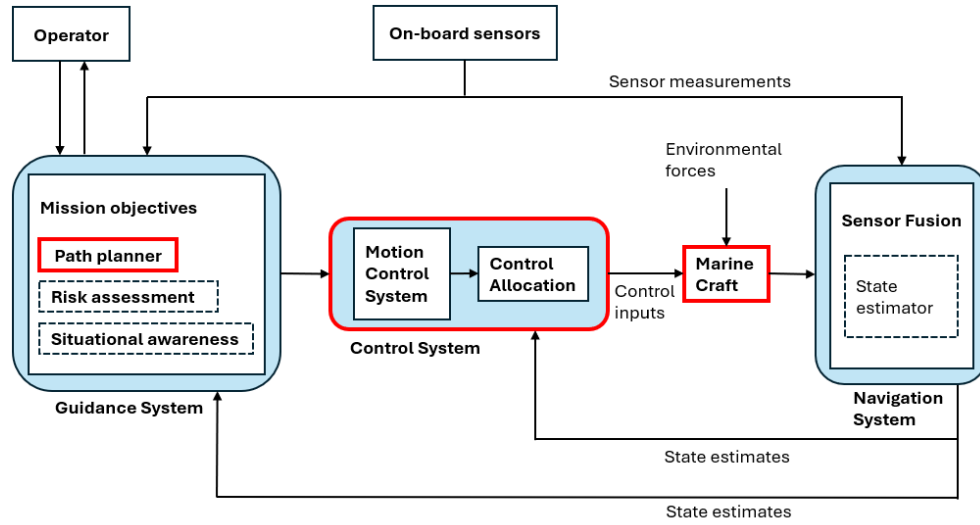


Figure 1.1: Overview of the custom Guidance, Navigation and Control system developed for this project. The main contributions are highlighted in red to show their place in the system architecture. These contributions include a path implementation guide, control system designs and a model of RAN. The figure is adapted from [1].

1.4 Report Structure

The report is structured to guide the reader from theoretical foundations to practical implementation and results. **Chapter 2** contains background theory and a comprehensive review of control allocation methodologies. **Chapter 3** describes the vessel model. **Chapter 4** presents the control methods developed to address the specific control requirements of the model. **Chapter 5** outlines the path generation strategies and guidance laws employed to evaluate control schemes for the vessel across various scenarios. The results are presented and discussed in **Chapter 6**. Finally, **Chapter 7** concludes the report and summarizes key findings and suggestions for future work. Additional references and supporting material are provided in the bibliography and appendix at the end.

Chapter 2

Background Theory and Control Allocation Methodologies

This chapter contains a comprehensive review of control allocation methods to be considered for this work, together with the theoretical background needed to understand both the surveyed techniques and the dual-pod catamaran (RAN-specific) approaches developed in **Chapter 4**.

2.1 Background Theory

This section provides the theoretical background necessary to comprehend the underlying principles of the methods reviewed in this report.

2.1.1 Mathematical concepts

Common mathematical operations and notions that appear throughout this report are summarized here for completeness.

Absolute value: 2-Norm The absolute value of a real number x , denoted $|x|$, measures its distance from zero on the real line. In vector spaces this generalizes to the 2-norm (Euclidean norm) of a vector $\mathbf{v} = [v_1, \dots, v_n]^T$, defined as

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}.$$

The 2-norm is always non-negative, and is zero if and only if $\mathbf{v} = \mathbf{0}$.

Euclidean distance Given two points $\mathbf{p} = [p_1, \dots, p_n]^T$ and $\mathbf{q} = [q_1, \dots, q_n]^T$ in $\mathbb{R}^n$, their Euclidean distance is

$$d(\mathbf{p}, \mathbf{q}) = \|\mathbf{p} - \mathbf{q}\|_2 = \sqrt{(p_1 - q_1)^2 + \dots + (p_n - q_n)^2}.$$

Transpose For any matrix $A \in \mathbb{R}^{m \times n}$, its transpose A^T is the $n \times m$ matrix whose (i, j) entry is A_{ji} :

$$(A^T)_{ij} = A_{ji}.$$

In particular, a column vector $\mathbf{v} \in \mathbb{R}^n$ transposes into the row vector $\mathbf{v}^T$.

Normalization Normalization scales a nonzero vector $\mathbf{v}$ to unit length while preserving its direction. The normalized vector is

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2},$$

so that $\|\hat{\mathbf{v}}\|_2 = 1$.

Trigonometry Trigonometric functions relate angles to ratios of sides in a right triangle or coordinates on the unit circle. Key definitions are

$$\sin \theta = \frac{\text{opposite}}{\text{hypotenuse}}, \quad \cos \theta = \frac{\text{adjacent}}{\text{hypotenuse}}, \quad \tan \theta = \frac{\sin \theta}{\cos \theta}.$$

On the unit circle, a point at angle θ has coordinates $(\cos \theta, \sin \theta)$.

Two-argument arctangent: atan2 The function $\text{atan2}(y, x)$ returns the angle θ whose tangent is y/x , taking into account the signs of both x and y to determine the correct quadrant. It is defined for $(x, y) \neq (0, 0)$ with range $-\pi < \theta \leq \pi$, and satisfies

$$\theta = \begin{cases} \arctan(y/x) & x > 0, \\ \arctan(y/x) + \pi & x < 0 \text{ and } y \geq 0, \\ \arctan(y/x) - \pi & x < 0 \text{ and } y < 0, \\ +\frac{\pi}{2} & x = 0, y > 0, \\ -\frac{\pi}{2} & x = 0, y < 0. \end{cases}$$

This ensures a continuous mapping of (x, y) to the full circle of angles.

2.1.2 Pseudo-Inverse Fundamentals

This summary is made using the textbook *Differential Equations, Linear Algebra and its Applications* [11]. The inverse of a square matrix A is another matrix A^{-1} such that:

$$AA^{-1} = A^{-1}A = I,$$

where I is the identity matrix. A matrix must be square and non-singular (i.e., $\det(A) \neq 0$) to have an inverse.

For a 2×2 matrix:

$$A = \begin{bmatrix} a & b & c & d \end{bmatrix}, \quad A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b & -c & a \end{bmatrix}.$$

If $\det(A) = 0$, the inverse does not exist.

For non-square or singular matrices, the Moore-Penrose pseudo-inverse A^+ provides a least-squares solution. It is defined such that:

$$AA^+A = A, \quad A^+AA^+ = A^+, \quad (AA^+)^T = AA^+, \quad (A^+A)^T = A^+A.$$

For tall matrices ($m > n$):

$$A^+ = (A^T A)^{-1} A^T, \quad \text{if } A^T A \text{ is invertible.}$$

For wide matrices ($m < n$):

$$A^+ = A^T (AA^T)^{-1}, \quad \text{if } AA^T \text{ is invertible.}$$

For any matrix, A^+ can be computed using the Singular Value Decomposition (SVD):

$$A = U \Sigma V^T, \quad A^+ = V \Sigma^+ U^T,$$

where Σ^+ is obtained by inverting nonzero singular values σ and leaving zeros unchanged.

2.1.3 Optimization Fundamentals

This summary is made using the tech report *Mergin Optimization and Control* [12]. Mathematical optimization problem seeks a decision variable $z \in \mathbb{R}^n$ that minimizes or maximizes an objective function $f(z)$ while satisfying constraints. A general formulation can be written as;

$$\begin{aligned} & \min_{z \in \mathbb{R}^n} f(z) \\ & \text{subject to} \quad c_i(z) = 0, \quad i \in E, \\ & \quad \quad \quad c_i(z) \geq 0, \quad i \in I. \end{aligned}$$

where $f(z)$ is the objective function and $c_i(z)$ represents the constraints in the disjoint index sets for equality (E) and inequality (I) constraints.

The feasible set is defined as the collection of all z that satisfy the constraints

$$\Omega = \{z \in \mathbb{R}^n \mid (c_i(z) = 0 \text{ for } i \in E) \wedge (c_i(z) \geq 0 \text{ for } i \in I)\}.$$

Ω is the search region in which every candidate solution must lie. It contains exactly those vectors z that satisfy all equality constraints and do not violate any inequality. Each equality constraint carves out a surface in $\mathbb{R}^n$, and each inequality carves out a half-space. The feasible set is the intersection of these regions. Any point outside Ω violates at least one constraint and is therefore inadmissible. In some instances, the domain of $f(z)$ further restricts the feasible set.

Convexity A set $\Omega \subseteq \mathbb{R}^n$ is *convex* if for any two points $u_1, u_2 \in \Omega$ and any $\lambda \in [0, 1]$ the convex combination

$$u_\lambda = \lambda u_1 + (1 - \lambda)u_2$$

also lies in Ω . A function $f : \Omega \rightarrow \mathbb{R}$ is *convex* if for all $u_1, u_2 \in \Omega$ and any $\lambda \in [0, 1]$

$$f(u_\lambda) \leq \lambda f(u_1) + (1 - \lambda)f(u_2).$$

When both the constraint set Ω and the objective f are convex, any local minimum is guaranteed to be a global minimum, since the cost surface has no other local minima. Furthermore, any nonnegative weighted sum of convex functions remains convex, so adding convex terms preserves overall convexity. A set is *non-convex* if there exist $u_1, u_2 \in \Omega$ and $\lambda \in (0, 1)$ such that $u_\lambda \notin \Omega$, and a function is non-convex if there exist u_1, u_2, λ for which

$$f(u_\lambda) > \lambda f(u_1) + (1 - \lambda)f(u_2).$$

In non-convex problems the feasible region or cost surface can fragment into multiple regions and local minima that are not global, making it difficult for standard solvers to guarantee the best solution.

Linear Programming (LP) A linear program has a linear objective and only linear constraints. In standard form it is written as

$$\begin{aligned} \min_{z \in \mathbb{R}^n} \quad & c^T z \\ \text{subject to} \quad & Az \leq b, \\ & Ez = d, \end{aligned}$$

where

- $c \in \mathbb{R}^n$ is the cost vector,
- $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$ define the inequality constraints $Az \leq b$,
- $E \in \mathbb{R}^{p \times n}$ and $d \in \mathbb{R}^p$ define the equality constraints $Ez = d$.

The feasible set $\{z \mid Az \leq b, Ez = d\}$ is a convex polyhedron. Simplex or interior-point methods exploit this structure to find the unique global optimum in $\mathcal{O}(n)$ to $\mathcal{O}(n^3)$ time, and techniques such as warm-start, anti-cycling rules, and slack variables maintain performance and feasibility under degeneracy.

Quadratic Programming (QP) A convex quadratic program extends LP by including a quadratic term in the objective. In standard form it reads

$$\begin{aligned} \min_{z \in \mathbb{R}^n} \quad & \frac{1}{2} z^T H z + c^T z \\ \text{subject to} \quad & Az \leq b, \\ & Ez = d, \end{aligned}$$

where

- $H \in \mathbb{R}^{n \times n}$ is symmetric positive semidefinite ($H \succeq 0$) to ensure convexity,
- $c \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $E \in \mathbb{R}^{p \times n}$, and $d \in \mathbb{R}^p$ are as in the LP formulation.

The feasible region remains a convex polyhedron, and solvers based on active-set, interior-point, or sequential quadratic programming reliably find the unique global minimizer with polynomial complexity in practice.

Model Predictive Control (MPC) Model Predictive Control (MPC) is an optimal control strategy that integrates dynamic optimization with feedback. At every timestep t , MPC solves a finite-horizon open-loop optimization problem using the current state of the system as the initial condition and a model of the system to calculate the effects of the control allocation for each step in the horizon. This generates an optimal sequence of future control actions, but in general, only the first control input u_t is used directly. The process is then repeated at the next timestep using a moving horizon approach. The two main variants are State Feedback MPC and Output Feedback MPC. Both of these versions can be implemented for linear and nonlinear optimization problems, and are described by **Algorithm 1** and **Algorithm 2**, extracted from [12].

Algorithm 1 State Feedback MPC

```

for  $t = 0, 1, 2, \dots$  do
    Obtain the current state  $x_t$ .

    Solve a dynamic optimization problem on the prediction horizon from
     $t$  to  $t + N$  with  $x_t$  as the initial condition.

    Apply the first control move  $u_t$  from the solution.
end for

```

Algorithm 2 Output Feedback MPC

```

for  $t = 0, 1, 2, \dots$  do
    Compute an estimate of the current state  $\hat{x}_t$  based on the available
    measurements.

    Solve a dynamic optimization problem on the prediction horizon from
     $t$  to  $t + N$  with  $\hat{x}_t$  as the initial condition.

    Apply the first control move  $u_t$  from the solution.
end for

```

2.1.4 Machine Learning Fundamentals

This summary is made using the textbook *Artificial Intelligence: A Modern Approach* [13]. Machine learning is the study of algorithms and statistical models that enable computers to perform tasks by learning patterns from data rather than relying on explicit instructions. It spans a wide range of paradigms, including Neural

Network (NN) for approximating complex nonlinear mappings and Reinforcement Learning (RL) for optimizing sequential decision-making, both of which have been successfully applied to control-allocation problems.

Neural Networks NN's are computational models able to approximate complex functions by learning patterns in data through interconnected layers of simple processing units called neurons. Each neuron receives inputs, processes them using weights and biases, and applies an activation function to produce an output. These networks can capture intricate relationships between input and output by adjusting the weights through a learning process.

Reinforcement Learning RL is a framework in which an agent learns to make decisions by interacting with an environment. At each time step, the agent observes a state s , selects an action a , and receives a reward r . The goal is to learn a policy π that maximizes a expected cumulative reward, which can be formulated as

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

where γ is a discount factor.

2.2 Review of Control Allocation Methodologies

An overview of the different approaches and corresponding references discussed in this section is presented in **Table 2.1**, **2.2** and **2.3**. They clarify the extent of research conducted on each topic and functions as a guide to the reader. Some references fall under multiple approaches and are therefore placed roughly under a topic. Optimization based methods using pseudo-inverses are placed under the *Optimization-Based* topic. All methods utilizing NN or RL theory are placed under the *Machine-Learning-Based* topic.

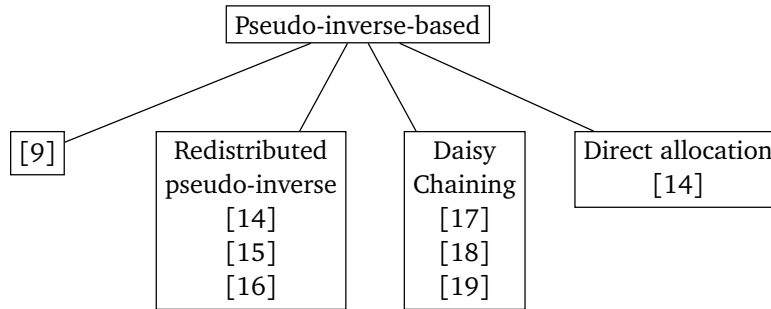


Figure 2.1: Overview of pseudo-inverse based control-allocation methods, placed under the pseudo-inverse, redistributed pseudo-inverse, daisy chaining or direct allocation topic.

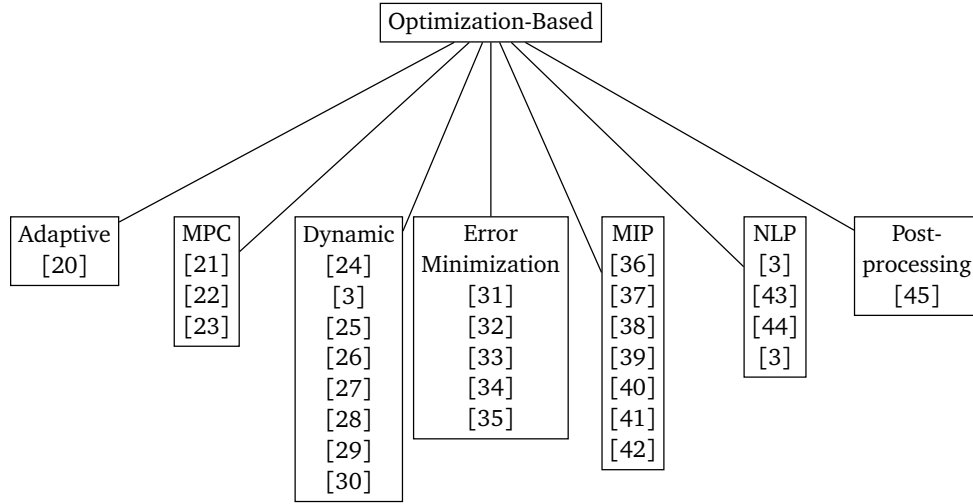


Figure 2.2: Overview of optimization-based control-allocation methods, placed under the Adaptive Control, Model Predictive Control (MPC), Dynamic Allocation, Error Minimization, Mixed-Integer Programming (MIP), Nonlinear Programming (NLP) or Post-Processing topic.

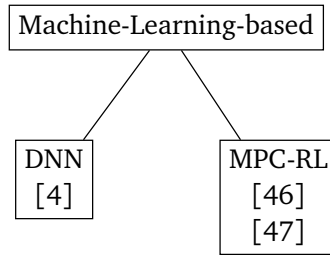


Figure 2.3: Overview of control-allocation methods utilizing machine-learning techniques, categorized into Deep Neural Network (DNN) based allocation and hybrids of Model Predictive control (MPC) and Reinforcement learning (RL) based allocation.

2.2.1 A Survey on Control Allocation

The survey by Johansen and Fossen [2] provides a detailed overview of control allocation methods for over-actuated systems before 2013, classifying algorithms into linear and nonlinear approaches. It emphasizes optimization-based methods for handling constraints, faults, and secondary objectives, highlighting their growing use across aerospace, maritime, and automotive industries. Below, relevant theory and methods from the survey are provided in limited detail to give insight into the assessments made in determining which approaches merit further examination.

According to the survey, control allocation algorithms often assume a linear

effector model on the form

$$\tau = Bu \quad (2.1)$$

where $B \in \mathbb{R}^{m \times p}$ is a matrix describing the relationship between the control inputs $u \in \mathbb{R}^p$ and the virtual control inputs $\tau \in \mathbb{R}^m$ representing forces and moments produced jointly by the effectors.

To find u , one solution is to invert (2.1). While B is not a square matrix, the equation cannot be directly inverted. The survey [2] presents several linear constrained allocation methods that try to solve this problem;

- Methods based on single generalized inverses can be used, but they do not guarantee that constraints on the input $u \in \mathbb{U}$ are satisfied [14].
- A Redistributed pseudo-inverse method solves the unconstrained control allocation problem using the pseudo-inverse, and repeats a redistribution procedure until either a feasible solution is found, or no further information improvement can be made. The method is relatively simple and often effective, but it does not guarantee that a feasible solution is found whenever possible, nor that the resulting control allocation minimizes the allocation error [15] [16].
- A daisy chaining method solves the allocation problem by organizing the effectors into two or more groups. It solves the control allocation problem for the highest priority group first. If any effectors in the group saturate, the entire group remains fixed, while the remaining control allocation is solved by the next group with the highest priority. The algorithm continues if a feasible solution is not achieved and additional groups are available. Depending on the grouping strategy, this approach may result in suboptimal solutions where effectors are not fully utilized to minimize allocation error. ([17], [18], [19]). The method seems better suited for vessels with a large number of effectors. For a vessel with two identical azimuth pods placed symmetrically under the vessel, the grouping and prioritization logic does not seem logical.
- A direct allocation method can be used. It begins with an unconstrained control allocation using a pseudo-inverse. If the input constraints are not satisfied by the solution, the method adjusts the control input to lie on the boundary of the attainable generalized forces set. [14]. The method only tries to ensure feasibility at a given time step. This enables it to handle constraints like actuator limits, but not actuator rates. If the optimal control input jumps significantly from one time step to the next, it can exceed the actuator rates constraints.
- Another approach is error minimization using Linear Programming (LP) or Quadratic Programming (QP) [31]. The approach is to minimize the weighted error between the allocated virtual control input u and the desired one τ_c , subject to actuator constraints. By formulating the problem as an LP with polyhedral constraints, it can be solved efficiently using numerical methods like the simplex algorithm. Slack variables can guarantee

feasibility, and warm-start techniques can improve real-time performance. It may require safeguards to handle degeneracies such as never making progress due to revisiting the same points over and over. By formulating the problem as a QP it can be solved using numerical QP methods like the active set, interior point or fixed-point methods. Benefits of the QP formulation include smoother control solutions, better numerical stability, handling of slack variables and easy rate constraint handling. Similar to the LP formulation, the QP formulation have a problem with guaranteeing a maximum number of iterations and computational time using the active set or interior point algorithms, which can lead to sub-optimal solutions in real-time.

- Explicit approaches to constrained error minimization utilize Multiparametric Quadratic Programming (MPQP) to determine a Piecewise linear (PWL) solution function for the commanded control input τ_c [32] [33]. The approach divides the solution space into multiple linear segments. Using efficient data structures to search the solutions space, the online evaluation times for this approach can be highly efficient. A time-varying B-matrix and constraint sets, along with complex fault tolerance and reconfiguration requirements, may render this approach less practical or difficult to implement due to high memory requirements. [34]
- Methods able to incorporate actuator dynamics can have a meaningful impact on performance. A adaptive control allocation method is proposed in [20]. It's an optimization-based control allocation method integrated with parameter estimation to accommodate for an unknown, time-varying B-matrix due to losses and faults. A dynamic control allocation approach by [24] allocates the required control effort while optimizing secondary objectives and constraints. More sophisticated actuator dynamics can be addressed using MPC to solve constrained control allocation problems ([21], [22]). MPC uses a model to predict actuator dynamics, solving the control allocation problem over a future horizon. It improves dynamic performance by pre-acting on actuator dynamics. MPC requires much computational power due to the increased number of optimization variables which can be problematic for real-time applications. However, in 2011, control allocation using MPC was shown to be feasible using efficient QP solvers [23].
- A similar strategy to MPC, is one that considers the current and past history of commanded virtual forces and moments to minimize the error between the achieved and desired moments in control allocation [35].
- Another alternative for including actuator constraints involves post-processing the static control allocation, where the post-processor overdrives the actuator to compensate for the dynamics of first or second order linear actuator models [45]. Post-processing might be computationally less demanding than looking at the future horizon the way it's done with MPC [2]. A backside to this approach might be less optimal solutions due to the lack of ability to do long term strategic control allocation based on future events.
- A thrust allocation scheme with active azimuth control was proposed by

Sørdalen in 1997 [9] for ships using rotatable azimuth thrusters. It's a method utilizing pseudo-inverses in combination with an extended thrust concept. It handles constraints by saturation strategies and filtering of azimuth angle commands. The singular value decomposition is used to handle cases when temporary controllability is weak due to thrusters being aligned and thrust direction is the same.

Of the many linear effector control allocation models presented by the survey [2], those that can handle constraints on limits to azimuth rotation, and especially those that can handle rate constraints as well, seems promising with respect to the research questions. The linear formulation might not give the desired performance that is needed due to time-varying linearization having insufficient accuracy, effector limitations not being described accurately by the convex polyhedral sets, or if more complex cost functions are needed [2]. Introducing nonlinearity can increase complexity and computations, but might give better performance [2]. The survey presents a few different control allocation methods for nonlinear effector models:

- Nonlinear programming methods for control allocation, introduced by Johansen et al. (2004) [3] and Poonamallee et al. (2004) [43], aim to handle complex actuator constraints and nonlinearities. One such method is Sequential Quadratic Programming (SQP). SQP is an iterative optimization method that solves nonlinear programming problems by approximating the cost function quadratically and the constraints linearly, refining the solution through successive quadratic subproblems [44]. Strong nonlinearities in control allocation can introduce non-convexity, leading to local minima and degraded performance, necessitating additional mechanisms for global optimization [43]. Johansen et al. [3] proposed an approach where the cost function is approximated quadratically, and constraints are linearized once per sample, leading to a computationally efficient method similar to SQP. While effective in many cases, this method may require multiple iterations when large force variations occur between samples [2]. General purpose nonlinear solvers for all control allocation problems remains impractical [2], making specific implementations with such methods less generalizable.
- Mixed Integer Linear Programming (MILP) methods are Mixed Integer Programming (MIP) optimization techniques used when the problem involves piecewise linear functions, non-convex constraint sets, or discrete decision variables. ([36], [37], [38]) While MILP solvers are computationally complex and challenging to verify for real-time safety-critical applications, simplified enumeration methods combined with numerical quadratic programming can effectively handle practical non-convex control allocation problems ([39], [40]).
- A Dynamic optimum-seeking method, as proposed by [3], reformulate static nonlinear control allocation problems using a control Lyapunov function to ensure stable convergence to an optimal solution. By augmenting the

cost function with a penalty-function to enforce constraints, the method derives an update law that asymptotically tracks the optimal control allocation without requiring direct numerical optimization, reducing computational complexity. Performance losses may occur due to deviations from instantaneous optimal allocation, and convergence issues can arise in non-convex cases [25]. Finite-time convergence [26] and adaptive modeling ([27], [28]) have been proposed to enhance the approach. The adaptive approach have been extended to dynamically account for actuator dynamics in the control allocation ([29], [30]), and internal dynamics in the context of model reference adaptive control ([48], [49])

The control allocation methods for nonlinear effector models presented by the survey might give better performance than some of the linear methods, but they might be harder to implement and verify for safety-critical applications. Given the information in the survey and references therein, methods utilizing MPC and MILP are promising candidates with respect to the research questions. This is due to their ability to handle different types of constraints and find optimal solutions. A common backside to these approaches is that they can be too computationally demanding for real time applications, leading to suboptimal performance. That was the case in 2013, but it might not hold true anymore. If the solver based methods does not converge or if they take too much time to compute for higher speeds, a pseudo-inverse based method might work as a fallback strategy, since they can guarantee a solution and are generally fast to compute.

2.2.2 Model predictive Control (MPC)

Made possible by faster compute and efficient algorithms, MPC based methods have gained considerable attraction in recent years due to their ability to optimize thrust commands over an optimization horizon ([10], [8], [50], [41], [47], [46]). One such application is DP, a control mode for precise steering of a vessels position and heading for low speed maneuvering and station-keeping [1].

In [10], a non-angular MPC-based thrust allocation framework for marine surface vessels in DP-operations is proposed. This method computes the thrust angles outside of the optimization problem, giving good real-time properties while having a significant optimization horizon length. Backsides of this approach are that thrust angles and amplitudes must be calculated from vectors after the optimization before they can be used by the propulsion system, and that thrust rates and thrust constraints tend to be too conservative.

In [8] a fault tolerant control allocation approach is proposed. It divides the control allocation problem into a main controller and a control allocator unit. The main controller operates in a d -dimensional space to generate the desired virtual control. The control allocator utilize a pseudo-inverse method, and maps the total control demand onto individual actuator settings. The approach guarantee the determination of the actuators' commands such that the input constraints are satisfied and the desired control effort is produced. This is done by incorporat-

ing feasibility considerations directly into the main controller, which utilizes MPC due to its ability to handle both state and input constraints.

In [50] a method for guaranteed feasible control allocation is proposed. A feasible region is determined by characterizing the polyhedral feasible region of the pseudo inverse solution. This region serves as a constraint for multiple MPC problems, which produce the desired virtual controls. The paper focus on computational efficiency, and tries to approximate the feasible region using hypercubes to reduce computational complexity. Simulation results show reduced computational cost using the approximation. However, the approximation might increase the amount of MPC problems that needs to be solved to get the desired accuracy.

2.2.3 Mixed Integer Programming (MIP)

Control allocation using MIP methods optimizes actuator commands by using integer variables to switch discrete modes (e.g., on/off) and continuous variables to set control levels, ensuring the best use of limited authority under complex constraints. This approach allows exact modeling of actuator constraints, mode switching and fault accommodation, yielding globally optimal allocations under complex safety and performance requirements. However, these formulations leads to exponentially large search trees and rich combinatorial constraints, which quickly increases the Random Access Memory (RAM) requirements and complexity. Too high memory requirements can make the problem unmanageable for a computer and the complexity can make solvers get stuck in local minima. Different methods for handling these challenges has been proposed. ([36], [37], [38], [39], [41], [42])

In [42], MILP and Mixed Integer Quadratic Programming (MIQP) control allocation strategies are investigated for DP systems, with particular attention to integrating operational constraints and balancing computational efficiency against model accuracy. LP offers the greatest simplicity, and QP yields a naturally balanced thrust distribution, but neither can accommodate non-convex constraints. MILP overcomes this limitation by enforcing logical requirements such as forbidden sectors that prevent thrusters from operating in undesirable directions and by modeling thrusters whose reverse efficiency is reduced. It achieves this through approximations that are linear over multiple segments of the nonlinear power-to-thrust curve, where the downside is an increase in computation time. MIQP emerges as an attractive option when the objective function is quadratic. It preserves the integer-constraint flexibility of MILP while handling quadratic cost terms, thereby avoiding the need for extensive segment-wise linear approximations.

In [41] a Generic Algorithm (GA) is applied as the optimization method in a MPC framework to solve control allocation problems involving MIQP. The results was comparable to traditional solvers, like the commercial optimization solver Gurobi, but it manages to solve MIQP's where solvers like Gurobi may get stuck. This is due to the GA's ability to escape local optima when dealing with non-

convexity, where gradient-based solvers can struggle. However, there is no guarantee of converging to the global minimum. Combining MIP with MPC gives a huge increase in variables, leading to significantly slower performance than Gurobi. In addition, the performance is highly dependent on tuning.

2.2.4 Neural Networks (NN)

In recent time, new methods based fully or partially on data-driven machine learning have been proposed [4] [46] [47].

In [4], the customary on-the-fly constrained optimizer is replaced by an autoencoder-style Deep Neural Network (DNN) trained entirely offline. The network's encoder maps the 3-DOF force request $\tau = [\tau_x, \tau_y, \tau_\psi]^T$ to a vector of thruster commands $\hat{\mathbf{u}}$ using a series of recurrent and dense layers, while a decoder reconstructs the generalized forces via the known thruster configuration matrix $B(\alpha)$. Training data are generated synthetically by sampling within each thruster's operational limits and computing the corresponding generalized forces, which are standardized before input. The network is then trained by minimizing a composite loss $\mathcal{L}$, consisting of individual terms that penalize the reconstruction error, violations of the magnitude and rate limits, power usage, and forbidden azimuth sectors. Weights k_i balance these objectives during offline optimization. In deployment, the trained network executes with fixed, low latency suitable for real-time control. Should thruster availability or mission priorities change, retraining can be performed offline in approximately 2 minutes on a modest Graphics Processing Unit (GPU).

In [47], RL is embedded directly into MPC by treating the MPC controller as a parameterized policy. Rather than using a NN, RL calibrates the parameter vector θ that defines all aspects of the MPC problem: stage and terminal cost weights, constraint tightening margins, slack penalties and dynamics parameters, thus preserving MPC interpretability, structure, and hard-constraint guarantees while allowing data-driven adaptation. At each time step k , given the current state s_k , the MPC solves a finite-horizon optimization problem that predicts future motion and computes the control sequence minimizing a cost of path-following/docking objectives, control effort, and slack-based penalties for actuator limits and forbidden-sector violations. A Least-Squares Temporal-Difference (LSTD) variant further improves sample efficiency and stability, yielding a structured, interpretable controller that outperforms fixed-tuned MPC and black-box RL in long-term performance and robustness.

In [46] different RL-augmented MPC methods are presented. One of which suggest embedding RL directly into the MPC, similar to [47]. Another is an Ellipsoid-Tube Robust Nonlinear MPC tuned by RL. This MPC framework augments a nominal nonlinear MPC by enclosing the predicted state trajectory in an ellipsoidal "tube" that conservatively captures all disturbances and model inaccuracies. Simulation studies in obstacle-avoidance scenarios demonstrate that this MPC framework outperforms both nominal MPC and non-learned robust variants in terms of

trajectory-tracking accuracy, while preserving guaranteed obstacle avoidance.

Chapter 3

Vessel Model

To develop and test control methods under realistic scenarios, a new catamaran vessel model was developed. The model was made using theory from [1] and approximate model specifications of RAN. The MATLAB `otter.m` model from the Marine Systems Simulator toolbox [51] served as a reference for the implementation. The model was implemented in C++ due to the languages computational efficiency and open-source purposes. This chapter describes the theory and methods used to model the vessel dynamics, including the relevant coordinate systems, hydrodynamic and hydrostatic forces, Coriolis and added mass effects, as well as the modeling of environmental disturbances such as ocean currents. The numerical integration method used to propagate the vessel states is also described. Furthermore, it incorporates a model of the propulsion from the azimuth pod's, updated by a separate numerical integration.

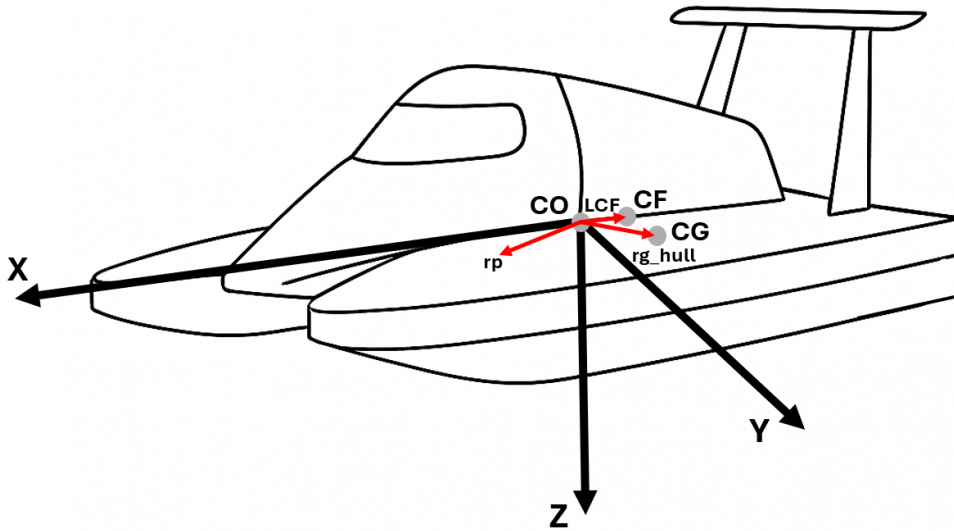


Figure 3.1: Coordinate Frames. Payload location, CG and CF relative to CO. Picture made with *DALL-E*.

3.1 Model Specifications

Table 3.1 defines the parameters used for modeling the USV RAN, specifying each symbol alongside its numerical value and corresponding unit.

Symbol	Value	Units	Description
L	5.0	[m]	Vessel length
Beam	3.0	[m]	Vessel beam
m	800.0	[kg]	Vessel mass
R_{44}	0.4 Beam	[m]	Roll radius of gyration
R_{55}	0.25 L	[m]	Pitch radius of gyration
R_{66}	0.25 L	[m]	Yaw radius of gyration
T_{surge}	5.0	[s]	Sway time constant
T_{sway}	6.0	[s]	Surge time constant
T_{yaw}	5.0	[s]	Yaw time constant
Beam _{pont}	0.70	[m]	Pontoon beam
y_{pont}	1.1	[m]	Pontoon lateral offset from centerline
$C_{w,\text{pont}}$	1	—	Waterline area coefficient
$C_{b,\text{pont}}$	0.5	—	Block coefficient
α	$[-\frac{\pi}{2}, \frac{\pi}{2}]$	[rad]	Azimuth angle range
k_{pos}	880	[N]	Positive bollard pull per pod
k_{neg}	880	[N]	Negative bollard pull per pod
T_n	0.5	[s]	Propeller speed time constant
T_α	0.5	[s]	Azimuth angle time constant

Table 3.1: Key parameters for the RAN USV model, listing geometric dimensions, mass and inertia properties, hydrodynamic coefficients and propulsion limits.

3.2 Coordinate Frames

The BODY frame coordinate system is defined according to the right hand rule, where the X axis points forward, the Y axis points toward the port side, and the Z axis points downward.

The model defines the Coordinate Origin (CO) as the center of the vessel, with BODY frame coordinate:

$$CO = (x, y, z) = (0, 0, 0) \quad (3.1)$$

The vector from the CO to the Center of Gravity (CG) for the hull is defined as

$$r_{g_{\text{hull}}} = [-0.5, 0.0, -0.2] \quad (3.2)$$

The vector from the CO to the payload location (front cabin) is defined as:

$$r_p = [0.75, 0.0, -0.2] \quad (3.3)$$

The vector from the CO to the Longitudinal Center of Flotation (LCF) is defined as:

$$LCF = [-0.2, 0, 0] \quad (3.4)$$

3.3 States

The model assumes we have available measurements or estimates of the body velocities in the BODY frame:

$$\boldsymbol{v} = [u, v, w, p, q, r] \quad (3.5)$$

and the positions and Euler angles in the NED frame:

$$\boldsymbol{\eta} = [x, y, z, \phi, \theta, \psi] \quad (3.6)$$

The state 12 dimensional state vector $\boldsymbol{x}$ is defined as:

$$\boldsymbol{x} = [\boldsymbol{v}, \boldsymbol{\eta}] \quad (3.7)$$

The states u and v allows us to calculate the speed U in the horizontal plane.

$$U = \sqrt{u^2 + v^2} \quad (3.8)$$

3.4 Ocean current and relative velocity

RAN has an Acoustic Doppler Current Profiler (ADCP) under the hull, capable of measuring the ocean current speed V_c and crab angle β_c . Together with the heading ψ we can calculate the relative velocity $\boldsymbol{v}_r$.

$$u_c = V_c \cos(\beta_c - \psi) \quad (3.9)$$

$$v_c = V_c \sin(\beta_c - \psi) \quad (3.10)$$

$$\boldsymbol{v}_c = [u_c \quad v_c \quad 0 \quad 0 \quad 0 \quad 0]^T \quad (3.11)$$

$$\boldsymbol{v}_r = \boldsymbol{v} - \boldsymbol{v}_c \quad (3.12)$$

Further, we can calculate the ocean current acceleration term:

$$\dot{\boldsymbol{v}}_c = \mathbf{0}_{6 \times 1} - \mathbf{S}(\boldsymbol{v}_{0:3}) \boldsymbol{v}_{c,0:3} \quad (3.13)$$

where $\mathbf{S}$ is the Skew-Symmetric matrix:

$$\mathbf{S}(\boldsymbol{\lambda}) = \begin{bmatrix} 0 & -\lambda_3 & \lambda_2 \\ \lambda_3 & 0 & -\lambda_1 \\ -\lambda_2 & \lambda_1 & 0 \end{bmatrix}, \boldsymbol{\lambda} = \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \lambda_3 \end{bmatrix} \quad (3.14)$$

3.5 Inertia and trim

The displaced volume, which determines the buoyant force supporting both hull and payload, is

$$\nabla = \frac{m + m_p}{\rho} \quad (3.15)$$

where m is the hull mass, m_p the payload mass, and ρ the fluid density. From this volume and the pontoon geometry we compute the vessel's draft as the depth required to displace ∇ :

$$T_{\text{draft}} = \frac{\nabla}{2C_{b,\text{pont}}B_{\text{pont}}L} \quad (3.16)$$

with $C_{b,\text{pont}}$ the pontoon block coefficient, B_{pont} its beam, and L the waterline length. The inertia dyadic for the hull only at the CG can be computed:

$$\mathbf{I}_G^{\text{CG}} = \text{diag}(R_{44}^2, R_{55}^2, R_{66}^2) \quad (3.17)$$

Including the payload shifts the overall center of gravity to

$$\mathbf{r}_g^{\text{corrected}} = \frac{m\mathbf{r}_g^{\text{hull}} + m_p\mathbf{r}_p}{m + m_p} \quad (3.18)$$

Finally, we can express the total inertia matrix at CG as

$$\mathbf{I}_g = \mathbf{I}_g^{\text{CG}} - m(\mathbf{S}(\mathbf{r}_g^{\text{hull}})\mathbf{S}(\mathbf{r}_g^{\text{hull}})) - m_p(\mathbf{S}(\mathbf{r}_p)\mathbf{S}(\mathbf{r}_p)) \quad (3.19)$$

3.6 System Mass and Coriolis Matrices

To calculate the equations of motion for the marine craft, we calculate the rigid-body-kinetics. The rigid body mass matrix in the CG is defined as:

$$M_{RB}^{CG} = \begin{bmatrix} (m + m_p)I_3 & 0_{0:3} \\ 0_{0:3} & I_g \end{bmatrix} \quad (3.20)$$

The rigid body Coriolis and centripetal matrix in the CG is defined as:

$$C_{RB}^{CG} = \begin{bmatrix} (m + m_p)S(v_2) & 0_{0:3} \\ 0_{0:3} & S(I_g v_2) \end{bmatrix} \quad (3.21)$$

To transform these rigid-body matrices into the CO from the CG, we can use the transformation matrix H :

$$H(r_{bg}^b) = \begin{bmatrix} I_3 & S^T(r_{bg}^b) \\ 0_{3 \times 3} & I_3 \end{bmatrix} \quad (3.22)$$

and it's transposed

$$H^T(r_{bg}^b) = \begin{bmatrix} I_3 & 0_{3 \times 3} \\ S(r_{bg}^b) & I_3 \end{bmatrix} \quad (3.23)$$

Finally, the matrices is expressed in the CO as:

$$M_{RB} = H^T(r_{bg}^b)M_{RB}^{CG}H(r_{bg}^b) \quad (3.24)$$

$$C_{RB} = H^T(r_{bg}^b)C_{RB}^{CG}H(r_{bg}^b) \quad (3.25)$$

3.7 Hydrodynamic added mass and Coriolis

In order to compensate for fluid flow phenomena, caused by pressure from the fluid water, the model uses hydrodynamic concepts and potential theory. The hydrodynamic added mass is modeled as:

$$X_{\dot{u}} = -0.1 \, m \quad (3.26)$$

$$Y_{\dot{v}} = -1.5 \, m \quad (3.27)$$

$$Z_{\dot{w}} = -1.0 \, m \quad (3.28)$$

$$K_{\dot{p}} = -0.2 \, I_{G(1,1)} \quad (3.29)$$

$$M_{\dot{q}} = -0.8 \, I_{G(2,2)} \quad (3.30)$$

$$N_{\dot{r}} = -1.7 \, I_{G(3,3)} \quad (3.31)$$

$$\mathbf{M}_A = \begin{bmatrix} -X_{\dot{u}} & 0 & 0 & 0 & 0 & 0 \\ 0 & -Y_{\dot{v}} & 0 & 0 & 0 & 0 \\ 0 & 0 & -Z_{\dot{w}} & 0 & 0 & 0 \\ 0 & 0 & 0 & -K_{\dot{p}} & 0 & 0 \\ 0 & 0 & 0 & 0 & -M_{\dot{q}} & 0 \\ 0 & 0 & 0 & 0 & 0 & -N_{\dot{r}} \end{bmatrix} \quad (3.32)$$

Using M_A and v_r from **Eq. 3.12**, the hydrodynamic added Coriolis matrix can be calculated.

$$M_{\text{sym}} = \frac{1}{2}(M_A + M_A^\top) \quad (3.33)$$

$$M_{\text{sym}} = \begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix} \quad (3.34)$$

$$\dot{v}_1 = M_{11} v_{r,1:3} + M_{12} v_{r,4:6} \quad (3.35)$$

$$\dot{v}_2 = M_{21} v_{r,1:3} + M_{22} v_{r,4:6} \quad (3.36)$$

$$\mathbf{C}_A = \begin{bmatrix} \mathbf{0}_{3 \times 3} & -S(\dot{v}_1) \\ -S(\dot{v}_1) & -S(\dot{v}_2) \end{bmatrix} \quad (3.37)$$

where S is the skew symmetric matrix defined by **Eq. 3.14** Finally, we define the system mass and Coriolis matrices:

$$\mathbf{M}_{\text{sys}} = \mathbf{M}_{RB} + \mathbf{M}_A \quad (3.38)$$

$$\mathbf{C}_{\text{sys}} = \mathbf{C}_{RB} + \mathbf{C}_A \quad (3.39)$$

3.8 Hydrostatics

The Hydrostatic restoring coefficients are modeled as:

$$A_w^{\text{pont}} = C_w^{\text{pont}} L B_{\text{pont}} \quad (3.40)$$

$$I_T = 2 \cdot \left(\frac{1}{12} L B_{\text{pont}}^3 \frac{6 C_w^{\text{pont}^3}}{(1 + C_w^{\text{pont}})(1 + 2 C_w^{\text{pont}})} \right) + 2 A_w^{\text{pont}} y_{\text{pont}}^2 \quad (3.41)$$

$$I_L = 0.8 \cdot 2 \left(\frac{1}{12} B_{\text{pont}} L^3 \right) \quad (3.42)$$

$$K_B = \frac{1}{3} \left(\frac{5 T_{\text{draft}}}{2} - \frac{1}{2} \frac{\nabla}{L B_{\text{pont}}} \right) \quad (3.43)$$

$$B M_T = \frac{I_T}{\nabla}, \quad B M_L = \frac{I_L}{\nabla} \quad (3.44)$$

$$K M_T = K_B + B M_T, \quad K M_L = K_B + B M_L \quad (3.45)$$

$$K_G = T_{\text{draft}} - r_g^{\text{corrected}(3)} \quad (3.46)$$

$$G M_T = K M_T - K_G, \quad G M_L = K M_L - K_G \quad (3.47)$$

The hydrostatic matrix in the center of flotation (CF) is then given by:

$$\mathbf{G}_{\text{CF}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & G_{33} & 0 & 0 & 0 \\ 0 & 0 & 0 & G_{44} & 0 & 0 \\ 0 & 0 & 0 & 0 & G_{55} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (3.48)$$

Using the transformation matrix $\mathbf{H}$, the hydrostatic matrix in the CO is given by:

$$\mathbf{G} = \mathbf{H}^T (\mathbf{r}_{\text{LCF}}) \mathbf{G}_{\text{CF}} \mathbf{H} (\mathbf{r}_{\text{LCF}}) \quad (3.49)$$

Further, we can use $\mathbf{G}$ to calculate the Natural damping frequencies:

$$\omega_3 = \sqrt{\frac{G_{33}}{M_{\text{sys}}(3,3)}}, \quad \omega_4 = \sqrt{\frac{G_{44}}{M_{\text{sys}}(4,4)}}, \quad \omega_5 = \sqrt{\frac{G_{55}}{M_{\text{sys}}(5,5)}} \quad (3.50)$$

The Linear damping terms is given by:

$$X_u = -\frac{M_{(1,1)}^{\text{sys}}}{T_{\text{surge}}} \quad (3.51)$$

$$Y_v = -\frac{M_{(2,2)}^{\text{sys}}}{T_{\text{sway}}} \quad (3.52)$$

$$Z_w = -2 \cdot 0.3 \omega_3 M_{(3,3)}^{\text{sys}} \quad (3.53)$$

$$K_p = -2 \cdot 0.2 \omega_4 M_{(4,4)}^{\text{sys}} \quad (3.54)$$

$$M_q = -2 \cdot 0.4 \omega_5 M_{(5,5)}^{\text{sys}} \quad (3.55)$$

$$N_r = -\frac{M_{(6,6)}^{\text{sys}}}{T_{\text{yaw}}} \quad (3.56)$$

We can calculate linear damping using the relative velocities and nonlinear yaw damping as:

$$X_h = X_u v_r(1) \quad (3.57)$$

$$Y_h = Y_v v_r(2) \quad (3.58)$$

$$Z_h = Z_w v_r(3) \quad (3.59)$$

$$K_h = K_p v_r(4) \quad (3.60)$$

$$M_h = M_q v_r(5) \quad (3.61)$$

$$N_h = N_r (1 + 10 |v_r(6)|) v_r(6) \quad (3.62)$$

Finally, we have the tau vector for damping Forces and Moments:

$$\tau_{\text{damp}} = [X_h \ Y_h \ Z_h \ K_h \ M_h \ N_h]^T \quad (3.63)$$

3.9 Cross flow drag

The cross-flow drag principle is used to calculate the nonlinear damping force in sway, heave, pitch and yaw. The calculation begins by dividing the vessel into $N = 20$ longitudinal strips along its length:

$$\Delta x = \frac{L}{20} \quad (3.64)$$

Each strip has a corresponding 2D drag coefficient obtained from Hoerner's empirical curve [51] (Not included here):

$$C_d^{2D} = \text{Hoerner}(B, T) \quad (3.65)$$

At each strip location $x_L \in [-L/2, L/2]$, we compute the local transverse velocities due to sway and yaw:

$$v_{\text{eff}}(x_L) = v_r + x_L r \quad (3.66)$$

And similarly, the vertical velocities due to heave and pitch:

$$w_{\text{eff}}(x_L) = w_r + x_L q \quad (3.67)$$

We then compute the signed square flow velocities:

$$U_h(x_L) = |v_{\text{eff}}(x_L)| v_{\text{eff}}(x_L) \quad (3.68)$$

$$U_v(x_L) = |w_{\text{eff}}(x_L)| w_{\text{eff}}(x_L) \quad (3.69)$$

The sway and heave forces from cross-flow drag are integrated over the vessel:

$$Y_h = - \sum_{x_L} \frac{1}{2} \rho T C_d^{2D} U_h(x_L) \Delta x \quad (3.70)$$

$$Z_h = - \sum_{x_L} \frac{1}{2} \rho T C_d^{2D} U_v(x_L) \Delta x \quad (3.71)$$

The pitch and yaw moments are also integrated over the strips, with moment arm x_L :

$$M_h = - \sum_{x_L} \frac{1}{2} \rho T C_d^{2D} x_L U_v(x_L) \Delta x \quad (3.72)$$

$$N_h = - \sum_{x_L} \frac{1}{2} \rho T C_d^{2D} x_L U_h(x_L) \Delta x \quad (3.73)$$

Finally, we assemble the full cross-flow damping vector:

$$\boldsymbol{\tau}_{\text{crossflow}} = [0 \quad Y_h \quad Z_h \quad 0 \quad M_h \quad N_h] \quad (3.74)$$

3.10 Payload forces and moments

To account for the effects of a payload in the cabin space on the vessel, the model calculates the gravitational force and the resulting moment it generates due to its offset from the vessel's center of gravity. Using the roll and pitch angles from the vessel's pose vector $\boldsymbol{\eta}$:

$$\phi = \eta_4, \quad \theta = \eta_5 \quad (3.75)$$

The gravitational force acting on the payload in the inertial frame, $[0, 0, m_p g]^T$, can be transformed into the body-fixed frame:

$$\mathbf{f}_{\text{payload}} = \mathbf{R}_{zyx}^T(\phi, \theta, \psi) \begin{bmatrix} 0 \\ 0 \\ m_p g \end{bmatrix} \quad (3.76)$$

where $\mathbf{R}_{zyx}$ is the rotation matrix from the body-fixed frame to the inertial frame using the ZYX Euler angle convention, defined as:

$$\begin{aligned} c_\phi &= \cos(\phi), & s_\phi &= \sin(\phi), \\ c_\theta &= \cos(\theta), & s_\theta &= \sin(\theta), \\ c_\psi &= \cos(\psi), & s_\psi &= \sin(\psi) \end{aligned}$$

$$\mathbf{R}_{zyx} = \begin{bmatrix} c_\psi c_\theta & -s_\psi c_\phi + c_\psi s_\theta s_\phi & s_\psi s_\phi + c_\psi c_\phi s_\theta \\ s_\psi c_\theta & c_\psi c_\phi + s_\psi s_\theta s_\phi & -c_\psi s_\phi + s_\psi c_\phi s_\theta \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix} \quad (3.77)$$

The moment generated by the f_{payload} force due to the payload's position vector $\mathbf{r}_p$, is then calculated using the skew-symmetric cross product matrix:

$$\mathbf{m}_{\text{payload}} = \mathbf{S}(\mathbf{r}_p)\mathbf{f}_{\text{payload}} \quad (3.78)$$

Finally, the total generalized force vector from the payload, combining both force and moment, is given by:

$$\mathbf{g}_0 = \begin{bmatrix} \mathbf{f}_{\text{payload}} \\ \mathbf{m}_{\text{payload}} \end{bmatrix} \quad (3.79)$$

3.11 Trim condition

The trim condition represents the equilibrium state of the vessel in the absence of external inputs. If a payload is added off-center in the vessels cabin space, it introduces steady restoring forces and moments, shifting the equilibrium configuration. The model takes this into account by computing a trim vector $\boldsymbol{\eta}_0$ to offset this static deflection. First, we extract the relevant 3×3 block from the hydrostatic stiffness matrix $\mathbf{G}$, corresponding to the heave, roll, and pitch directions:

$$\mathbf{G}_{\text{sub}} = \mathbf{G}_{3:5, 3:5} \quad (3.80)$$

Next, we extract the corresponding gravitational force and moment vector entries due to the payload:

$$\mathbf{g}_{0,\text{sub}} = \begin{bmatrix} g_0^{(3)} \\ g_0^{(4)} \\ g_0^{(5)} \end{bmatrix} \quad (3.81)$$

To find the displacement and rotation required to balance these effects, we solve the linear system:

$$\boldsymbol{\eta}_0^{\text{sub}} = \mathbf{G}_{\text{sub}}^{-1} \mathbf{g}_{0,\text{sub}} \quad (3.82)$$

We then insert the result into a full six-DOF vector, setting the other components to zero:

$$\boldsymbol{\eta}_0 = \begin{bmatrix} 0 \\ 0 \\ \eta_0^{(3)} \\ \eta_0^{(4)} \\ \eta_0^{(5)} \\ 0 \end{bmatrix} \quad (3.83)$$

Finally, we subtract this trim offset from the vessel's state vector to shift the reference frame to the new equilibrium:

$$\boldsymbol{\eta} := \boldsymbol{\eta} - \boldsymbol{\eta}_0 \quad (3.84)$$

This adjustment ensures that the vessel model accounts for the payload's effect.

3.12 Azimuth Pod modeling

The vessel model needs to calculate the forces in surge and sway, and the yaw moment given by the current propeller speeds n and azimuth angles α . For this purpose the model uses relative propeller speeds

$$n \in [-1, 1] \quad (3.85)$$

where -1 corresponds to max negative bollard pull, and 1 corresponds to max positive bollard pull. This makes interpretation easier and can improve convergence for numerical solvers due to better numerical stability. In addition, the pod's azimuths are constrained to the interval

$$\alpha \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right] \quad (3.86)$$

An approximation of the thrust T given by the relative propeller speed n for pod i can be modeled as:

$$T_i(n_i) = \begin{cases} k_{\text{pos}} n_i |n_i|, & n_i \geq 0, \\ k_{\text{neg}} n_i |n_i|, & n_i < 0, \end{cases} \quad i = l, r \quad (3.87)$$

where k_{pos} is the maximum bollard pull for positive propeller speed and k_{neg} is the maximum bollard pull for negative propeller speed. The propeller and The azimuth pod setup is illustrated by **Figure 3.2**.

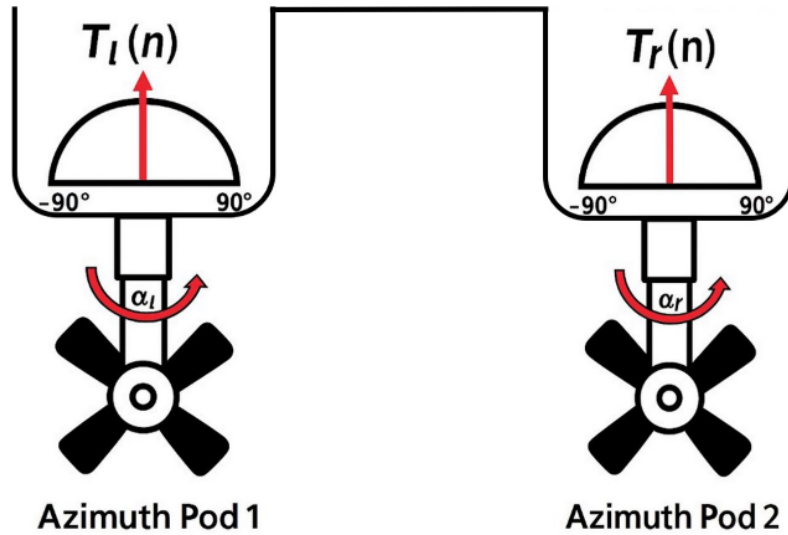


Figure 3.2: Illustrative picture of the azimuth pod setup on RAN, seen from behind in a 2D plane. $T(n)$ is the thrust given from propeller speed n . α is the azimuth angle of the pod, deciding thrust direction. α is physically constrained to an interval of $\pm 90^\circ$. The picture was made using *DALL-E*.

The thrust values, lever arms, and angles of the pod's are used to find the surge (X) and sway (Y) forces and yaw (N) moment in the vessels CO. In the following formulations we simplify by assuming perfect water conditions around the hull and that the pods do not interfere with each others slipstream.

Control of surge force is maximum when the pods are pointing straight forwards, defined as 0° , or straight backwards, defined as $\pm 180^\circ$. This is because the propeller is perfectly aligned with the x-axis in the BODY frame. It is 0 when the pods point perfectly sideways, defined as $\pm 90^\circ$, because the propeller is perfectly aligned with the y axis in the BODY frame. The total surge force τ_X given by the pod on the left (Port side) and the pod on the right (Starboard) can then be calculated using:

$$\tau_X = T_l \cos(\alpha_l) + T_r \cos(\alpha_r); \quad (3.88)$$

The same trigonometric logic used to find τ_X can be applied to find the total sway force τ_Y expression:

$$\tau_Y = T_l \sin(\alpha_l) + T_r \sin(\alpha_r); \quad (3.89)$$

To calculate the yaw moment τ_N , we decompose the thrust from each pod into its x- and y-components using $\cos(\alpha)$ and $\sin(\alpha)$. Each of these components produces a torque around the CO, depending on the lever arms to the CO. The lateral components of the thrusts (pointing in the y-direction) create torque through their longitudinal offsets l_{x1} (left thruster) and l_{x2} (right thruster). Given that $l_{x1} = l_{x2}$, we can write total yaw moment from the x component of the lever arms as:

$$\tau_{N_x} = l_x (T_l \sin(\alpha_l) + T_r \sin(\alpha_r))$$

The longitudinal components (pointing in the x-direction) create torque through their lateral offsets l_{y1} (left thruster) and l_{y2} (right thruster). The total y component of the lever arms can then be modeled as

$$\tau_{N_y} = -(l_{y1} T_l \cos(\alpha_l) + l_{y2} T_r \cos(\alpha_r))$$

The minus sign appears because a forward thrust on the port side (positive y) causes a negative yaw (clockwise), according to the right-hand rule applied to the vessel's coordinate system. Combining the longitudinal and lateral components, the total yaw moment can be modeled as:

$$\tau_N = \tau_{N_x} + \tau_{N_y} \quad (3.90)$$

Finally, the forces and moment from the control inputs can be combined into one vector τ .

$$\tau = [\tau_X, \tau_Y, \tau_N]^T \quad (3.91)$$

3.13 Kinematic transformation for position and state derivative update

To update the position of the marine vehicle, we first need to perform a kinematic transformation from the body frame to the inertial frame. This transformation is

done using the 6x6 matrix J :

$$J = \begin{bmatrix} R_{zyx}(\phi, \theta, \psi) & 0_{3 \times 3} \\ 0_{3 \times 3} & T_{zyx}(\phi, \theta) \end{bmatrix} \quad (3.92)$$

where R_{zyx} is the rotation matrix defined in equation 3.77 and T_{zyx} is the transformation matrix mapping the angular velocities in the body frame to the inertial frame.

$$T_{zyx} = \begin{bmatrix} 1 & \sin(\phi) \tan(\theta) & \cos(\phi) \tan(\theta) \\ 0 & \cos(\phi) & -\sin(\phi) \\ 0 & \frac{\sin(\phi)}{\cos(\theta)} & \frac{\cos(\phi)}{\cos(\theta)} \end{bmatrix} \quad (3.93)$$

To compute the state derivative $\dot{\mathbf{x}}$, we need to sum the various forces and moments acting on the system. These include the forces $\boldsymbol{\tau}$ from the control inputs given by **Eq. 3.91**, damping forces $\boldsymbol{\tau}_{\text{damp}}$ given by **Eq. 3.63**, cross-flow drag forces $\boldsymbol{\tau}_{\text{crossflow}}$ given by **Eq. 3.74**, as well as the Coriolis forces C_{sys} described by **Eq. 3.39** and gravitational effects G acting on the system. The total force term is calculated as:

$$\mathbf{F} = \boldsymbol{\tau} + \boldsymbol{\tau}_{\text{damp}} + \boldsymbol{\tau}_{\text{crossflow}} - C_{\text{sys}} \boldsymbol{v}_r - G \boldsymbol{\eta} \quad (3.94)$$

The acceleration is computed by solving the equation of motion using the system's mass matrix M_{sys} .

$$\dot{\mathbf{v}} = M_{\text{sys}}^{-1} \mathbf{F} \quad (3.95)$$

where $\mathbf{v}$ is the velocity vector and $\mathbf{F}$ is the total force vector. Next, we compute the velocity derivative and the position derivative. The velocity derivative is the sum of the velocity update $\boldsymbol{v}_c$ and the computed acceleration:

$$\dot{\mathbf{v}}_{\text{vel}} = \boldsymbol{v}_c + \dot{\mathbf{v}} \quad (3.96)$$

For the position update, we use the kinematic transformation matrix J to map the velocity to the inertial frame:

$$\dot{\mathbf{x}}_{\text{pos}} = J \boldsymbol{v}_{1:6} \quad (3.97)$$

Finally, we combine the velocity and position derivatives into the full 12 dimensional state derivative:

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{v}}_{\text{vel}} \\ \dot{\mathbf{x}}_{\text{pos}} \end{bmatrix} \quad (3.98)$$

This complete state derivative can be used by a numerical integration method, such as Runge Kutta 4, to update the system's state $\mathbf{x}$.

3.14 Vessel State Propagation using RK4 Integration

The Runge-Kutta 4th order method (RK4) is an iterative technique for numerically solving ordinary differential equations (ODEs). It approximates the solution of a differential equation by taking the weighted average of four estimates of the state

at each time step. The RK4 method is used to propagate the state vector defined by Eq. 3.7 of the marine vessel model in discrete time steps.

Given the state vector $\mathbf{x}$, the parameters m_p (payload mass), V_c (velocity), β_c (control input), h (time step), n (propeller speed), and α (azimuth angle), the the state derivative $\dot{\mathbf{x}}$, defined in Eq. 3.98, can be computed. The RK4 method estimates the state evolution over the interval h by computing a weighted average of four derivative evaluations:

$$\mathbf{k}_1 = h \cdot \dot{\mathbf{x}}(x, m_p, V_c, \beta_c, h, n, \alpha) \quad (3.99)$$

$$\mathbf{k}_2 = h \cdot \dot{\mathbf{x}}\left(x + \frac{1}{2}k_1, m_p, V_c, \beta_c, h, n, \alpha\right) \quad (3.100)$$

$$\mathbf{k}_3 = h \cdot \dot{\mathbf{x}}\left(x + \frac{1}{2}k_2, m_p, V_c, \beta_c, h, n, \alpha\right) \quad (3.101)$$

$$\mathbf{k}_4 = h \cdot \dot{\mathbf{x}}(x + k_3, m_p, V_c, \beta_c, h, n, \alpha) \quad (3.102)$$

These intermediate values approximate the incremental change in state across the time step. The final updated state is given by:

$$\mathbf{x}_{\text{new}} = \mathbf{x} + \frac{1}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \quad (3.103)$$

3.15 Input matrix (B)

To enable pseudo-inverse control allocation, each thruster can be decomposed into its surge and sway components:

$$u = \begin{bmatrix} F_1 \cos \alpha_1 \\ F_1 \sin \alpha_1 \\ F_2 \cos \alpha_2 \\ F_2 \sin \alpha_2 \end{bmatrix}, \quad (3.104)$$

with

$$F_i = \begin{cases} k_{\text{neg}}, & n_i < 0, \\ k_{\text{pos}}, & n_i \geq 0, \end{cases} \quad (i = 1, 2). \quad (3.105)$$

The resulting 3×4 input matrix B maps these actuator components to the vessel's surge force, sway force, and yaw moment:

$$\mathbf{B} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ -l_{y1} \cos \alpha_1 & l_x \sin \alpha_1 & -l_{y2} \cos \alpha_2 & l_x \sin \alpha_2 \end{bmatrix}. \quad (3.106)$$

Such that the generalized force vector is given by

$$\tau = \mathbf{B} u \quad (3.107)$$

3.16 Pod State Propagation using Euler Integration

The dynamics of the propeller speeds and azimuth angles is assumed to be a first-order process. Hence, the pod's states n and α can be updated using Euler integration. Euler integration is a simple first-order numerical method that advances the state by adding the product of its time derivative and the time step to the current value, providing an approximate solution to the underlying differential equations

The propeller speed dynamics of each pod can then be modeled as:

$$n_{k+1} = n + \frac{h}{T_n} (n_c - n), \quad n \in [n_{\min} = -1, n_{\max} = 1] \quad (3.108)$$

where h is the sampling time, T_n is the propeller speed time constant and $n_{\min} \leq n \leq n_{\max}$ is enforced after the update step.

Similarly, the azimuth angle dynamics of each pod can be modeled by:

$$\alpha_{k+1} = \alpha + \frac{h}{T_\alpha} (\alpha_c - \alpha), \quad \alpha \in [\alpha_{\min} = -\frac{\pi}{2}, \alpha_{\max} = \frac{\pi}{2}] \quad (3.109)$$

where h is the sampling time, T_α is the angle time constant and $\alpha_{\min} \leq \alpha \leq \alpha_{\max}$ is enforced after the update step.

Chapter 4

Control Allocation Methods and Implementation

In this chapter, the control systems developed for RAN based on selected methodologies reviewed in **Chapter 2** and the vessel model described in **Chapter 3** are presented. This includes motion controllers and control allocation methods developed for path following and DP applications. Motion control is needed to compute the virtual control input τ_c that the developed control allocation methods can use to compute propeller speeds and azimuth angles. One pseudo inverse based and two optimization based control allocation methods are developed that use motion controllers. Additionally, an MPC capturing the whole control system has been developed for more demanding DP and berthing maneuvers. The developed methods controls the DOF surge, sway and yaw.

4.1 Deciding on Control Allocation

In **Chapter 2** many different methods for allocation were reviewed, and some are better qualified to meet the needs of the control allocation problem at hand.

Azimuth Pod Constraints The azimuth pods on RAN has both bound and rate constraints. Most of the reviewed methods can handle bounds, but not all can handle rate constraints. Being able to handle the rate constraints may have considerable effect, since the 4 DOF that is available to control the 3-DOF surge, sway and yaw is not independent because of time delays. In addition, to avoid wear and tear, the allocation should preferably not change considerably from one timestep to the next. Furthermore, the allocation method should avoid configurations where the slipstream from one pod is directed at the other pods propeller disc, reducing it's performance.

Convexity The piecewise-quadratic thrust mapping $T(n)$ in Eq. 3.87 and the force decomposition in Eq. 3.91,

$$F: (n, \alpha) \mapsto \begin{bmatrix} T(n) \cos \alpha \\ T(n) \sin \alpha \end{bmatrix},$$

is inherently nonlinear, and hence the inverse from desired forces back to (n, α) is non-convex. In particular, there exist two admissible command pairs (n_1, α_1) , (n_2, α_2) and a weight $\lambda \in (0, 1)$ for which the convex combination

$$(n_\lambda, \alpha_\lambda) = \lambda(n_1, \alpha_1) + (1 - \lambda)(n_2, \alpha_2)$$

fails to satisfy the convexity condition outlined in Chapter 2.1.3,

$$F(n_\lambda, \alpha_\lambda) \not\leq \lambda F(n_1, \alpha_1) + (1 - \lambda)F(n_2, \alpha_2),$$

Consequently, the overall allocation problem is non-convex.

Decision To handle non-convexity and the pod's bounds and rate constraints, the problem can be formulated as a nonlinear optimization problem, and solved with a nonlinear numerical solver. The open-source nonlinear solver IPOPT can be used, given that the objective and constraint functions are smooth and have available derivatives. Furthermore, to explicitly handle rate constraints and plan ahead in time, an MPC framework can be used. Should the chosen numerical solver not converge, a fallback method not reliant on numerical solvers should be considered. For this purpose, a pseudo-inverse-based approach can work. It might not guarantee feasible solutions, but it does guarantee a solution. In addition, taking the pseudo-inverse demands minimal compute, such that a solution can be generated quickly should the numerical solver fail or converge too slowly.

4.2 Motion Control

A Nomoto Heading PID motion controller was developed for path following and a MIMO PID motion controller was developed for DP.

4.2.1 Nomoto Heading PID

The Nomoto Heading PID controller described in [1] was implemented to compute the required yaw moment τ_N , while using a constant positive surge force $\tau_X > 0$ and zero sway force $\tau_Y = 0$. The controller is based on the first-order Nomoto model for yaw dynamics:

$$T \dot{r}(t) + r(t) = K \tau_N(t), \quad (4.1)$$

where $r = \dot{\psi}$ is the yaw rate, T the Nomoto time constant, K the model gain, and τ_N the control moment. Inverting this model gives a feedforward term which is augmented by a PID action on the heading error.

Let $I_\psi = M_{5,5}$ be the effective yaw inertia from the mass–damping matrix M . Then the nominal gain is

$$K = \frac{T}{I_\psi}. \quad (4.2)$$

We select PID gains to achieve a second-order closed-loop behavior with natural frequency ω_n and damping ratio ζ :

$$K_p = I_\psi \omega_n^2, \quad K_d = I_\psi \left(2\zeta \omega_n - \frac{1}{T} \right), \quad T_d = \frac{K_d}{K_p}, \quad T_i = \frac{10}{\omega_n}. \quad (4.3)$$

At each control step of duration h , the heading error is wrapped to $(-\pi, \pi]$:

$$e_\psi = \text{ssa}(\psi - \psi_d), \quad z_\psi \leftarrow z_\psi + h e_\psi, \quad (4.4)$$

where z_ψ is the accumulated integral of e_ψ . The desired yaw moment is then

$$\tau_N = \underbrace{\frac{T}{K} a_d + \frac{1}{K} r_d}_{\text{feedforward}} - K_p \left[e_\psi + T_d (r - r_d) + \frac{1}{T_i} z_\psi \right]. \quad (4.5)$$

Here $a_d = \ddot{\psi}_d$ and $r_d = \dot{\psi}_d$ are the desired yaw acceleration and rate. The feed-forward term compensates for the Nomoto dynamics, while the PID portion drives the heading error to zero.

4.2.2 Nonlinear MIMO PID

For DP applications, where complex maneuvering in any direction can be desired, a motion controller based on the MIMO PID described in [1] was implemented to calculate desired forces and moments. The controller computes the 3-DOF control vector $\tau = [\tau_X, \tau_Y, \tau_N]^T$ via a cascaded position–velocity structure.

At each step of size h , given desired north-east positions $(x_{n,d}, y_{n,d})$, heading ψ_d , current pose η , body velocities $v = [u, v, r]^T$, and ocean current (V_c, β_c) , the position error in the body frame and the heading error are calculated:

$$\Delta X_{\text{NED}} = \begin{bmatrix} x_{n,d} - x_n \\ y_{n,d} - y_n \end{bmatrix}, \quad e_\psi = \text{ssa}(\psi_d - \psi) \quad (4.6)$$

$$\begin{bmatrix} e_x \\ e_y \end{bmatrix} = \begin{bmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{bmatrix} \Delta X_{\text{NED}} \quad (4.7)$$

A proportional outer-loop then converts these errors to desired body-frame velocities:

$$v_d = \begin{bmatrix} u_d \\ v_d \\ r_d \end{bmatrix} = \begin{bmatrix} K_{px} e_x \\ K_{py} e_y \\ K_{p\psi} e_\psi \end{bmatrix} \quad (4.8)$$

The ocean current is represented in the NED-frame and rotated into the BODY-frame:

$$V_{c,\text{NED}} = \begin{bmatrix} V_c \cos \beta_c \\ V_c \sin \beta_c \end{bmatrix}, \quad V_{c,\text{body}} = \begin{bmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{bmatrix} V_{c,\text{NED}}, \quad (4.9)$$

and the relative velocity becomes

$$\mathbf{v}_r = \begin{bmatrix} u - V_{c,\text{body}}(0) \\ v - V_{c,\text{body}}(1) \\ r \end{bmatrix}. \quad (4.10)$$

The inner-loop velocity error is then

$$\mathbf{v}_e = \mathbf{v}_d - \mathbf{v}_r. \quad (4.11)$$

We extract the 3×3 sub-mass matrix M_3 from the full inertia-damping matrix M as

$$M_3 = \begin{bmatrix} M_{0,0} & M_{0,1} & M_{0,5} \\ M_{1,0} & M_{1,1} & M_{1,5} \\ M_{5,0} & M_{5,1} & M_{5,5} \end{bmatrix}. \quad (4.12)$$

Choose design parameters

$$\Omega_b = \text{diag}(\omega_{b,1}, \omega_{b,2}, \omega_{b,3}), \quad Z = \text{diag}(\zeta_1, \zeta_2, \zeta_3), \quad (4.13)$$

and compute the natural-frequency matrix

$$\Omega_n = \text{diag}(\omega_{n,1}, \omega_{n,2}, \omega_{n,3}), \quad \omega_{n,i} = \frac{\omega_{b,i}}{\sqrt{1 - 2\zeta_i^2 + \sqrt{4\zeta_i^4 - 4\zeta_i^2 + 2}}}. \quad (4.14)$$

Then form the PID gain matrices:

$$K_p = M_3 \Omega_n^2, \quad K_d = 2 M_3 Z \Omega_n, \quad K_i = \frac{K_p \Omega_n}{10}. \quad (4.15)$$

To compute the derivative and integral terms with filtering and anti-windup, let $\mathbf{v}_e^-$, $\dot{\mathbf{v}}_f^-$, $\mathbf{z}^-$ denote their values at the previous step. Then

$$\dot{\mathbf{v}}_f = \alpha \frac{\mathbf{v}_e - \mathbf{v}_e^-}{h} + (1 - \alpha) \dot{\mathbf{v}}_f^-, \quad \mathbf{z} = \text{clamp}(\mathbf{z}^- + h \mathbf{v}_e, -\mathbf{z}_{\max}, \mathbf{z}_{\max}). \quad (4.16)$$

Finally, the MIMO control output is computed as

$$\boldsymbol{\tau} = K_p \mathbf{v}_e + K_d \dot{\mathbf{v}}_f + K_i \mathbf{z}. \quad (4.17)$$

4.3 Control Allocation: Pseudo Inverse

At higher vessel speeds, the distance traveled during a timestep h between allocation updates increases, which may degrade path-following performance if allocation is not computed rapidly enough. As a simple and fast fallback and as a benchmark against other allocation methods, a pseudo-inverse approach can be employed. As discussed in **Chapter 2**, pseudo-inverse methods often involve complex constraint handling, since constraints are not naively satisfied by the pseudo-inverse.

The implementation outlined in **Algorithm 3** computes least-squares actuator forces via a pseudo-inverse and then converts them into individual thrust magnitudes and azimuth angles using a mapping that ensures the azimuth angles remain within $\pm 90^\circ$ bound. If any computed propeller speeds violate the constraints $n_i \in [-1, 1]$, the virtual control input τ is scaled accordingly and the allocation is recomputed. For sufficiently small forces, no violation occurs and the algorithm terminates. The extended allocation model uses the input matrix

$$B \in \mathbb{R}^{3 \times 4}, \quad \tau = \begin{bmatrix} \tau_X \\ \tau_Y \\ \tau_N \end{bmatrix} \in \mathbb{R}^3,$$

and the maximum bollard pull coefficients k_{pos} and k_{neg} for forward and reverse operation, respectively. The pseudo-inverse produces an intermediate force vector

$$u_{\text{ext}} = \{F_{1x}, F_{1y}, F_{2x}, F_{2y}\},$$

which is then mapped into the relative propeller speeds n_i and azimuth angles α_i . The azimuth angle is initially mapped to

$$\alpha_i^{\text{raw}} \leftarrow \text{atan2}(u_{iy}, u_{ix}) \quad (4.18)$$

But to make sure the constraint on $\pm 90^\circ$ is not violated, the angle is flipped if necessary. If the angle is flipped, it's compensated for by changing the sign of the propeller speed direction s_i .

$$(\alpha_i, s_i) = \begin{cases} (\alpha_i^{\text{raw}} - \pi, -1), & \alpha_i^{\text{raw}} > \frac{\pi}{2}, \\ (\alpha_i^{\text{raw}} + \pi, -1), & \alpha_i^{\text{raw}} < -\frac{\pi}{2}, \\ (\alpha_i^{\text{raw}}, +1), & \text{otherwise.} \end{cases} \quad (4.19)$$

Each n_i satisfies the nonlinear thrust relationship

$$F_i = k_i n_i |n_i| \quad (4.20)$$

where

$$k_i = k_{\text{pos}} \quad \text{if } s_i = +1, \quad (4.21)$$

$$k_i = k_{\text{neg}} \quad \text{if } s_i = -1. \quad (4.22)$$

Solving for n_i yields

$$|n_i| = \sqrt{\frac{F_i}{k_i}} \quad (4.23)$$

where

$$F_i = \sqrt{u_{ix}^2 + u_{iy}^2} \quad (4.24)$$

Applying the sign given by flipping the azimuth, the normalized thrust is given by

$$n_i = s_i \sqrt{\frac{F_i}{k_i}} \quad (4.25)$$

The calculated propeller speeds n_1 and n_2 may lie outside the allowed interval $[-1, 1]$. In that case, all elements of τ is scaled equally by

$$\frac{1}{\max(|n_1|, |n_2|)} \quad (4.26)$$

and the allocation is recomputed. Finally, the resulting propeller speeds and azimuth angles

$$\{n_1, \alpha_1, n_2, \alpha_2\}$$

are returned. A possible backside to the presented constraint handling is that the output can have large jumps, due to the sign switch, when the azimuth angle values pass back and forth over their maximum and minimum values.

4.4 Control Allocation: Nonlinear Optimization

One of the many advantages of optimization over pseudo-inverse-based methods, as seen in **Chapter 2**, is that they can handle constraints directly, thereby avoiding extra logic and post-processing of the results. The nonlinear optimization framework Opti from the CasADi library was used to implement an optimization problem that minimize the difference between the desired and the commanded forces (surge and sway) and moment (yaw) for a given timestep h , capable of constraint handling. The propeller speed and azimuth angle bounds are modeled as hard constraints. Soft constraints are implemented to avoid values that cause loss of surge control and thrust to enter another pod's slipstream by penalizing them in the cost function. This formulation does not model rate constraints on propeller speed and azimuth angle directly, but there are soft penalties on large changes in commanded control inputs. To obtain better numerical stability and easier tuning, relative propeller speeds $n \in [-1, 1]$ are used, where -1 represents the maximum reverse thrust and 1 represents the maximum forward thrust. Because the objectives and constraint functions are smooth and have available derivatives, the numerical solver IPOPT can be used. The allocation method is described by **Algorithm 4**.

The main objective of minimizing τ is done using the quadratic formulation

$$J_\omega = (\tau_X - \hat{\tau}_X)^2 + (\tau_Y - \hat{\tau}_Y)^2 + (\tau_N - \hat{\tau}_N)^2 \quad (4.27)$$

where τ is the desired forces (X, Y) and moment (N), and $\hat{\tau}$ is the forces and moment given by the optimization parameters n_i and α_i . The calculation of $\hat{\tau}$ from the control inputs is based on the thruster mapping and the lever arms of the azimuth pods relative to the center of origin (CO), as expressed in **Eq. 3.91**, with full details provided in Chapter 3.12.

To avoid configurations where one pod is directing thrust into another pod's slipstream, degrading its performance, such configurations are penalized in the cost function. Instead of using barrier-functions that go towards infinity at their forbidden boundary, we use exponential soft penalties, that can be weighted in the cost function, for better numerical stability. For pod 1 (port-side) we use

$$f_1 = \exp\left(-\frac{(\alpha_1 + \frac{\pi}{2})^2}{0.2}\right) \quad (4.28)$$

that adds a exponentially rising cost when α approaches -90 degrees. For pod 2 (starboard-side) we use

$$f_2 = \exp\left(-\frac{(\alpha_2 - \frac{\pi}{2})^2}{0.2}\right) \quad (4.29)$$

that adds a exponentially rising cost when α approaches 90 degrees. Together, they form the objective function for avoiding slip-stream disturbances,

$$J_{\text{slipstream}} = f_1 + f_2 \quad (4.30)$$

The objective functions f_1 and f_2 are visualized in **Figure 4.1** to illustrate their behavior.

If both pod's are rotated by $\pm 90^\circ$, surge control is lost, since no combination of n_1 and n_2 can control surge motion in such configurations. Using a soft exponential penalty similar to **Eq.'s 4.28 - 4.29** with the absolute value of each alpha minus 90 degrees inside the exponential expressions, azimuth angle configurations of plus and minus 90 degrees are penalized for both pods. For penalizing only when both pods approach $\pm 90^\circ$ at the same time, the exponential penalties are combined by multiplication.

$$J_{\text{surge-loss}}(\alpha_1, \alpha_2) = \exp\left(-\frac{(|\alpha_1| - \frac{\pi}{2})^2 + (|\alpha_2| - \frac{\pi}{2})^2}{0.1}\right) \quad (4.31)$$

To minimize unnecessary energy usage, wear and tear, changes in control inputs is penalized. This is done using the difference between the current and the commanded control inputs using the quadratic formulations:

$$J_n = (n_1^{\text{cmd}} - n_1)^2 + (n_2^{\text{cmd}} - n_2)^2 \quad (4.32)$$

$$J_\alpha = (\alpha_1^{\text{cmd}} - \alpha_1)^2 + (\alpha_2^{\text{cmd}} - \alpha_2)^2 \quad (4.33)$$

To balance the objectives, each of them is multiplied by a positive weighting constant ω . Finally, the optimization problem can be solved with a nonlinear solver.

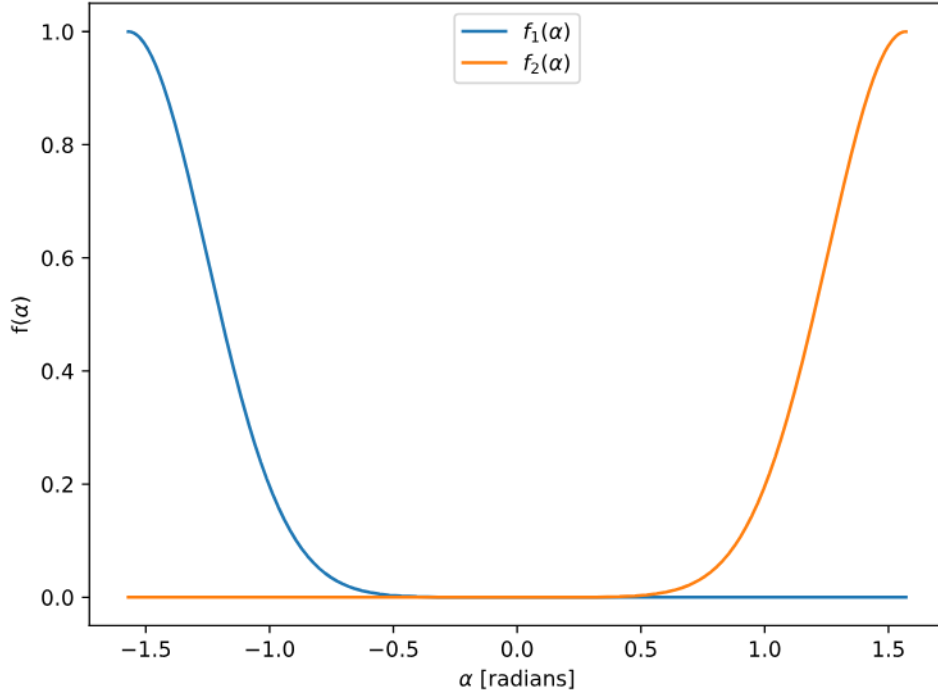


Figure 4.1: Illustration of the soft exponential penalty function f_1 in Eq. 4.28 and f_2 Eq. 4.28. Penalties for directing thrust into another propellers slipstream

4.5 Control Allocation: MPC

In order to explicitly take propeller speed and azimuth angle rate constraints into account, we can expand upon **Algorithm 4**, by formulating it as an MPC problem with a model of the pod dynamics as described by Eq.'s 3.108 - 3.109. The new formulation, described by **Algorithm 5**, optimizes the commanded control with step size Δ over a horizon N . It returns the optimized commanded control input values from the first step in the horizon.

4.6 Control System: MPC for DP and Berthing

A MPC control system was developed with the goal of enabling RAN to do complex and demanding DP and berthing tasks. **Algorithm 3, 4 and 5**, all assume that the desired τ is given by a motion controller, in this case by the MIMO PID controller described in **Chapter 4.2.2** for DP. However, the PID only cares about the current situation and does not plan maneuvers forward in time. This shortsightedness can limit the vessel's ability to handle complex and demanding maneuvers. Controlling surge sway and yaw, the USV RAN, having 4 DOF, is overactuated, but not to the same extent as a vessel with two sets of two independent, perpendicu-

lar, fixed thruster devices, due to time delays. Adding constraint handling worsens the problem as the control space gets limited. Hence, it is not obvious that RAN is able to fully control all these DOF, such that it can follow a path while keeping a specified heading.

The developed MPC control system described by **Algorithm 6** casts the motion control and control allocation tasks as a finite-horizon optimal control problem, which assumes the desired path is a straight line between a start and a goal waypoint. The decision variables are the state trajectory X_k , the current pod commands (n_k, α_k) , and their commanded updates $(n_k^{\text{cmd}}, \alpha_k^{\text{cmd}})$. The cost aggregates the weighted absolute values of the cross-track error, the relative remaining distance of the vessel's projection on the line between waypoints to the goal, the heading error, changes in commanded propeller speeds and azimuth angles, and large speeds. The goal is to minimize the position and heading errors while also weighting in which manner the vessel achieves this minimization. The penalty on high speed is there to ensure that the vessel does not move too fast compared to the computational time of the MPC, such that performance degradation is avoided. The vessel dynamics combine kinematics and a lumped-parameter model of surge, sway and yaw, integrated with a fourth-order Runge–Kutta (RK4) step. The model for surge, sway and yaw generated by the pods and the constraints that enforce actuator bounds and first-order rate-of-change dynamics are the same ones used in **Algorithm 5**. The resulting nonlinear program is solved at each time step, and only the first control update is applied in a receding-horizon fashion. The method is clearly described by **Algorithm 6**.

To find the distance from the projection of the vessel, on the line between the start and goal waypoints, to the goal, first the x and y components of the distance from the goal to the vessel are found.

$$X_{\text{dist}} = x_g - X(0) \quad (4.34)$$

$$Y_{\text{dist}} = y_g - X(1) \quad (4.35)$$

Further, we need to find the x and y components of the path direction

$$P_{\text{error},x} = x_g - x_s \quad (4.36)$$

$$P_{\text{error},y} = y_g - y_s \quad (4.37)$$

$$L = \sqrt{P_{\text{error},x}^2 + P_{\text{error},y}^2} \quad (4.38)$$

$$X_{\text{dir}} = \frac{P_{\text{error},x}^2}{L} \quad (4.39)$$

$$Y_{\text{dir}} = \frac{P_{\text{error},y}^2}{L} \quad (4.40)$$

Finally,

$$D_{\text{proj}} = X_{\text{dist}}X_{\text{dir}} + Y_{\text{dist}}Y_{\text{dir}} \quad (4.41)$$

For easier tuning and numerical stability, we normalize the projected distance error such that the distance from the start to the goal position is 1.

$$D_{\text{proj, norm}} = \frac{D_{\text{proj}}}{L} \quad (4.42)$$

To avoid negative cost from the Norm, which can happen if the vessel goes past the goal, we can use it's absolute value.

$$|D_{\text{proj, norm}}| = \sqrt{D_{\text{proj, norm}}^2} \quad (4.43)$$

Using **Eq's 4.34-4.37** and **Eq. 4.41** the cross-track error y_e for the vessel relative to it's projection on the line between the start and goal positions can be computed.

$$y_{e,x} = X_{\text{dist}} - D_{\text{proj}} P_{\text{error},x} \quad (4.44)$$

$$y_{e,y} = Y_{\text{dist}} - D_{\text{proj}} P_{\text{error},y} \quad (4.45)$$

$$y_e = \sqrt{y_{e,x}^2 + y_{e,y}^2} \quad (4.46)$$

The heading error can be found using the difference between the desired and the current heading. To avoid negative cost, the absolute value of the difference can be used.

$$|\psi_{\text{error}}| = \sqrt{(X(2) - \psi_d)^2} \quad (4.47)$$

To avoid the possibility of negative cost terms, the square root is used to calculate the absolute value of the projected distance, cross-track error and heading error. The graph resulting from taking the square root of real valued functions, such as these, is not smooth, due to a jump in the first-derivative, such that the numerical solver IPOPT can't be used.

$$|a| = \sqrt{a^2}, \quad \frac{d}{da}|a| = \frac{a}{\sqrt{a^2}} = \text{sgn}(a) \quad (4.48)$$

To avoid this jump, a small value ε can be added to smooth out the derivatives.

$$|\hat{a}| = \sqrt{a^2 + \varepsilon}, \quad \frac{d}{da}|\hat{a}| = \frac{a}{\sqrt{a^2 + \varepsilon}}, \quad \varepsilon = 10^{-4} \quad (4.49)$$

The effect of using the approximation $|\hat{a}|$, compared to using $|a|$ directly, is illustrated in **Figure 4.2**

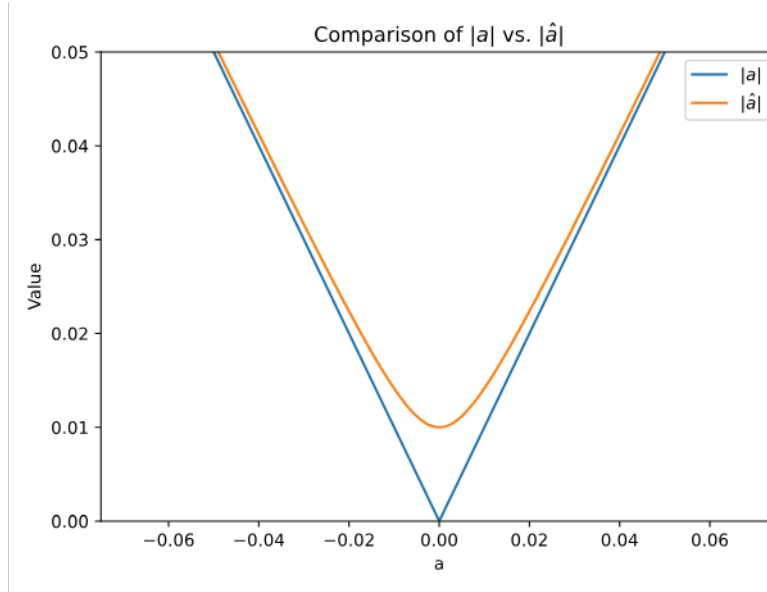


Figure 4.2: Comparing $\sqrt{a^2}$ to $\sqrt{a^2 + \varepsilon}$ shows how adding a small $\varepsilon > 0$ removes the nondifferentiable corner at $a = 0$. The smoothed curve has continuous derivatives, which enables gradient-based solvers like IPOPT to find a solution reliably.

The RK4 step for propagating the state in **Algorithm 6** uses simplified model dynamics based on **Chapter 3**. We consider the continuous-time ODE

$$\dot{X} = f(X, \tau, V_c, \beta_c), \quad (4.50)$$

where V_c and β_c are the ocean current speed and direction in the NED frame, and the state X and forces and moment τ are defined as:

$$X = [x \ y \ \psi \ u \ v \ r]^T, \quad \tau = [\tau_X \ \tau_Y \ \tau_N]^T. \quad (4.51)$$

Over each time step Δt the classical fourth-order Runge–Kutta update reads

$$\begin{aligned} k_1 &= f(X_k, \tau_k, V_c, \beta_c), \\ k_2 &= f\left(X_k + \frac{\Delta t}{2} k_1, \tau_k, V_c, \beta_c\right), \\ k_3 &= f\left(X_k + \frac{\Delta t}{2} k_2, \tau_k, V_c, \beta_c\right), \\ k_4 &= f(X_k + \Delta t k_3, \tau_k, V_c, \beta_c), \\ X_{k+1} &= X_k + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4). \end{aligned} \quad (4.52)$$

The simplified kinematics and dynamics of the vessel $f(X, \tau, V_c, \beta_c)$, used by the RK4 method, are described by the following state derivatives in the inertial and

body-fixed reference frames:

$$f(X, \tau, V_c, \beta_c) = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \\ \dot{u} \\ \dot{v} \\ \dot{r} \end{bmatrix} = \begin{bmatrix} u \cos \psi - v \sin \psi \\ u \sin \psi + v \cos \psi \\ r \\ \frac{\tau_X - X_u u_{\text{rel}}}{m_{\text{eff}}} \\ \frac{\tau_Y - Y_v v_{\text{rel}} - D_y v_{\text{rel}} |v_{\text{rel}}|}{m_{\text{eff}}} \\ \frac{\tau_N - N_r^{(\ell)} r - N_r^{(\text{nl})} |r| r}{I_{zz, \text{eff}}} \end{bmatrix}, \quad (4.53)$$

where u_{rel} and v_{rel} are the relative velocities calculated by transforming the ocean current to its north and east components

$$V_{c,n} = V_c \cos(\beta_c), \quad V_{c,e} = V_c \sin(\beta_c),$$

transforming them into the BODY frame

$$u_c = V_{c,n} \cos \psi + V_{c,e} \sin \psi, \quad v_c = -V_{c,n} \sin \psi + V_{c,e} \cos \psi,$$

and then subtracting them from the total velocities

$$u_{\text{rel}} = u - u_c, \quad v_{\text{rel}} = v - v_c.$$

The kinematic part of f computes the rates of change of the vessel's inertial-frame position and heading by using the total body-fixed velocities u and v . In particular, the rate of change of x comes from projecting both surge and sway into the inertial x axis (using the heading ψ). Similarly, the rate of change of y comes from projecting u and v into the inertial y axis. The rate of change of heading ψ is simply the yaw rate r . In the dynamics, the rate of change of surge u is found by combining the commanded surge thrust and a linear damping term that acts on the water-relative surge speed u_{rel} , both divided by the effective surge mass. The rate of change of sway v is assembled by combining the commanded lateral thrust, a linear damping term acting on the water-relative sway speed v_{rel} and a quadratic (cross-flow) drag term acting on v_{rel} , all divided by the effective sway mass. Finally, the rate of change of yaw r balances the commanded yaw moment against both linear damping on r and a nonlinear damping term proportional to $|r|r$, divided by the effective yaw inertia. The numerical values used for effective masses, damping coefficients, and drag factors are given in **Table A.1** in the appendix.

Algorithm 3 Control Allocation: Pseudo-Inverse with Constraint Handling

Require: Desired force $\tau \in \mathbb{R}^3$, allocation matrix $B \in \mathbb{R}^{3 \times 4}$, thrust gains $k_{\text{pos}}, k_{\text{neg}} > 0$

Ensure: Thruster commands $\{n_1, \alpha_1, n_2, \alpha_2\}$

1: **Compute pseudo-inverse:**

$$B^+ \leftarrow B^T (B B^T)^{-1} \in \mathbb{R}^{4 \times 3}$$

2: **Compute extended force vector:**

$$u_{\text{ext}} \leftarrow B^+ \tau = [u_{1x}, u_{1y}, u_{2x}, u_{2y}]^T$$

3: **for** $i = 1, 2$ **do**

4: Compute magnitude and raw angle for each pod:

$$F_i \leftarrow \sqrt{u_{ix}^2 + u_{iy}^2}$$

$$\alpha_i^{\text{raw}} \leftarrow \text{atan2}(u_{iy}, u_{ix})$$

5: Clamp angle and record flip:

$$(\alpha_i, s_i) = \begin{cases} (\alpha_i^{\text{raw}} - \pi, -1) & \alpha_i^{\text{raw}} > \frac{\pi}{2}, \\ (\alpha_i^{\text{raw}} + \pi, -1) & \alpha_i^{\text{raw}} < -\frac{\pi}{2}, \\ (\alpha_i^{\text{raw}}, +1) & \text{otherwise.} \end{cases}$$

6: Select gain:

$$k_i \leftarrow \begin{cases} k_{\text{pos}} & s_i > 0, \\ k_{\text{neg}} & s_i < 0. \end{cases}$$

7: Compute normalized thrust:

$$n_i \leftarrow s_i \sqrt{\frac{F_i}{k_i}}$$

8: **end for**

9: **if** $\max(|n_1|, |n_2|) > 1$ **then**

10: Scale $\tau \leftarrow \frac{\tau}{\max(|n_1|, |n_2|)}$

11: **goto** line 1

12: **end if**

13: **return** $\{n_1, \alpha_1, n_2, \alpha_2\}$

Algorithm 4 Control Allocation: Nonlinear Optimization with constraint handling**Require:** Desired moments (τ_X, τ_Y, τ_N) , current states $(n_1^0, n_2^0, \alpha_1^0, \alpha_2^0)$ **Ensure:** Optimal thrust commands n_i^* and azimuth angles α_i^* for pods $i = 1, 2$ 1: **Parameters:**

$$k_{\text{pos}}, k_{\text{neg}}, \ell_x, \ell_{y,1}, \ell_{y,2}, n_1^{\text{cmd}}, n_2^{\text{cmd}}, \alpha_1^{\text{cmd}}, \alpha_2^{\text{cmd}},$$

$$\omega_\tau, \omega_{\text{slip}}, \omega_{\text{surge-loss}}, \omega_n, \omega_\alpha$$

2: **Thrust mapping:**

$$T_i(n_i^{\text{cmd}}) = \begin{cases} k_{\text{pos}} n_i^{\text{cmd}} |n_i^{\text{cmd}}|, & n_i \geq 0, \\ k_{\text{neg}} n_i^{\text{cmd}} |n_i^{\text{cmd}}|, & n_i < 0, \end{cases} \quad i = 1, 2$$

3: **Model expressions:**

$$\begin{aligned} \hat{\tau}_X &= T_1(n_1^{\text{cmd}}) \cos \alpha_1^{\text{cmd}} + T_2(n_2^{\text{cmd}}) \cos \alpha_2^{\text{cmd}}, \\ \hat{\tau}_Y &= T_1(n_1^{\text{cmd}}) \sin \alpha_1^{\text{cmd}} + T_2(n_2^{\text{cmd}}) \sin \alpha_2^{\text{cmd}}, \\ \hat{\tau}_N &= \ell_x [T_1(n_1^{\text{cmd}}) \sin \alpha_1^{\text{cmd}} + T_2(n_2^{\text{cmd}}) \sin \alpha_2^{\text{cmd}}] \\ &\quad - [\ell_{y,1} T_1(n_1^{\text{cmd}}) \cos \alpha_1^{\text{cmd}} + \ell_{y,2} T_2(n_2^{\text{cmd}}) \cos \alpha_2^{\text{cmd}}]. \end{aligned}$$

4: **Objectives:**

$$\begin{aligned} J_\tau &= (\tau_X - \hat{\tau}_X)^2 + (\tau_Y - \hat{\tau}_Y)^2 + (\tau_N - \hat{\tau}_N)^2, \\ J_{\text{slip}} &= \exp\left(-\frac{(\alpha_1 - \frac{\pi}{2})^2}{0.2}\right) + \exp\left(-\frac{(\alpha_2 - \frac{\pi}{2})^2}{0.2}\right), \\ J_{\text{surge-loss}} &= \exp\left(-\frac{(|\alpha_1 - \frac{\pi}{2}|)^2 + (|\alpha_2 - \frac{\pi}{2}|)^2}{0.1}\right), \\ J_n &= (n_1^{\text{cmd}} - n_1^0)^2 + (n_2^{\text{cmd}} - n_2^0)^2, \\ J_\alpha &= (\alpha_1^{\text{cmd}} - \alpha_1^0)^2 + (\alpha_2^{\text{cmd}} - \alpha_2^0)^2. \end{aligned}$$

5: **Cost function:**

$$J = \omega_\tau J_\tau + \omega_{\text{slip}} J_{\text{slip}} + \omega_{\text{surge-loss}} J_{\text{surge-loss}} + \omega_n J_n + \omega_\alpha J_\alpha.$$

6: **Optimization problem:**

$$\min_{n_1^{\text{cmd}}, n_2^{\text{cmd}}, \alpha_1^{\text{cmd}}, \alpha_2^{\text{cmd}}} J \quad \text{s.t.} \quad n_i \in [-1, 1], \alpha_i \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right].$$

7: Solve with Nonlinear solver (IPOPT).

8: **if** converged **then**9: **return** $(n_{1,0}^{\text{cmd}}, \alpha_{1,0}^{\text{cmd}}, n_{2,0}^{\text{cmd}}, \alpha_{2,0}^{\text{cmd}})$ 10: **else**11: **return** (0, 0, 0, 0) or fallback to **Algorithm 3**12: **end if**

Algorithm 5 Control allocation: MPC Nonlinear Optimization

Require: Desired moments (τ_X, τ_Y, τ_N) , current states $(n_1^0, n_2^0, \alpha_1^0, \alpha_2^0)$ **Ensure:** first-step pod commands $(n_{1,0}^*, n_{2,0}^*, \alpha_{1,0}^*, \alpha_{2,0}^*)$ 1: **Parameters:**

$$k_{\text{pos}}, k_{\text{neg}}, \ell_x, \ell_{y,1}, \ell_{y,2}, T_n, T_\alpha, \omega_\tau, \omega_{\text{slip}}, \omega_{\text{surge-loss}}, \omega_n, \omega_\alpha, \Delta, N$$

2: **Initialize state variables at $k = 0$:**

$$n_{i,0} = n_i^0, \quad \alpha_{i,0} = \alpha_i^0 \quad (i = 1, 2)$$

3: **for** $k = 0, \dots, N - 1$ **do**4: **Dynamics:**

$$n_{i,k+1} = n_{i,k} + \frac{\Delta}{T_n} (n_{i,k}^{\text{cmd}} - n_{i,k}), \quad \alpha_{i,k+1} = \alpha_{i,k} + \frac{\Delta}{T_\alpha} (\alpha_{i,k}^{\text{cmd}} - \alpha_{i,k}).$$

5: **Bounds:**

$$n_{i,k}^{\text{cmd}} \in [-1, 1], \quad n_{i,k} \in [-1, 1], \quad \alpha_{i,k}^{\text{cmd}} \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right], \quad \alpha_{i,k} \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right].$$

6: **$\hat{\tau}$ mapping:** $\hat{\tau}_k = \tau_{\text{mapping}}(\mathbf{n}_k^{\text{cmd}}, \boldsymbol{\alpha}_k^{\text{cmd}})$ ▷ see Eqns. of Alg. 47: **Stage Cost:** $J += J_k(\boldsymbol{\tau}, \hat{\boldsymbol{\tau}}_k, \mathbf{n}_k^{\text{cmd}}, \mathbf{n}_k, \boldsymbol{\alpha}_k^{\text{cmd}}, \boldsymbol{\alpha}_k, \boldsymbol{\omega})$ ▷ see Eqns. of Alg. 48: **end for**9: **Optimization problem:**

$$\min_{\sum_{k=0}^{N-1} n_{1,k}^{\text{cmd}}, n_{2,k}^{\text{cmd}}, \alpha_{1,k}^{\text{cmd}}, \alpha_{2,k}^{\text{cmd}}} J \quad \text{s.t. the dynamics and bounds at each step}$$

10: Solve with suitable nonlinear solver (IPOPT).

11: **if** converged **then**12: **return** $(n_{1,0}^{\text{cmd}}, \alpha_{1,0}^{\text{cmd}}, n_{2,0}^{\text{cmd}}, \alpha_{2,0}^{\text{cmd}})$ 13: **else**14: **return** $(0, 0, 0, 0)$ or fallback to **Algorithm 3**15: **end if**

Algorithm 6 MPC Control System**Require:** $X_0, (x_s, y_s), (x_d, y_d), \psi_d, N, \Delta t$ **Ensure:** first-step pod commands $(n_{1,0}^*, n_{2,0}^*, \alpha_{1,0}^*, \alpha_{2,0}^*)$.

- 1: **Parameters:** $m_{\text{eff}}, I_{zz,\text{eff}}, X_u, Y_v, D_y, N_r^{(\ell)}, N_r^{(\text{nl})}, \ell_x, \ell_{y,1}, \ell_{y,2}, k_{\text{pos}}, k_{\text{neg}}, T_n, T_\alpha, n_{\min}, n_{\max}, \alpha_{\min}, \alpha_{\max}$
- 2: **Decision variables:** trajectories $\{X_k \in \mathbb{R}^6, n_k, \alpha_k, n_k^{\text{cmd}}, \alpha_k^{\text{cmd}}\}_{k=0}^N$
- 3: set initial condition X_0
- 4: Error computation variables

$$P_x = x_d - x_s, \quad P_y = y_d - y_s, \quad L = \sqrt{P_x^2 + P_y^2}, \quad X_{\text{dir}} = \frac{P_x}{L}, \quad Y_{\text{dir}} = \frac{P_y}{L},$$

- 5: **for** $k = 0$ to $N - 1$ **do**
- 6: **$\hat{\tau}$ mapping:** $\hat{\tau}_k = \tau_{\text{mapping}}(n_k^{\text{cmd}}, \alpha_k^{\text{cmd}})$ ▷ see Eqns. of Alg. 4
- 7: **Model Dynamics:**

$$X_{k+1} = \text{RK4}(X_k, [\tau_{X,k}, \tau_{Y,k}, \tau_{N,k}]^T)$$

- 8: **Computation of objectives:**

$$\begin{aligned} X_{\text{dist},k} &= x_d - x_k, \quad Y_{\text{dist},k} = y_d - y_k, \\ D_{\text{proj}} &= X_{\text{dist},k} X_{\text{dir}} + Y_{\text{dist},k} Y_{\text{dir}}, \quad d_k = \sqrt{\left(\frac{D_{\text{proj}}}{L}\right)^2 + \varepsilon}, \\ y_{e,x} &= X_{\text{dist},k} - D_{\text{proj}} P_x, \quad y_{e,y} = Y_{\text{dist},k} - D_{\text{proj}} P_y, \\ e_{\perp,k} &= \sqrt{y_{e,x}^2 + y_{e,y}^2 + \varepsilon}, \quad h_k = \sqrt{(\psi_k - \psi_d)^2 + \varepsilon}, \\ U_k &= \sqrt{u_k^2 + v_k^2 + \varepsilon}, \quad U_{k,e} = \exp\left(\frac{U_k - 0.8}{0.1}\right) \end{aligned}$$

- 9: **Cost update:**

$$\begin{aligned} J &+= w_1 e_{\perp,k}^2 + w_2 d_k^2 + w_3 h_k^2 + w_4 U_{k,e} \\ &+ w_5 \|n_k^{\text{cmd}} - n_k\|^2 + w_6 \|\alpha_k^{\text{cmd}} - \alpha_k\|^2 \end{aligned}$$

- 10: **Actuator-rate dynamics:**

$$n_{k+1} = n_k + \frac{\Delta t}{T_n}(n_k^{\text{cmd}} - n_k), \quad \alpha_{k+1} = \alpha_k + \frac{\Delta t}{T_\alpha}(\alpha_k^{\text{cmd}} - \alpha_k)$$

- 11: **Bounds:**

$$n_{\min} \leq n_k \leq n_{\max}, \quad \alpha_{\min} \leq \alpha_k \leq \alpha_{\max}$$

- 12: **end for**

- 13: **Optimization problem:**

$$\begin{aligned} \min_{\substack{X_{0:N}, n_{0:N}, \alpha_{0:N}, \\ n_{0:N-1}^{\text{cmd}}, \alpha_{0:N-1}^{\text{cmd}}}} J \quad & \text{s.t. dynamics, actuator-rate and bounds} \end{aligned}$$

- 14: Solve the nonlinear program with a suitable solver (IPOPT)

- 15: **if** converged **then**

- 16: **return** $(n_{1,0}^{\text{cmd}}, \alpha_{1,0}^{\text{cmd}}, n_{2,0}^{\text{cmd}}, \alpha_{2,0}^{\text{cmd}})$

- 17: **else**

- 18: **return** $(0, 0, 0, 0)$ or fallback to **Algorithm 3**

- 19: **end if**

Chapter 5

Path Planning and Guidance

This chapter outlines the path generation strategies and guidance laws employed to evaluate the vessel model across various scenarios and control schemes.

5.1 Path Planning

Two different path generation methods were used for testing the vessel. One for straight-line paths and another for smooth, continuous paths. Both methods calculate the derivatives of the closest points for path guidance.

5.1.1 Straight Line Path

The straight line path method projects the vessel onto the line between the WPT's and does a custom closest point search using the WPT list, WPT index and vessel position, with a "smart" WPT index update.

Line Projection

The line projection finds the closest point on the straight line between two WPT, as well as the first and second derivative of that point. Given two WPT's $\mathbf{A}$ and $\mathbf{B}$ and the vessel position $\mathbf{V}_p$, the projection of the vessel onto the line segment between $\mathbf{A}$ and $\mathbf{B}$ is computed as follows:

$$t = \frac{(\mathbf{V}_p - \mathbf{A})(\mathbf{B} - \mathbf{A})}{\|\mathbf{B} - \mathbf{A}\|^2} \quad (5.1)$$

To ensure the projection lies on the line segment between $\mathbf{A}$ and $\mathbf{B}$, the parameter t is clamped to the interval $[0, 1]$:

$$t = \max(0, \min(1, t)) \quad (5.2)$$

The position of the projected point is given by:

$$\mathbf{p}_{\text{proj}} = \mathbf{A} + t(\mathbf{B} - \mathbf{A}) \quad (5.3)$$

The direction of the path at the projection point is simply the normalized direction of the segment:

$$\dot{\mathbf{p}}_{\text{proj}} = \frac{\mathbf{B} - \mathbf{A}}{\|\mathbf{B} - \mathbf{A}\|} \quad (5.4)$$

Since the path is a straight line, the second derivative is zero:

$$\ddot{\mathbf{p}}_{\text{proj}} = \mathbf{0} \quad (5.5)$$

Closest point search

The closest point search does two things. It determines the point on the current or next path segment that is closest to the vessel's position, and it updates the active WPT index if the vessel has progressed past the current segment.

For each segment, it calculates the vessel projection using **Eq. 5.3**, calculates the along-track error x_e , the cross-track error y_e and finds a total position error using

$$Pos_{\text{error}} = \sqrt{x_e^2 + y_e^2} \quad (5.6)$$

where the along track error and cross track error is given by

$$\lambda = \text{atan2}(\dot{y}_{\text{path}}, \dot{x}_{\text{path}}); \quad (5.7)$$

$$x_e = (x_{\text{vessel}} - x_{\text{path}}) \cos(\lambda) + (y_{\text{vessel}} - y_{\text{path}}) \sin(\lambda); \quad (5.8)$$

$$y_e = -(x_{\text{vessel}} - x_{\text{path}}) \sin(\lambda) + (y_{\text{vessel}} - y_{\text{path}}) \cos(\lambda); \quad (5.9)$$

The first segment is always calculated. If the current WPT index is pointing at the last WPT, the closest point on the first segment is returned together with the along track and cross track errors for guidance. If the current WPT index is not pointing at the last WPT, then we project the vessel onto the next segment and calculates it's along-track, cross-track and positional error. The positional error for the closest point from each segment is then compared to decide which segment the vessel is currently on. The closest point giving the smallest positional error is then returned. If the closest point is on the next segment, the WPT index gets updated, such that this segment becomes the first one to be calculated in the next iteration. The process is described clearly in Algorithm 7.

Algorithm 7 Get Closest Path Point from Straight Line path

```

1: Input: Vessel position  $V_p$ , waypoint index wpt_index
2: Output: Closest path point  $p$ , along-track error  $x_e$ , cross-track error  $y_e$ 

3: Step 1: Projection onto the approaching Segment
4: - Closest point = projection onto the segment approaching the current WPT.
5: - Compute along-track error  $x_e$ 
6: - Compute cross-track error  $y_e$ 
7: - Compute  $Pos_{error}$ 
8: if (approaching the last WPT) then
9:   - Return closest point,  $x_e$ ,  $y_e$ 
10: end if

11: Step 2: Projection onto the departing Segment
12: - Repeat steps 4 to 7 for the departing segment.

13: Step 3: Choose the Closest Projection
14: if (approaching segment error < departing segment error) then
15:   - Return the closest point from the approaching segment
16: else
17:   - wpt_index++
18:   - Return the closest point from the departing segment
19: end if

```

5.1.2 Continuous-Curvature Path Using Fermat's Spiral

As stated in research question Q1, we want to look at the performance of the vessel in confined waters at low speeds. For realistic testing of the vessel performance under such conditions, a suitable path or trajectory is needed. Presumably, this is a path with a shape that is predictable when generated, and one that the vessel is able to follow with minimal error in position and heading.

One such path smoothing method, designed for marine vessels, is presented in [52], where the formulation was improved later in [53]. They formulate a continuous-curvature path generation method using Fermat's Spiral, which is defined as:

$$r = k\sqrt{\theta} \quad (5.10)$$

where r is the radial distance, θ is the polar angle and k is a scaling constant.

Fermat's spiral (FS) features zero curvature at its origin, which enables a seamless connection with a straight line, avoiding the abrupt curvature change typically seen when joining a straight line and a circular arc in Dubins paths. Moreover, unlike clothoids, which are computationally demanding, FS is defined by straightforward parametric equations that are easy to compute. One drawback is that calculating the arc length of an FS arc necessitates the evaluation of a Gaussian hypergeometric function. However, this function converges absolutely and does not constrain the domain over which the length can be determined. Additionally,

the FS parameterization can be formulated without any speed singularities, making the spiral well suited for path-tracking applications. [52]

A nice property of the segmented path is that should one need to adjust it due to the generated path going through a "forbidden" zone, changes locally on the path can be done predictably due to the robust mathematical formulation and that local changes on the path, caused by moving a waypoint, only change the path locally. One clear limitation is that if the waypoints are angled and too close, violating the path length needed to make the Fermat's Spiral segment and its mirrored counterpart with a given curvature constraint, the smooth path cannot be generated. However, such a path might not be feasible to follow for the vessel, limiting the significance of this limitation.

The remaining part of this section is dedicated as an implementation guide of the Fermat's Spiral path theory presented by these papers. For a thorough explanation of the theory, reading [52] is recommended. An FS path example is shown in Figure 5.1.

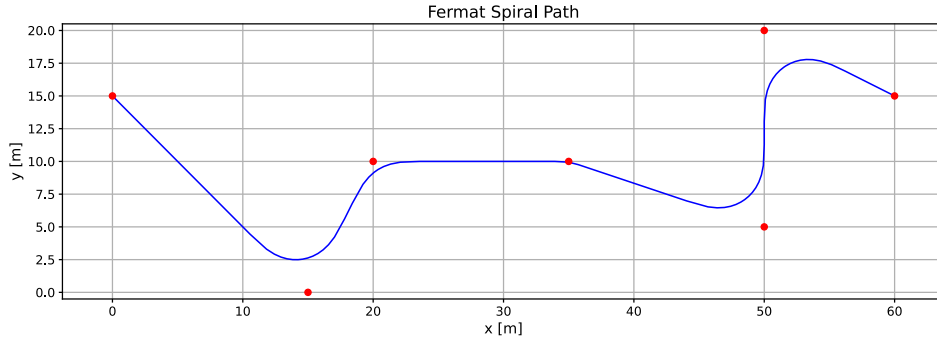


Figure 5.1: Showcase of how a Fermat's Spiral path (in blue) can look like when connecting waypoints (in red).

Computing the FS parameters

Given a WPT that is between two other WPT's the FS segment parameters around that WPT can be computed in the following way. First, the unit vectors for the incoming and outgoing directions can be calculated using normalization:

$$\mathbf{v}_{\text{in}} = \frac{\mathbf{B} - \mathbf{A}}{\|\mathbf{B} - \mathbf{A}\|} \quad (5.11)$$

$$\mathbf{v}_{\text{out}} = \frac{\mathbf{C} - \mathbf{B}}{\|\mathbf{C} - \mathbf{B}\|} \quad (5.12)$$

The angle between the vectors is then given by

$$\delta_{\chi} = \cos^{-1}(\mathbf{v}_{\text{in}} \cdot \mathbf{v}_{\text{out}}) \quad (5.13)$$

The direction of the turn, which is either left or right, can be computed using the negative sign of the 2D cross product.

$$\rho = -\text{sign}(v_{\text{in},y} v_{\text{out},x} - v_{\text{in},x} v_{\text{out},y}) \quad (5.14)$$

Here -1 is a left turn, and $+1$ is a right turn. Using the angle vectors the heading angles of the entry χ_0 and exit χ_{end} can be calculated.

$$\chi_0 = \tan^{-1} \left(\frac{v_{in,y}}{v_{in,x}} \right) \quad (5.15)$$

$$\chi_{end} = \tan^{-1} \left(\frac{v_{out,y}}{v_{out,x}} \right) \quad (5.16)$$

The angle between the vectors δ_χ is used to find the theta value θ_{end} at the end of the FS segment and it's mirrored counterpart. The θ_{end} value can be computed using Halley's method, which is absolutely convergent for this problem. In this case the objective is to solve the nonlinear equation;

$$f(\theta, \delta_{\frac{\chi}{2}}) = 0, \quad (5.17)$$

where

$$\delta_{\frac{\chi}{2}} = \frac{\delta_\chi}{2} \quad (5.18)$$

and the function f is defined as

$$f(\theta, \delta_{\frac{\chi}{2}}) = \theta + \tan^{-1}(2\theta) - \delta_{\frac{\chi}{2}}. \quad (5.19)$$

The first and second derivatives of the function are given by:

$$f'(\theta) = 1 + \frac{2}{1 + 4\theta^2}, \quad (5.20)$$

$$f''(\theta) = -\frac{16\theta}{(1 + 4\theta^2)^2}. \quad (5.21)$$

Using Halley's method, the update rule for θ is:

$$\theta_{n+1} = \theta_n - \frac{2f(\theta_n, \delta_{\frac{\chi}{2}})f'(\theta_n)}{2(f'(\theta_n))^2 - f(\theta_n, \delta_{\frac{\chi}{2}})f''(\theta_n)}. \quad (5.22)$$

The point at which the FS and its mirrored segment both reach the value

$$\theta = \theta_{end}$$

is the midpoint of the turn where the two curves meet. The search implementation is shown in algorithm 8.

After the search, it's important to clamp the θ_{end} value to the user specified curvature constraint κ_{max} . The κ_{max} value should be set at or below the maximum curvature the vessel can safely follow at the desired speed.

$$\theta_{max} = \min(\theta_{end}, \theta_{\kappa,max}) \quad (5.23)$$

Algorithm 8 Compute θ_{end} using Halley's Method

Require: $\delta_{\frac{\chi}{2}}$ ▷ Corner angle
Ensure: θ_{end}
1: $\theta \leftarrow \delta_{\frac{\chi}{2}}$
2: $\text{tol} \leftarrow 0.001$
3: $\text{maxIter} \leftarrow 10$
4: **for** $i = 1$ to maxIter **do**
5: $\Delta\theta \leftarrow \frac{2f \cdot f'}{2(f')^2 - f \cdot f''}$
6: $\theta \leftarrow \theta - \Delta\theta$
7: **if** $|\Delta\theta| < \text{tol}$ **then**
8: **break**
9: **end if**
10: **end for**
11: **return** θ

Using the clamped value θ_{max} and the κ_{max} value the scaling factor for the spiral is computed as

$$k = \frac{2}{\kappa_{\text{max}}} \sqrt{\theta_{\text{max}}} \frac{3 + 4\theta_{\text{max}}^2}{(1 + 4\theta_{\text{max}}^2)^{1.5}} \quad (5.24)$$

Further, using k , the arc length ℓ to transition in and out of the spiral can be computed.

$$l_1 = k \sqrt{\theta_{\text{end}}} \cos(\theta_{\text{end}}), \quad (5.25)$$

$$h = k \sqrt{\theta_{\text{end}}} \sin(\theta_{\text{end}}), \quad (5.26)$$

$$\alpha = \frac{\pi - \delta_{\frac{\chi}{2}}}{2}, \quad (5.27)$$

$$l_2 = \frac{h}{\tan(\alpha)}, \quad (5.28)$$

$$\ell = l_1 + l_2. \quad (5.29)$$

Finally, the coordinates of the entry and exit point can be found.

$$\mathbf{wheel}_{\text{over}} = \mathbf{B} - \ell \mathbf{v}_{\text{in}} \quad (5.30)$$

$$\mathbf{pull}_{\text{out}} = \mathbf{B} + \ell \mathbf{v}_{\text{out}} \quad (5.31)$$

For convenience, we can split the positions into x and y coordinates.

$$x_0 = \mathbf{wheel}_{\text{over},x}, \quad y_0 = \mathbf{wheel}_{\text{over},y} \quad (5.32)$$

$$x_{\text{end}} = \mathbf{pull}_{\text{out},x}, \quad y_{\text{end}} = \mathbf{pull}_{\text{out},y} \quad (5.33)$$

Lastly, an alternative parametrization of FS which eliminates the parametric speed singularity at the origin is used by doing a change of variable.

$$u = \sqrt{\theta} \quad (5.34)$$

Parameterizations

Using **Eq. 5.34** the curve segments of the path, consisting of a FS segment and a corresponding mirrored FS segment, can be parameterized as:

$$\begin{aligned} x &= x_0 + k u \cos(\rho u^2 + \chi_0) \\ y &= y_0 + k u \sin(\rho u^2 + \chi_0) \end{aligned} \quad (5.35)$$

$$\begin{aligned} x_m &= x_{end} - k u \cos(\rho u^2 - \chi_{end}) \\ y_m &= y_{end} + k u \sin(\rho u^2 - \chi_{end}) \end{aligned} \quad (5.36)$$

The first derivative of the parameterizations are given by

$$\begin{aligned} \dot{x} &= k (\cos(\rho u^2 + \chi_0) - 2\rho u^2 \sin(\rho u^2 + \chi_0)) \\ \dot{y} &= k (\sin(\rho u^2 + \chi_0) + 2\rho u^2 \cos(\rho u^2 + \chi_0)) \end{aligned} \quad (5.37)$$

$$\begin{aligned} \dot{x}_m &= k (-\cos(\rho u^2 - \chi_{end}) + 2\rho u^2 \sin(\rho u^2 - \chi_{end})) \\ \dot{y}_m &= k (\sin(\rho u^2 - \chi_{end}) + 2\rho u^2 \cos(\rho u^2 - \chi_{end})) \end{aligned} \quad (5.38)$$

The second derivative of the parameterizations are given by

$$\begin{aligned} \ddot{x} &= k (-6\rho u \sin(\rho u^2 + \chi_0) - 4u^3 \cos(\rho u^2 + \chi_0)) \\ \ddot{y} &= k (6\rho u \cos(\rho u^2 + \chi_0) - 4u^3 \sin(\rho u^2 + \chi_0)) \end{aligned} \quad (5.39)$$

$$\begin{aligned} \ddot{x}_m &= k (6\rho u \sin(\rho u^2 - \chi_{end}) + 4u^3 \cos(\rho u^2 - \chi_{end})) \\ \ddot{y}_m &= k (6\rho u \cos(\rho u^2 - \chi_{end}) - 4u^3 \sin(\rho u^2 - \chi_{end})) \end{aligned} \quad (5.40)$$

For the straight-line segments of the path, **Eq. 5.1** and **Eq. 5.4** are used. However, instead of using the WPT positions as start and end points, the wheel-over point where the FS segment begins and the pull-out point where the mirrored segment begins are used.

Custom Closest Point Search

The papers [52] or [53] do not state an exact method as to how to search for the closest point on the fully assembled segmented path. A simple radius around waypoints for waypoint switching cannot be used as it simply does not make logical sense when the path is not going through the waypoints but curves past them. In addition, the waypoint we are going towards cannot be the current target waypoint, due to the calculation of the FS segments needing the next waypoint. Also, when on the FS curve, we are not going towards a waypoint target on a straight line path. Due to these challenges, a specialized way for a closest point search is proposed.

The method searches for the closest point on two straight-line segments, an FS segment and its mirrored counterpart, based on the current WPT index. The point with the least positional error of the 4 segments is chosen as the closest point on

the path. The WPT index update happens when the closest point is on the last straight-line segment. In the next search for the closest point, the previous last straight-line segment is now the first one. Exceptions are made for the handling of the initial wpt index and the last wpt index. The method is described in detail in **Algorithm 9**, where we assume a function

$$\text{computeFSParams}(wpt_{start}, wpt, wpt_{end})$$

that calculates **Eq.5.11-5.34** for the section of the path given by the current waypoint and it's neighbouring waypoints. It also assumes a function

$$\text{projectOntoSpiral}(V_p, \text{params}, \rho)$$

described by **Algorithm 10**, that returns the closest point on the spiral segment together with it's derivatives.

Algorithm 9 Get Closest Point on Fermat Spiral Path

Require: V_p ▷ Vessel position
Require: wpt_l ▷ Waypoint list
Require: in/out: wpt_i ▷ Current WPT index

Set waypoint values:

```

1: if  $wpt_i \leq 1$  then
2:    $wpt_i = 2$ 
3: end if
4:  $w_2 = wpt_l[wpt_i - 2]$ ,  $w_1 = wpt_l[wpt_i - 1]$ ,
    $w_0 = wpt_l[wpt_i]$ ,  $w_{+1} = wpt_l[wpt_i + 1]$ 

```

Find the projection and it's error for each segment:

```

5:  $params = \text{computeFSParams}(w_2, w_1, w_0)$ 
6: if  $wpt_i = 2$  then
7:    $P_1 = \text{projectOntoLine}(w_2, params.wheel\_over, V_p)$ 
8: else
9:    $P_1 = \text{projectOntoLine}(stored\_pull\_out, params.wheel\_over, V_p)$ 
10: end if
11:  $e_1 = \text{totalError}(V_p, P_1)$ 

12:  $P_2 = \text{projectOntoSpiral}(V_p, params, \rho = +1)$ ,  $e_2 = \text{totalError}(V_p, P_2)$ 
13:  $P_3 = \text{projectOntoSpiral}(V_p, params, \rho = -1)$ ,  $e_3 = \text{totalError}(V_p, P_3)$ 

14:  $prev\_out = params.pull\_out$ 
15:  $params = \text{computeFSParams}(w_1, w_0, w_{+1})$ 
16: if  $wpt_i < \text{size}(wpt_l) - 1$  then
17:    $P_4 = \text{projectOntoLine}(prev\_out, params.wheel\_over, V_p)$ 
18: else
19:    $P_4 = \text{projectOntoLine}(prev\_out, w_0, V_p)$ 
20: end if
21:  $e_4 = \text{totalError}(V_p, P_4)$ 

```

Select projection with minimum error:

```

22: if  $e_1$  is smallest then
23:   return  $P_1, e_1$ 
24: else if  $e_2$  is smallest then
25:   return  $P_2, e_2$ 
26: else if  $e_3$  is smallest then
27:   return  $P_3, e_3$ 
28: else
29:   if  $wpt_i < \text{size}(wpt_l) - 1$  then
30:      $stored\_pull\_out = prev\_out$ 
31:      $wpt_i \leftarrow wpt_i + 1$ 
32:   end if
33:   return  $P_4, e_4$ 
34: end if

```

Algorithm 10 is one way of implementing the *projectOntoSpiral* function.

Algorithm 10 Project Vessel Onto Fermat Spiral

Require: V : Vessel position

Require: $params$: Spiral parameters

Require: ρ : Segment direction flag (± 1)

Initialize

```

1: Set initial  $\theta = \theta_{end}$ , max iterations  $N = 100$ , tolerance  $\epsilon = 10^{-16}$ 
2: if  $\rho > 0$  then
3:    $angle = \chi_0$ 
4:    $(x_c, y_c) = (x_0, y_0)$ 
5: else
6:    $angle = \chi_{end}$ 
7:    $(x_c, y_c) = (x_{end}, y_{end})$ 
8: end if

```

Search for best θ value

```

9: for  $i = 1$  to  $N$  do
10:   $f = \text{computeF}(\theta, V, angle, \lambda, k, \rho, x_c, y_c, \theta_{end})$ 
11:   $f' = \text{computeFPrime}(\theta, V, angle, \lambda, k, \rho, x_c, y_c, \theta_{end})$ 
12:  if  $|f'| < 10^{-6}$  then
13:    Apply damping:  $f' = \text{sign}(f') \cdot 10^{-6}$ 
14:  end if
15:   $\delta\theta = f / f'$ 
16:  if  $\delta\theta < \epsilon$  or  $\delta\theta$  is NaN then
17:    Break loop
18:  end if
19:   $\theta = \theta - \text{sign}(\delta\theta) \cdot (\delta\theta)^2$ 
20:  Clamp  $\theta$  between  $[0, \theta_{end}]$ 
21: end for

```

Return spiral point and it's derivatives based on the θ value

```

22:  $P = \text{computeSpiralPoint}(\theta, angle, \rho, k, \rho, x_c, y_c, \theta_{end})$ 
23: if  $\rho > 0$  then
24:   $dP = \text{computeSpiralDerivative}(\theta, \dots)$ 
25:   $ddP = \text{computeSpiralSecondDerivative}(\theta, \dots)$ 
26: else
27:   $dP = -\text{computeSpiralDerivative}(\theta, \dots)$ 
28:   $ddP = -\text{computeSpiralSecondDerivative}(\theta, \dots)$ 
29: end if
30: return  $P, dP, ddP$ 

```

Notice that we switch the direction of the derivatives in **Algorithm 10** if $\rho < 0$

because the mirrored segment goes in the opposite direction of the FS segment. This is an important detail for guidance-law's using the path derivatives.

5.2 Guidance laws

This section describes the guidance methods used for path following and dynamic positioning (DP).

5.2.1 Line-of-Sight (LOS)

For path following, the Line-of-Sight (LOS) method [1] is employed to guide the vessel towards a target point, with a lookahead distance Δ from the closest point on the path using the point's tangent line. The reference path's direction, ψ_p , is calculated using the arctangent of the path velocity components v_y and v_x :

$$\psi_p = \arctan 2(\dot{y}_p, \dot{x}_p) \quad (5.41)$$

The cross-track error, y_e , is calculated based on the vessel's position (x_n, y_n) relative to the reference path $(x_{\text{path}}, y_{\text{path}})$ using the path's direction ψ_p .

$$y_e = -(x_n - x_p) \sin(\psi_p) + (y_n - y_p) \cos(\psi_p) \quad (5.42)$$

Finally, the desired heading angle, ψ_{ref} , is determined by adjusting the reference direction ψ_p with a correction based on the cross-track error y_e and the lookahead distance Δ .

$$\psi_{\text{ref}} = \psi_p - \arctan\left(\frac{y_e}{\Delta}\right) \quad (5.43)$$

To filter out high-frequency noise in the commanded LOS angle, ψ_{ref} is augmented with a discrete-time LOS-Observer [1]. Let θ_k be the LOS angle estimate at time step k and $\dot{\theta}_k$ its estimated rate of change. The observer is then defined by the following parameters:

$$T_f = \frac{1}{K_f + 2\sqrt{K_f} + 1}, \quad (5.44)$$

with sampling interval h and filter gain K_f . The update at each step is then

$$\xi_k = \theta_k - \dot{\theta}_k, \quad (5.45)$$

$$\theta_{k+1} = \theta_k + h\left(\dot{\theta}_k + K_f \text{ssa}(\psi_{\text{ref}} - \theta_k)\right), \quad (5.46)$$

$$\Phi = \exp(-h/T_f), \quad (5.47)$$

$$\xi_{k+1} = \Phi \xi_k + (1 - \Phi) \theta_{k+1}, \quad (5.48)$$

$$\dot{\theta}_{k+1} = \theta_{k+1} - \xi_{k+1}. \quad (5.49)$$

Here $\text{ssa}(\cdot)$ is the signed-saturation function. The outputs θ_{k+1} and $\dot{\theta}_{k+1}$ replace the raw ψ_{ref} and its numerical derivative when commanding the vessel's heading controller, yielding a smoother response and reduced sensitivity to measurement noise.

5.2.2 Dynamic Positioning (DP)

In the case of dynamic positioning, the guidance is simply given as the next waypoint (WPT) and a heading. The constant desired heading is either specified by the user, or computed using the angle between the next WPT and the previous WPT.

$$x_d = X_{WPTgoal} \quad (5.50)$$

$$y_d = Y_{WPTgoal} \quad (5.51)$$

$$\psi_d = \text{atan2}(Y_{WPTgoal} - Y_{WPTprev}, X_{WPTgoal} - X_{WPTprev}) \quad (5.52)$$

Chapter 6

Results and Discussion

In this chapter, results from simulating the vessel described in **Chapter 3** using the motion control and control allocation design in **Chapter 4** is presented. To evaluate DP and automatic berthing capabilities the system is tested using a four-square maneuverability test. For PF, performance is evaluated using both a straight line path and a Fermat's Spiral path, by placing waypoints such that they construct a right turn followed by a left turn. Results are generated under Ideal environmental conditions and with moderate water currents. In addition, the performance of each system is evaluated with pod failure under ideal environmental conditions.

Added Realism To generate more realistic simulation results, zero-mean Gaussian noise was added to each variable of the vessel's propagated state (computed with the RK4 method), such that

$$\boldsymbol{\nu} \sim \mathcal{N}(0, 0.01^2 I_6) \quad (6.1)$$

$$\boldsymbol{\eta} \sim \mathcal{N}(0, 0.001^2 I_6) \quad (6.2)$$

where $\boldsymbol{\nu}$ is defined by **Eq. 3.5** and $\boldsymbol{\eta}$ is defined by **Eq. 3.6**.

6.1 Path following

The PF capabilities of the vessel, using the control systems in **Table 6.1**, were evaluated using the tests outlined in **Table 6.2**.

PF System	Motion Control	Control Allocation
1	$\tau_X = 80, \tau_Y = 0,$ $\tau_N = \text{Nomoto PID Chapter 4.2.1}$	Pseudo Inverse: Algorithm 3
2	$\tau_X = 80, \tau_Y = 0,$ $\tau_N = \text{Nomoto PID Chapter 4.2.1}$	Nonlinear Opt.: Algorithm 4
3	$\tau_X = 80, \tau_Y = 0,$ $\tau_N = \text{Nomoto PID Chapter 4.2.1}$	MPC allocation: Algorithm 5

Table 6.1: Control systems used for path following. The motion controller generate virtual control signals that the allocators try to replicate by commanding the propeller speed and azimuth angles of each pod.

PF Test	Path Type	Conditions	Waypoints
1	Straight	Ideal conditions	(0, 0), (0, 40), (40, 40), (40, 80)
2	Fermat's Spiral	Ideal conditions	(0, 0), (0, 40), (40, 40), (40, 80)
3	Fermat's Spiral	Ocean current $0.1 \frac{m}{s}$ at 30°	(0, 0), (0, 60), (60, 60), (60, 120)
4	Fermat's Spiral	Pod 1 failure $n_1 = 0$	(0, 0), (0, 40), (40, 40), (40, 80)

Table 6.2: Description of PF tests. Under Ideal conditions pods are full operational and there are no wind or water currents affecting the vessel. Test 1 and 2 compares system performance for different paths under ideal conditions. In test 3, only a small ocean current is used to demonstrate how the vessel responds to ocean currents when there are no explicit ocean current compensation in the control loop. To evaluate system performance in the event of a pod failure, the propeller speed of pod 1 is set to zero under ideal environmental conditions in test 4.

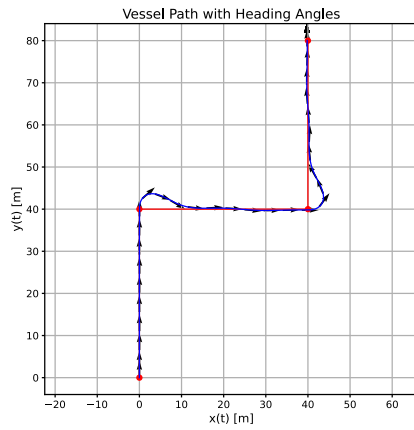
6.1.1 PF in Ideal Conditions

Under ideal conditions the PF systems in **Table 6.1** were tested using both a straight line path and a Fermat's Spiral path.

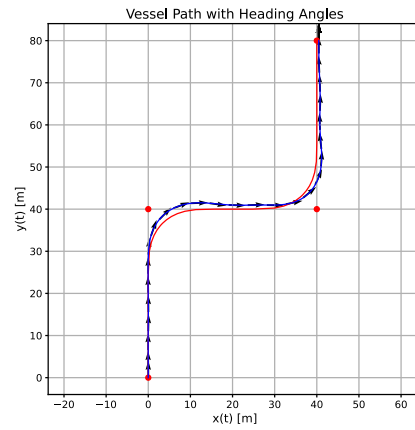
PF in Ideal conditions: System 1

System 1 uses a pseudo inverse based control allocation. If the yaw moment is zero, the pseudo inverse matrix multiplications will always result in a 0° commanded azimuth angle. The motion controller gives a constant $\tau_Y = 0$ at all times. Hence, only the propeller speed is used to control the vessel in System 1. The commanded and actual propeller speeds used to control the vessel when following a straight line path and a Fermat's spiral path is shown in **Figure 6.2**, and the resulting vessel responses can be seen in **Figure 6.1**. In **Figure 6.1** the

path deviation is considerably smaller when using the Fermat's Spiral path. From **Figure 6.2** it's clear that by following the Fermat's Spiral path, the vessel reach the last waypoint faster. In addition, there are smaller changes to the commanded propeller speed values. This showcases the importance of using paths that take vessel dynamics, like maximum turn radius, into account.



(a) Straight line path



(b) Fermat's spiral path

Figure 6.1: PF system 1 performance under ideal conditions. The red line is the desired path, the red dot's are the waypoints, the blue line is the path taken by the vessel and the black arrows shows the vessels heading along the path.

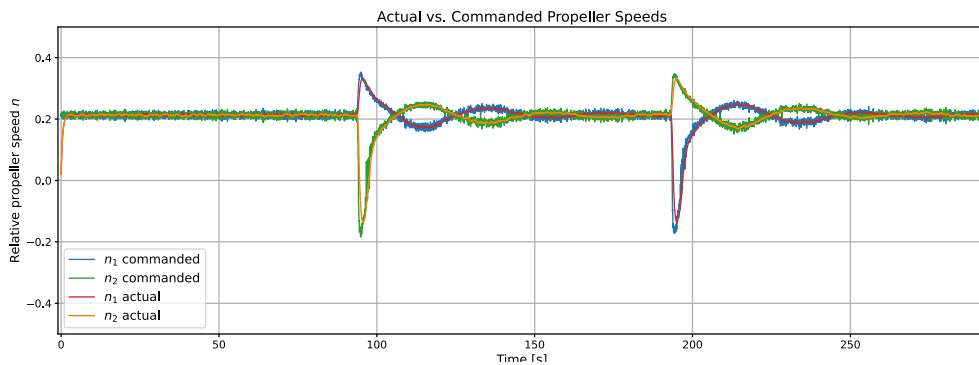


Figure 6.2: PF system 1. Actual vs. commanded propeller speed under ideal conditions using the straight line path.

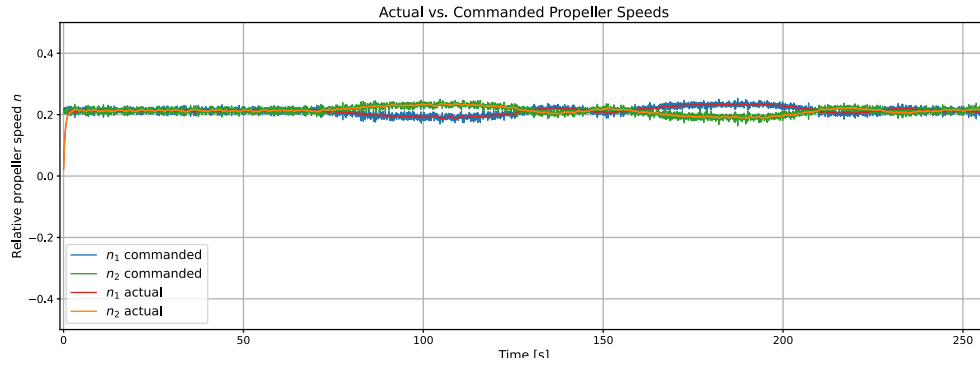


Figure 6.3: PF system 1. Actual vs. commanded propeller speed under ideal conditions using the Fermat's spiral path.

PF in Ideal conditions: System 2

The vessel response given by PF system 2 is practically identical to the one shown in **Figure 6.1** for PF system 1. However, in contrast to system 1, system 2 makes some use of the azimuth's, instead of only relying on the propellers. However, in the ideal case, this does not seem to make a difference. The propeller and azimuth commands and values can be seen in **Figure 6.4** and **6.5** respectively.

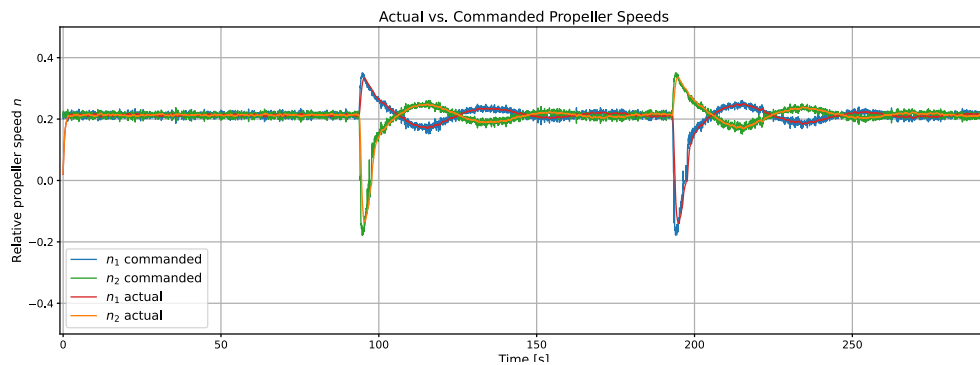


Figure 6.4: PF system 2. Actual vs. commanded propeller speeds when following the straight line path under ideal conditions.

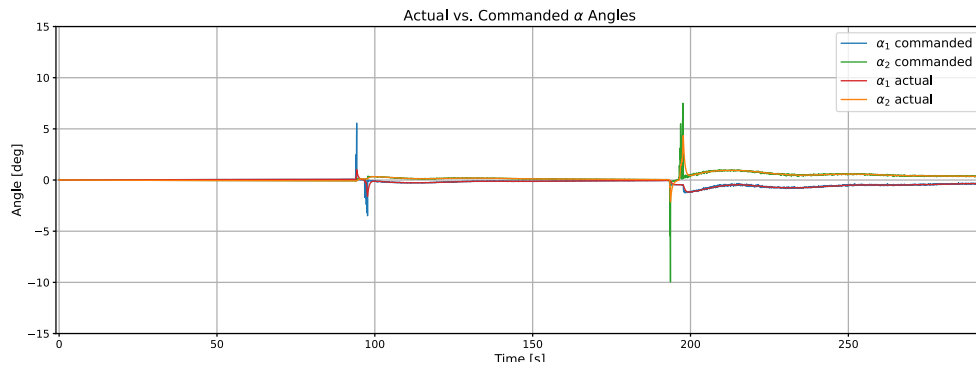


Figure 6.5: PF system 2. Actual vs. commanded Azimuth Angles when following the straight line path under ideal conditions.

PF in Ideal conditions: System 3

System 3 utilize the azimuths more than system 2 under ideal conditions. As shown in **Figure 6.7**, the system keeps each pod slightly outward almost at all times. However, the vessel response when using PF system 3 on the straight line and FS path were still practically identical to system 1 and 2. Although the azimuth angles are different, the commanded propeller speeds shown in **Figure 6.6** looks almost identical to the propeller speeds used by the other systems.

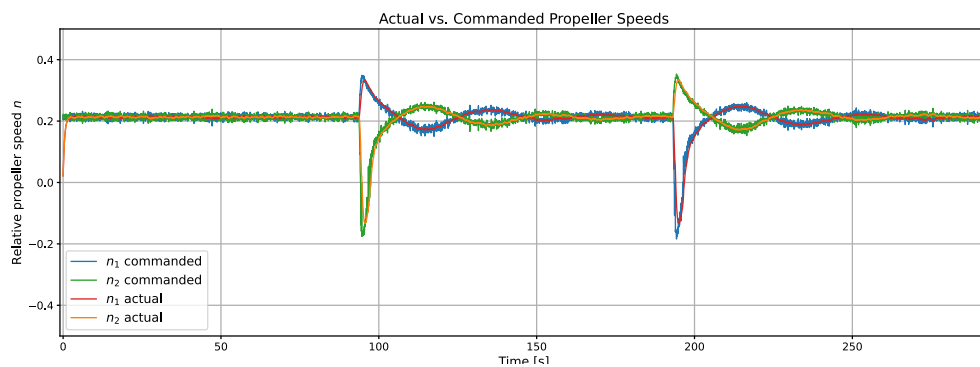


Figure 6.6: PF system 3. Actual vs. commanded propeller speeds under ideal conditions.

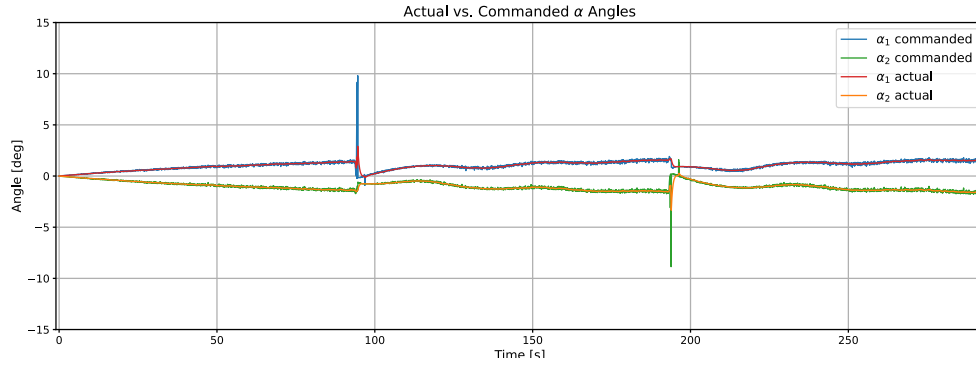


Figure 6.7: PF system 3. Actual vs. commanded Azimuth Angles under ideal conditions.

6.1.2 PF with Ocean Currents

With ocean currents the vessel response under all three PF control systems are still practically the same. The main difference between the systems is still that system 1's pseudo-inverse method does not use azimuths. Other than that, and some slightly different finish times, all systems had almost the exact same responses. Therefore, only figures from the vessel response using system 1 under ocean currents are shown here. With an ocean current of $0.1 \frac{m}{s}$ at an angle of 30° the vessel drifts away from the desired path, as shown in **Figure 6.9**. The distance the vessel drifts from the path is limited, because the Nomoto PID in the motion controller adjusts the heading of the vessel more and more the further away from the path the vessel goes. The heading correction and vessel drift eventually reach an equilibrium along the straight-line segments, where the vessel continues to follow the path with a stable cross-track error. The cross-track error over time is shown in **Figure 6.8**.

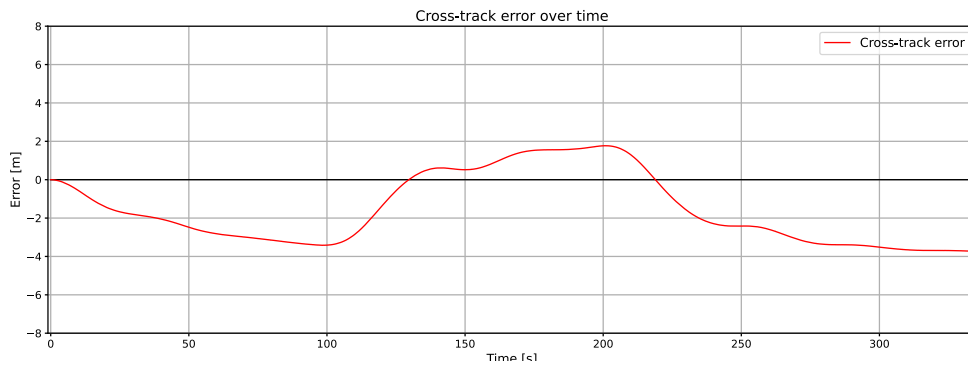


Figure 6.8: Cross-track error when using PF system 1 and following a FS path with ocean currents. Along each straight-line segment of the FS path, the cross-track error eventually stabilizes when the heading controller has adjusted the vessel's heading sufficiently.

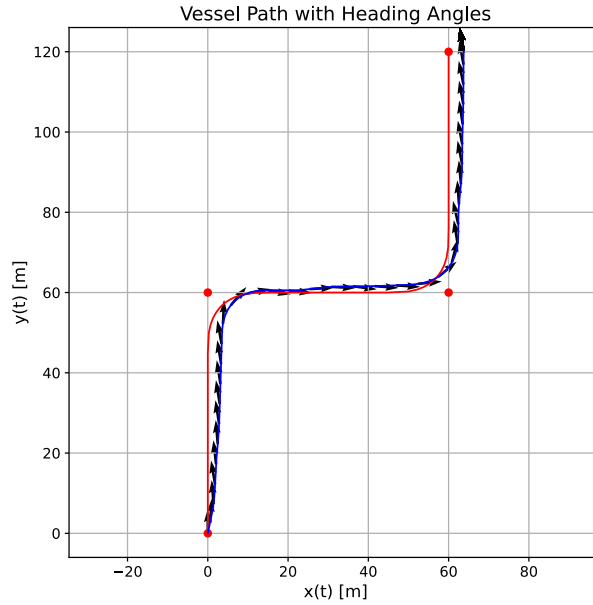


Figure 6.9: Vessel response using PF system 1, with ocean currents on a FS path. The red line is the desired path, the red dot's are the waypoints, the blue line is the path taken by the vessel and the black arrows shows the vessels heading along the path.

6.1.3 PF with Pod Failure

PF systems 2 and 3 from **Table 6.1** were tested with failure of pod 1 by keeping $n_1 = 0$. As mentioned in **Chapter 6.1.1**, PF system 1 does not use azimuth angles when $\tau_Y = 0$. Because of this, when there is a pod failure, system 1 steers with only one fixed propeller. This is not controllable, and hence no results are presented for system 1 with pod failure. As was the case when testing under ocean currents and ideal conditions, the vessel responses using systems 2 and 3 are essentially identical when one of the pods fails. Therefore, only the results from system 2 are presented here.

The vessels response using PF system 2, when pod 1 fails and only pod 2 remains, is shown in **Figure 6.10**. From the figure, it can be seen that the vessel moves forward at an angle compared to the path. At the same time, **Figure 6.11** shows that the remaining pod is placed at an angle approximately equal to the angle displacement of the vessel compared to the path, but in the opposite direction, in the BODY-frame. What is happening is that the pod is pushing the vessel forward along the path, while the yaw moment is balanced by the vessel's heading angle. If the heading of the vessel becomes slightly off, the yaw moment will make the vessel spin. From simple intuition, one can expect this yaw/heading equilibrium to be open-loop unstable, such that rapid corrections are needed when the

vessel starts turning one way or the other, which is why the system's commanded azimuth values in **Figure 6.11** are rapidly varying.

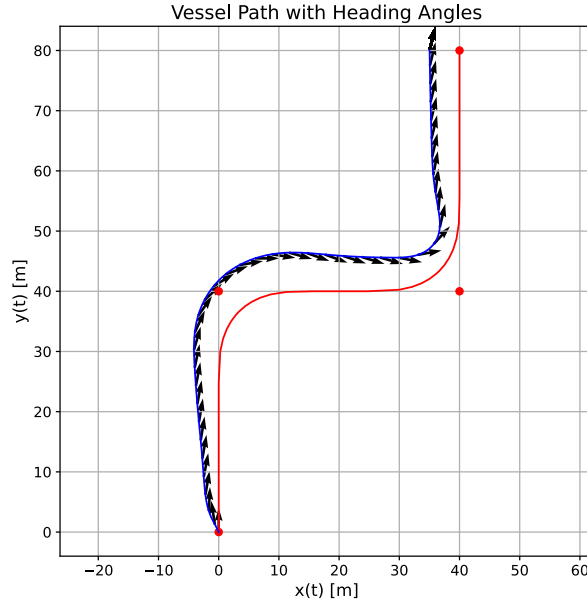


Figure 6.10: PF system 2 with pod (1) failure: Vessel response when following the FS path in ideal environmental conditions.

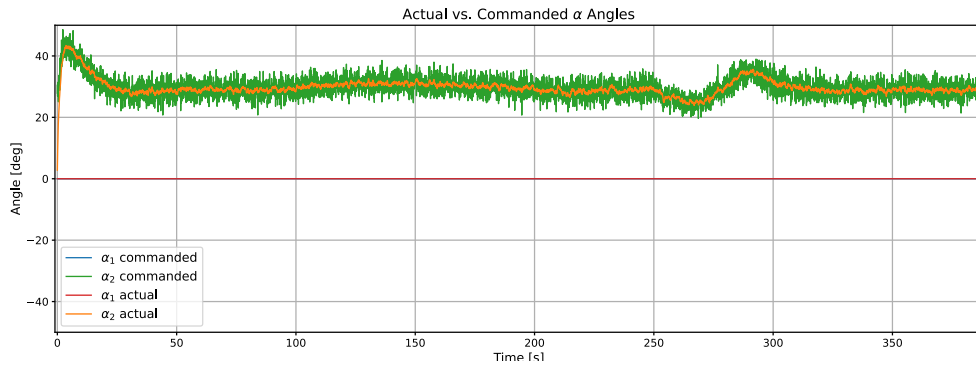


Figure 6.11: PF system 2 with pod (1) failure: Commanded vs Actual azimuth angles.

In contrast to the azimuth angle, the propeller speed shown in **Figure 6.12** of the remaining pod is almost constant, with an exception for the first few seconds. This is because the constant commanded surge force at 80 N can only be fulfilled by this pod alone, and not balanced between the two pods.

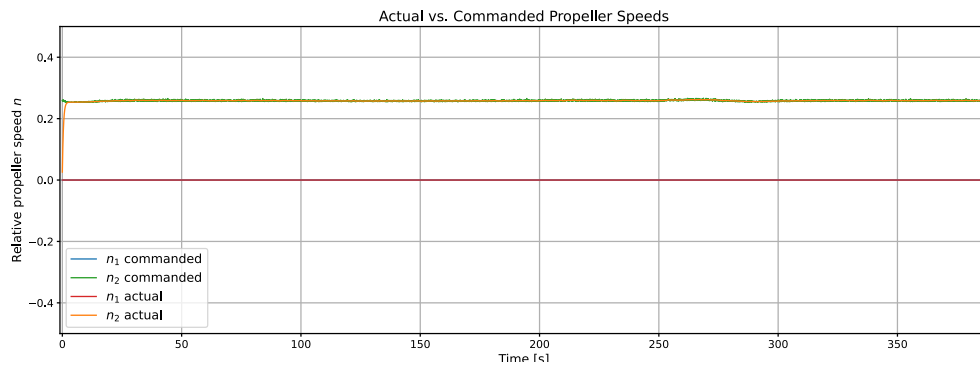


Figure 6.12: PF system 2 with pod (1) failure: Commanded vs Actual propeller speeds

6.2 Four-square test: Dynamic Positioning and Berthing

To understand how the different control schemes could work for DP and berthing scenarios, they were tested using a custom four-square test, inspired by the four-square test in . The test consist of four path segments. The first one is a surge control test, the second is a sway control test, the third is a reverse surge control test and the fourth tests surge, sway and yaw. Each test is ended once the vessel stands still and is oriented correctly at the end of a segment with small margins for error. The next test starts immediately after the current one ends. The technical details of each part is shown in **Table 6.3**. The four different control system configurations used for the four-square test are described by **Table 6.4**. For clarity, figures containing results from the four-square test, that have time on the horizontal axis, have black vertical lines splitting them into four parts, where each part belongs to one of the four tests.

Test	Description	Start (x, y)	End (x, y)	ψ (°)
1	0° heading towards the goal	(0, 0)	(0, 15)	90
2	90° heading towards the goal	(0, 15)	(15, 15)	90
3	180° heading towards the goal	(15, 15)	(15, 0)	90
4	45° heading towards the goal	(15, 0)	(0, 0)	135

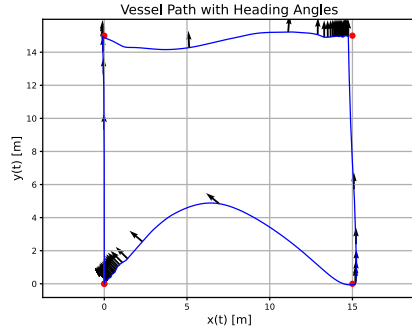
Table 6.3: Technical details of the four-square test used to evaluate DP and berthing capabilities.

DP System	Motion Control	Control Allocation
1	MIMO PID Chapter 4.2.2	Pseudo Inverse: Algorithm 3
2	MIMO PID Chapter 4.2.2	Nonlinear Opt.: Algorithm 4
3	MIMO PID Chapter 4.2.2	MPC allocation: Algorithm 5
4	(none)	MPC control system: Algorithm 6

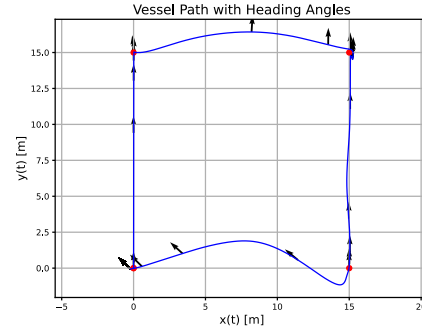
Table 6.4: Control system configurations for DP and berthing.

6.2.1 DP in Ideal conditions

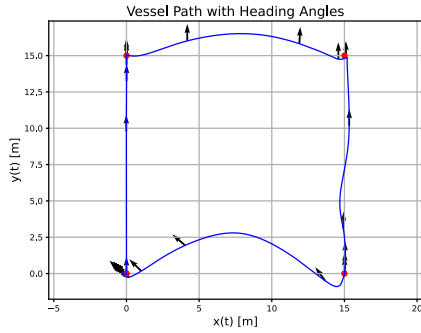
To showcase the behavior of each control system, they were tested under ideal conditions, with no wave or wind disturbances and fully operational pod's. The performance in terms of position and heading for each method is shown in **Figure 6.13**.



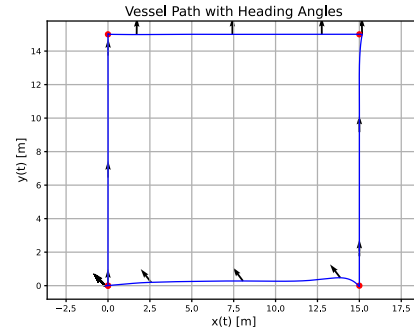
(a) DP system 1



(b) DP system 2



(c) DP system 3



(d) DP system 4

Figure 6.13: The four-square test performance of the vessel under each control system under ideal conditions.

DP Ideal Conditions: System 1 results

Figure 6.13a showcase the performance of control system 1 in Table 6.4. It has good surge control, but struggles a bit more with sideways motions. Looking at figure Figure 6.14 we see that the state errors reduce quickly, but once the state errors are low, and small precise movements are needed to converge perfectly towards the goal, the system struggles, and use a lot of time to get rid of the small remaining error. Looking at the computed control inputs for propeller speed in Figure 6.15 and azimuth angles in Figure 6.16, we observe that the algorithm calculates undesirable references that the pod's are not able to follow due to rate constraints. Each test segment gives a constant desired position and heading and the noise added to the propagated states of the vessel are not considerable. So why is the allocation of n and α so noisy? It is most certainly caused by the MIMO PID that generate far from optimal virtual control signals fo the allocator, as seen in Figure 6.17. This can explain the slowly dying state errors in Figure 6.14. Due to the subpar virtual control signal, it is difficult to draw definitive conclusions about how well or poorly Algorithm 3 that is used as the allocation method in DP

system 1 is or if it is only a better motion controller that is needed.

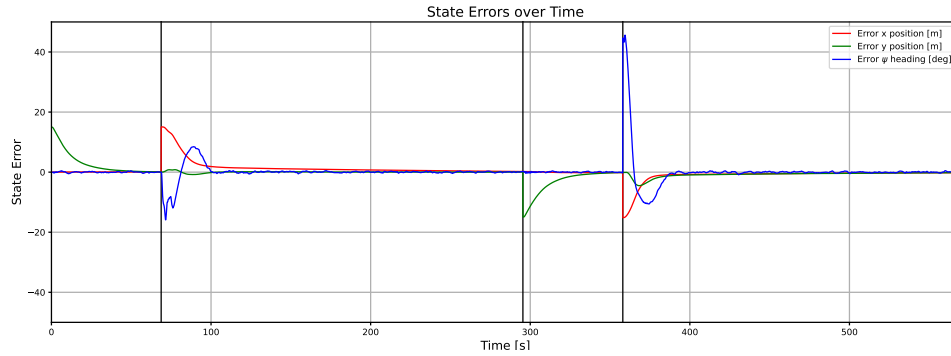


Figure 6.14: DP system 1: X, Y and heading error over time.

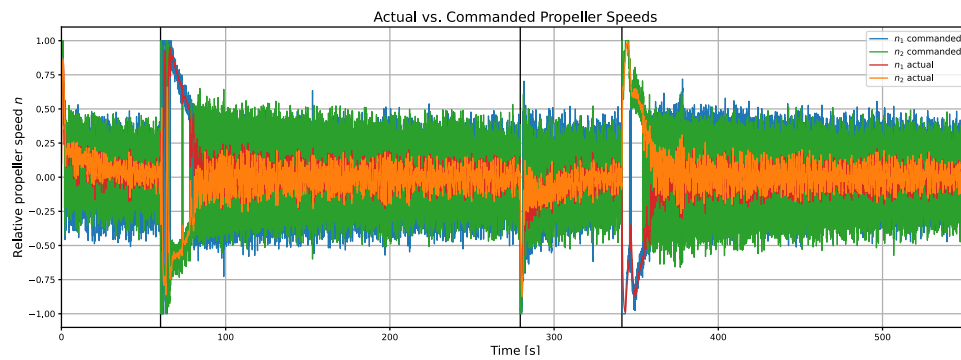


Figure 6.15: DP system 1: Actual vs. commanded propeller speeds.

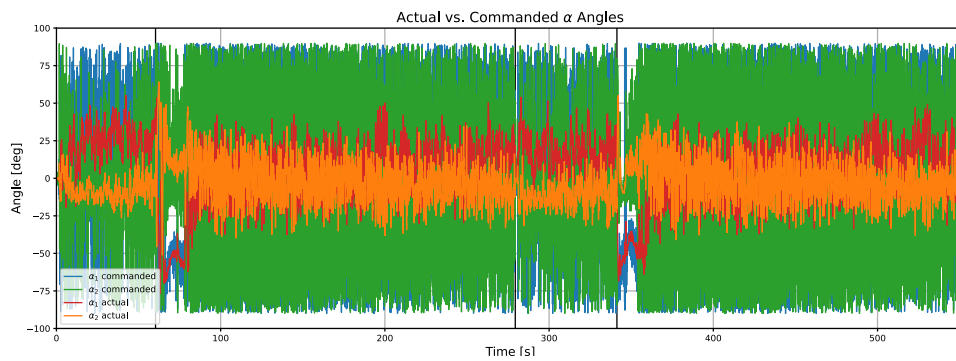


Figure 6.16: DP system 1: Actual vs. commanded azimuth angles.

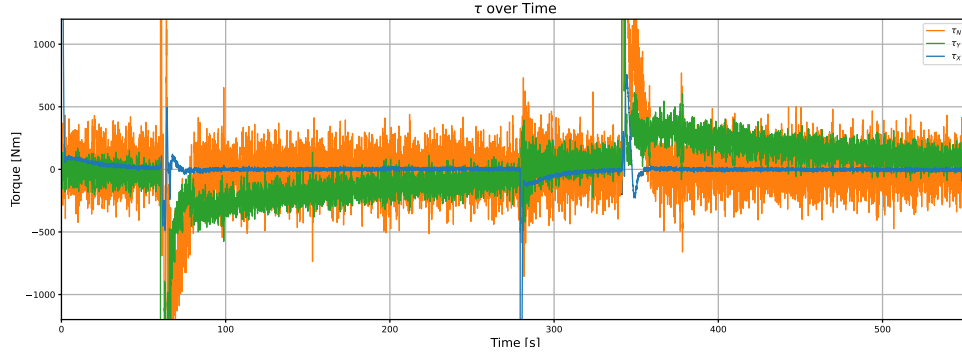


Figure 6.17: The virtual control inputs generated by the MIMO PID motion controller. The plot is made using DP system 1 and it is almost identical for system 2 and 3. The signal is far from realizable because of its high frequency, which explains the rapid changes in propeller speed and azimuth angles seen from the allocation methods used by DP system 1, 2, and 3 in **Table 6.4**.

DP Ideal Conditions: System 2 results

DP system 2 in **Table 6.4** uses nonlinear optimization to calculate the commanded control signals. The total state error is smaller compared to system 1, as seen in **Figure 6.18** and **Figure 6.19** and **6.20** shows a significant improvement in the stability of the control signals compared to system 1, even with a noisy virtual control signal. There is still high-frequency noise remaining in the commanded signals, but the amplitudes are more conservative, and therefore the discrepancies between the commanded and the actual propeller speed and azimuth angle values are smaller. It can be seen in **Figure 6.13b** that the consequence of the more conservative commands given by system 2 is a smoother response compared to the pseudo-inverse based allocation in system 1. Looking at **Figure 6.18**, the time required to reduce the state errors and complete the four-square test is also significantly reduced. The smoother response of system 2 is largely caused by the penalty on large control changes in **Algorithm 4**.

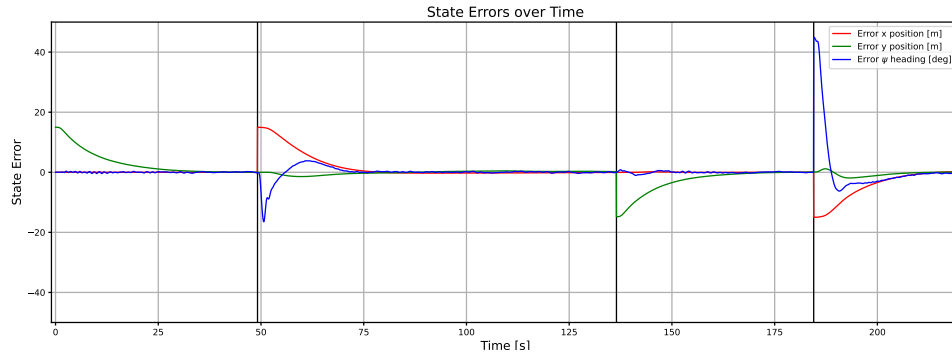


Figure 6.18: DP system 2: X, Y and heading error over time.

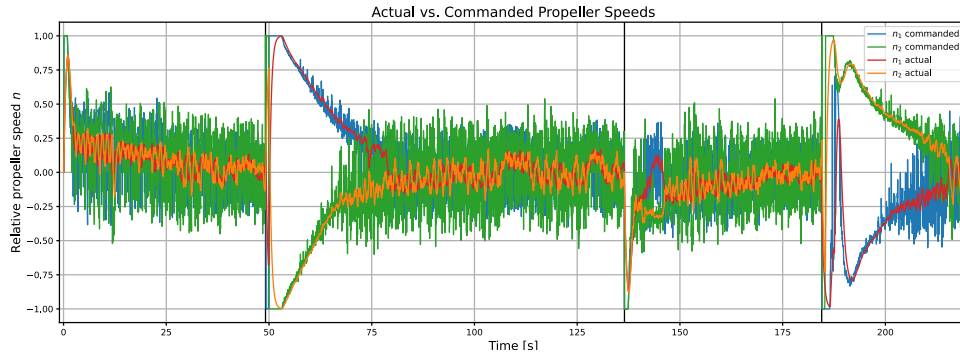


Figure 6.19: DP system 2: Actual vs. commanded propeller speeds.

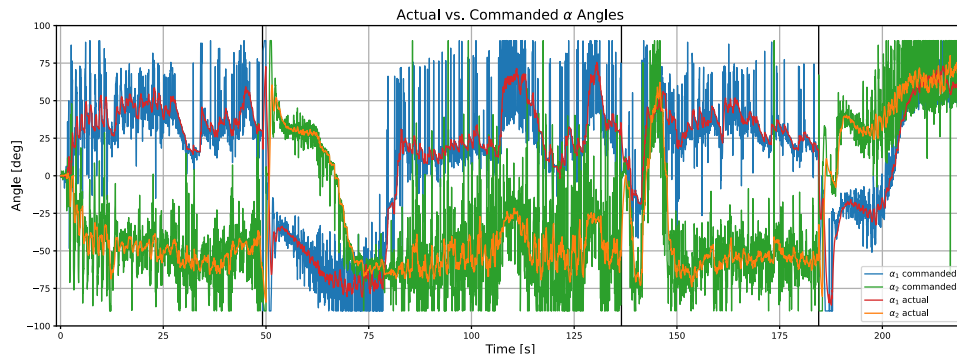


Figure 6.20: DP system 2: Actual vs. commanded azimuth angles.

DP Ideal Conditions: System 3 results

Control System 3 in Table 6.4 show a similar performance in Figure 6.13c to system 2. The actual and commanded propeller speeds displayed in Figure 6.22 is also similar to the propeller speeds in Figure 6.19 for system 2. The largest difference obtained by using an MPC formulation with actuator dynamics in system 3, compared to system 2, is an increase in large azimuth command changes, as seen in Figure 6.23.

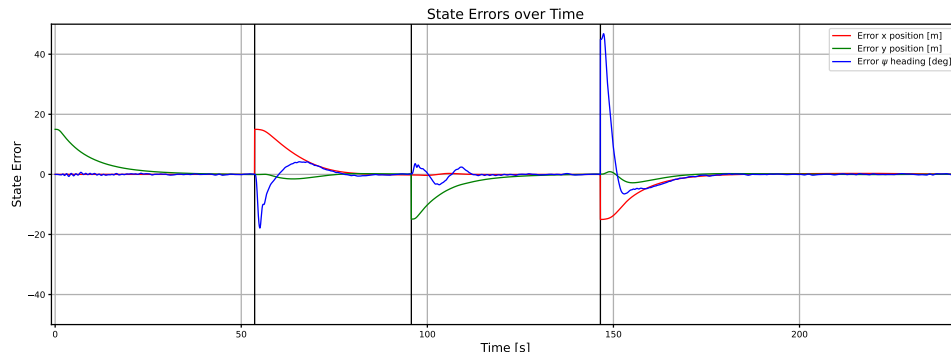


Figure 6.21: DP system 3: X, Y and heading error over time.

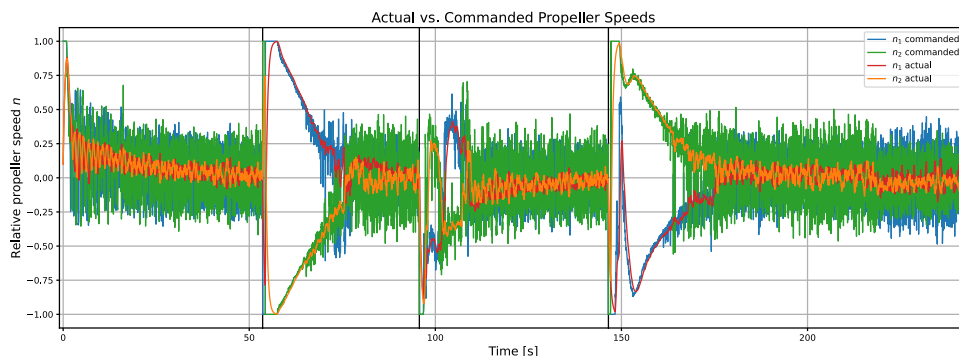


Figure 6.22: DP system 3: Actual vs. commanded propeller speeds.

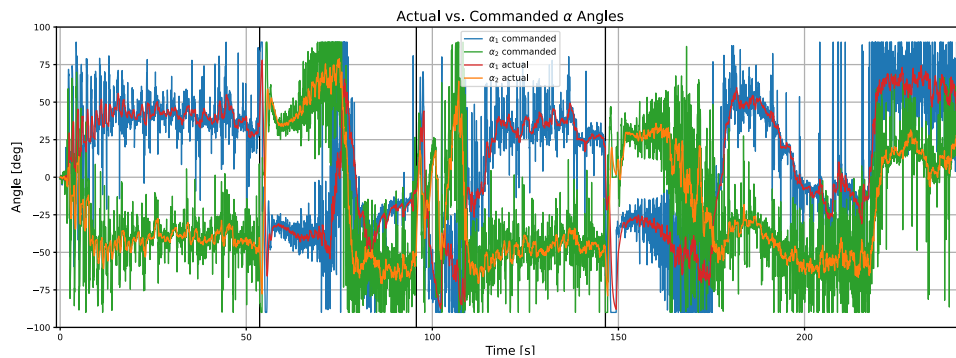


Figure 6.23: DP system 3: Actual vs. commanded azimuth angles.

DP Ideal Conditions: System 4 results

DP system 4 in Table 6.4, where an MPC uses a simplified model of the vessel to do motion control in addition to control allocation, displays significant improvements to the position and heading performance, as seen in Figure 6.13d. The commanded propeller inputs in Figure 6.25 and the commanded azimuth angles

in **Figure 6.26** are much smoother and the actuators are better able to follow the commands. In addition, compared to the other DP systems, the time used to complete the four-square test is significantly reduced, as seen in the propeller and azimuth angle figures.

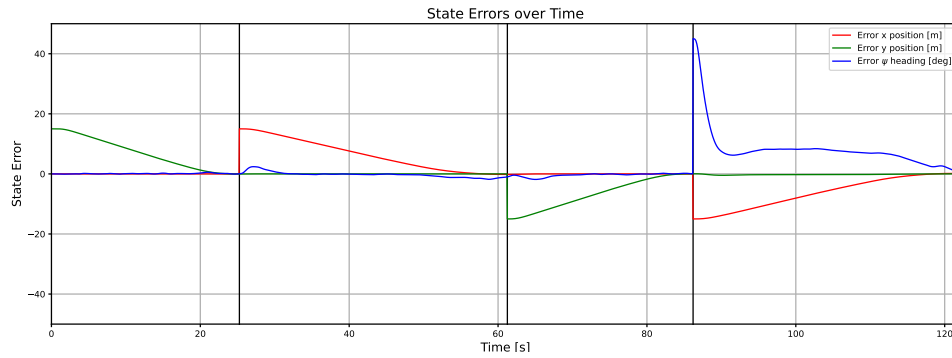


Figure 6.24: DP system 4: X, Y and heading error over time.

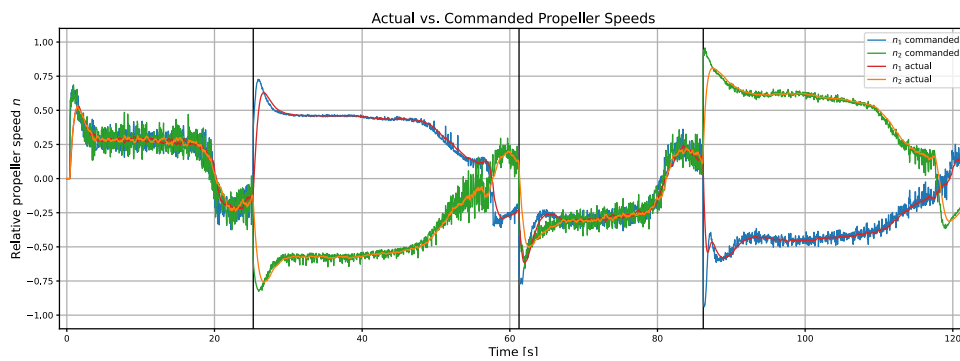


Figure 6.25: DP system 4: Actual vs. commanded propeller speeds.

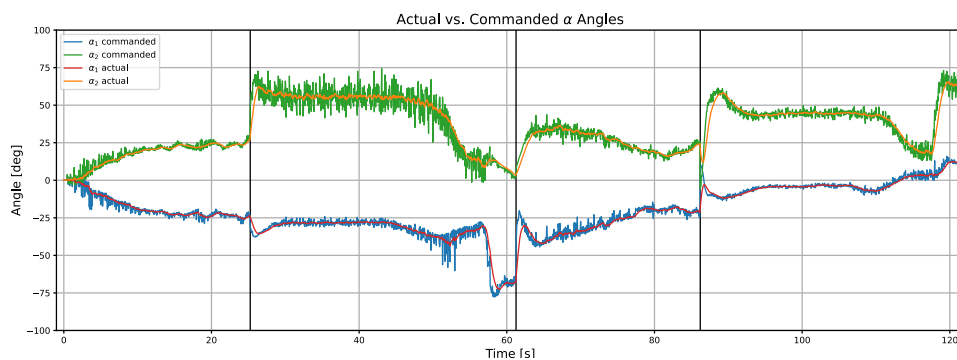


Figure 6.26: DP system 4: Actual vs. commanded azimuth angles.

6.2.2 Four-square test: with Ocean Currents

All systems from **Table 6.4** were evaluated using an ocean current of 0.3 m/s with an angle of 30° . The vessels position and heading response for all the systems that managed to complete the four-square test with ocean currents are shown in **Figure 6.27**.

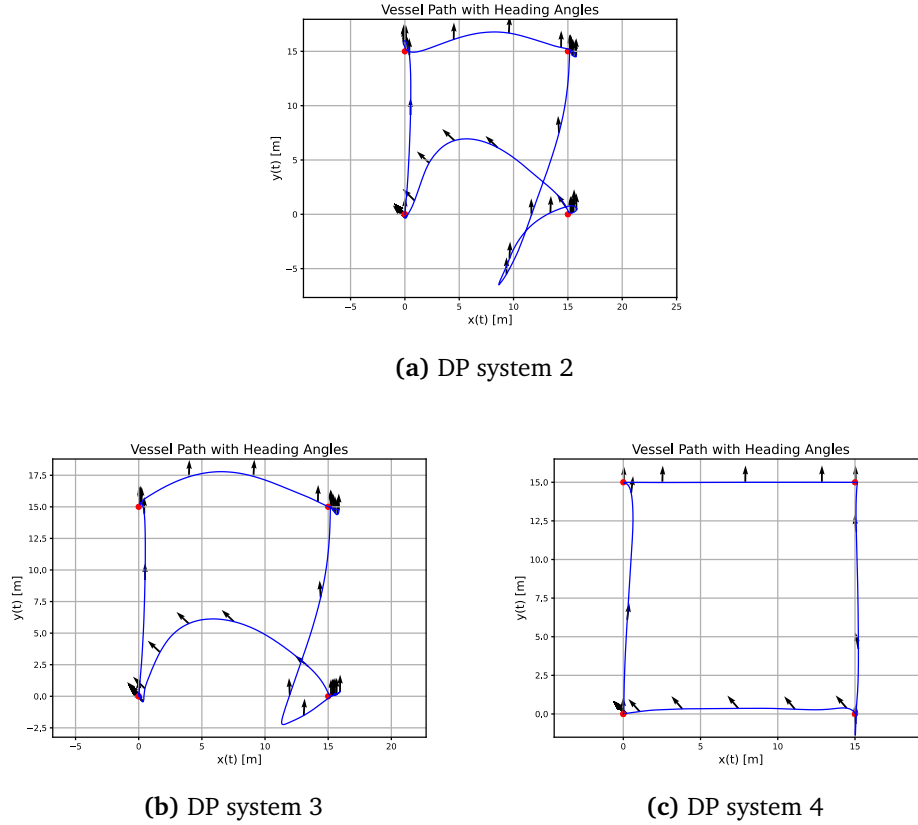


Figure 6.27: Vessel response in the four-square test under each control scheme, with ocean currents of 0.3 [m/s] with a 30° direction.

DP with currents: System 1 results

With ocean currents the problem of the actuators not being able to follow the commanded values, as shown in **Figure 6.15** and **6.16** becomes apparent. The vessel was not able to complete any test, because once the vessel got closer to a goal, it drifted away with the current. The slow convergence towards 0 for small state errors under ideal conditions, shown in **Figure 6.14**, already showed signs of controller issues, which became apparent with ocean currents.

DP with currents: System 2 results

With ocean currents the position and heading errors grow larger and does not disappear as quickly when compared to ideal conditions. This can be seen by comparing the errors under ideal conditions in **Figure 6.18** with the errors when ocean currents are added in **Figure 6.28**. In addition, the commanded propeller speeds and azimuth angles in **Figure 6.29** and **6.30** have higher amplitude noise, which might explain why the general performance degrades, and perhaps also why going backwards in the third segment of the four-square test in **Figure 6.27a** works poorly.

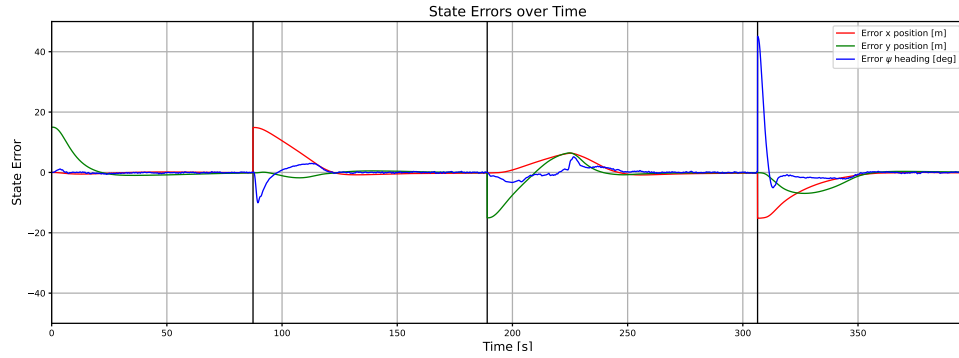


Figure 6.28: DP system 2 under ocean currents: X, Y and heading error over time.

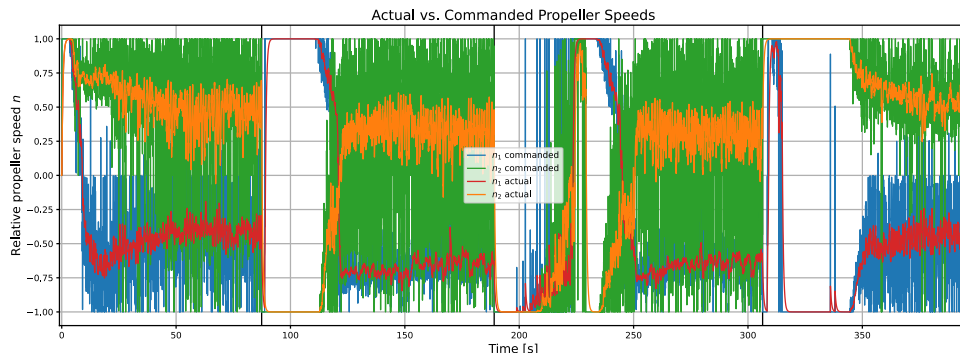


Figure 6.29: DP system 2 under ocean currents: Actual vs. commanded propeller speeds.

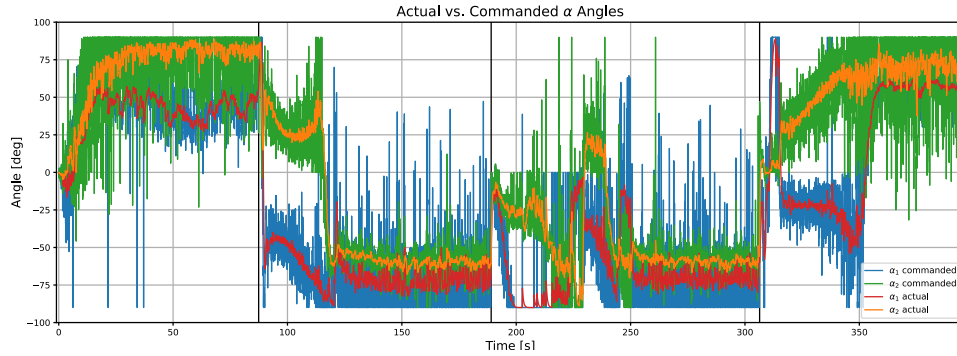


Figure 6.30: DP system 2 under ocean currents: Actual vs. commanded azimuth angles.

DP with currents: System 3 results

The same general effects of vessel response degradation seen for system 2 can be seen for system 3 in **Figure 6.27b**. However, system 3 is a bit better at reducing large errors while there are ocean currents. This can be seen by studying **Figure 6.31**, where the time used to reduce large errors are shorter compared to system 2 with ocean currents.

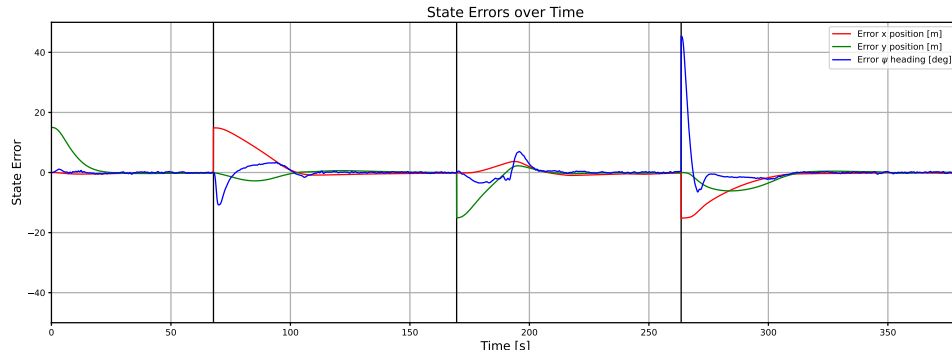


Figure 6.31: DP system 3 under ocean currents: X, Y and heading error over time.

DP with currents: System 4 results

With system 4, the vessel still performs exceptionally when there are ocean currents, as seen in **Figure 6.27c**. The azimuth angles in **Figure 6.34** are different from the ideal case, with large swings that was not there before. This is happening because the system changes the vessels heading to counteract the ocean current. Looking at the position and heading errors in **Figure 6.32**, we see that the MPC in system 4 actively sacrifice heading to counteract the ocean current and still move efficiently towards the goal state, where both the position and heading errors goes towards zero. The good performance under ocean currents is partially achieved by adjusting the heading of the vessel, but also by substantially increasing thrust

from the propellers, as seen when comparing the propeller speeds in **Figure 6.33** to the ideal case in **Figure 6.25**. Interestingly, the propeller speeds saturate significantly, which implies that currents stronger than 0.3 m/s could lead to much worse performance, since the vessel would no longer be able to compensate as effectively.

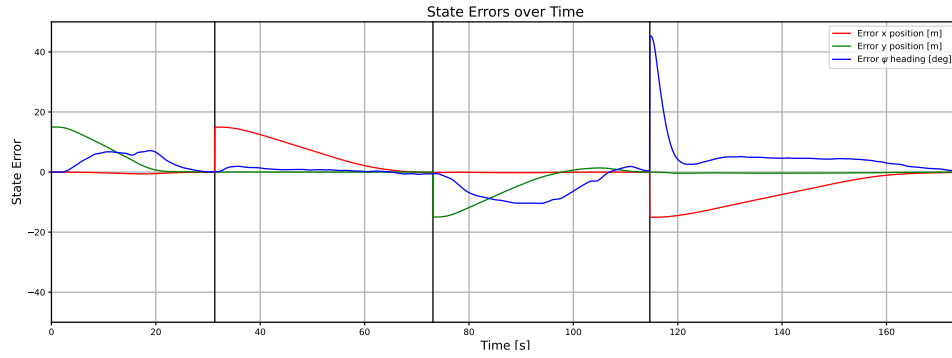


Figure 6.32: DP system 4 under ocean currents: X, Y and heading error over time.

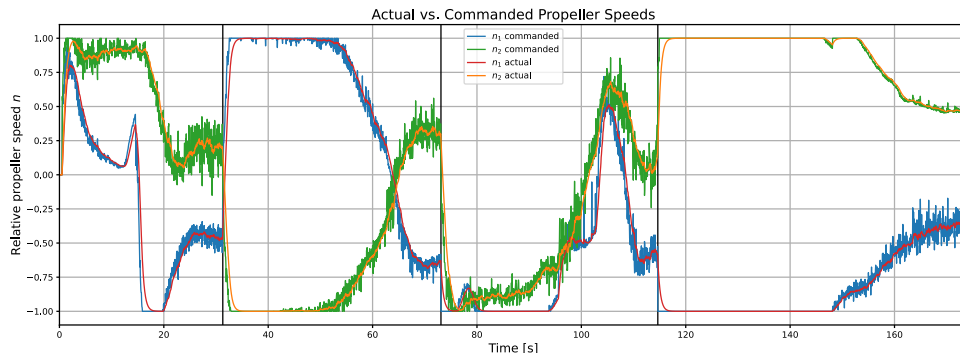


Figure 6.33: DP system 4 under ocean currents: Actual vs. commanded propeller speeds.

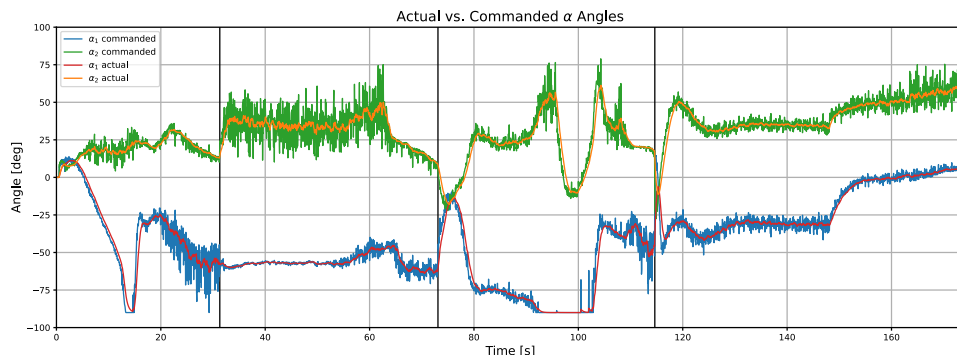


Figure 6.34: DP system 4 under ocean currents: Actual vs. commanded azimuth angles.

6.2.3 Four-square test: with Pod Failure

If the propeller on one of the two pod stops for any reason, RAN goes from having 4 to having 2 control inputs for controlling 3 DOF. This has now become an underactuated control problem, meaning that we cannot freely control all degrees of freedom. The fact that these two control inputs are not independent makes the situation worse.

DP with pod failure: System 4 results

Only System 4 in **Table 6.4**, the MPC framework capturing the whole control system, was able to go from start to goal in the four-square test with pod failure under ideal environmental conditions. To achieve the following results, the weights in the MPC had to be adjusted so that getting to the goal was of much greater importance than heading errors. The behavior can be seen in **Figure 6.35**. As expected, the underactuated system has considerable performance degradation. **Figure 6.36** show that the MPC prioritize minimizing the positional errors. However, the system is able to plan the maneuvers such that it ends up with a heading error of approximately 0° when the goal position is reached. The propeller speeds and azimuth angles used by the MPC system are shown in **Figure 6.37** and **Figure 6.38**, respectively. In these figures we see that the pod changes its orientation and thrust direction frequently, in contrast to the fully operational case. Since the pod dynamics is faster than the vessel hull dynamics, the vessel can essentially wiggle back and fourth in order to control all 3 DOF over time. The ability to plan like this based on the dynamics is most certainly the reason why the MPC with positional and heading objectives is the only system able to finish the test with pod failure.

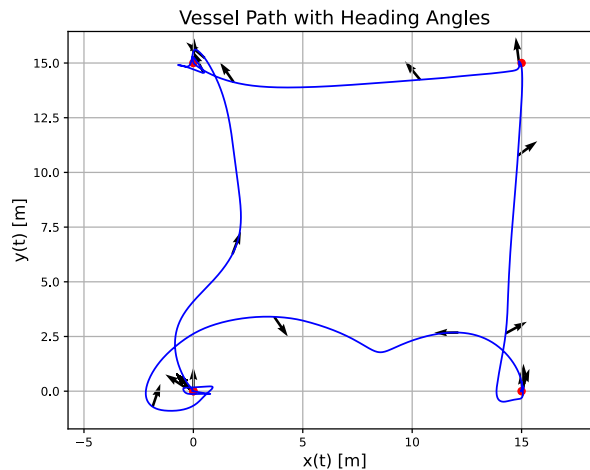


Figure 6.35: DP system 4 with pod failure ($n_1 = 0$): Vessel response.

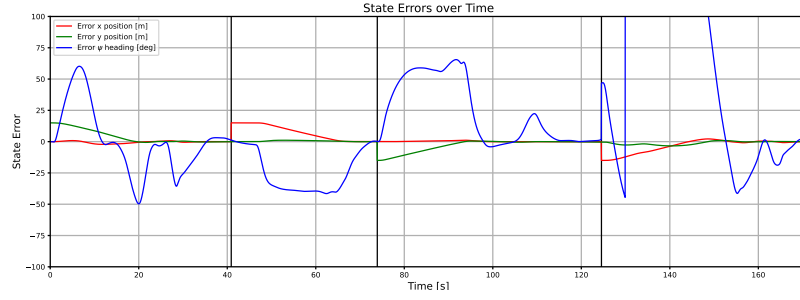


Figure 6.36: DP system 4 with pod failure ($n_1 = 0$): X, Y and heading error over time.

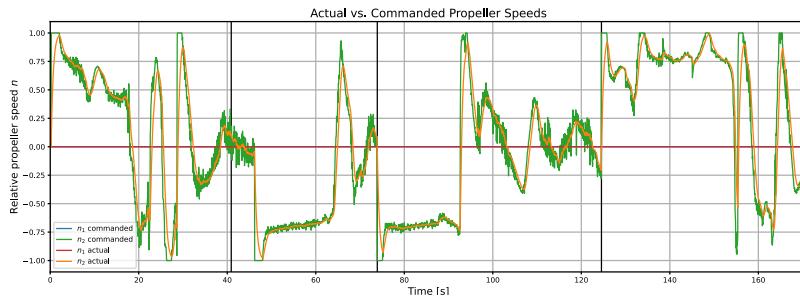


Figure 6.37: DP system 4 with pod failure ($n_1 = 0$): Actual vs. commanded propeller speeds.

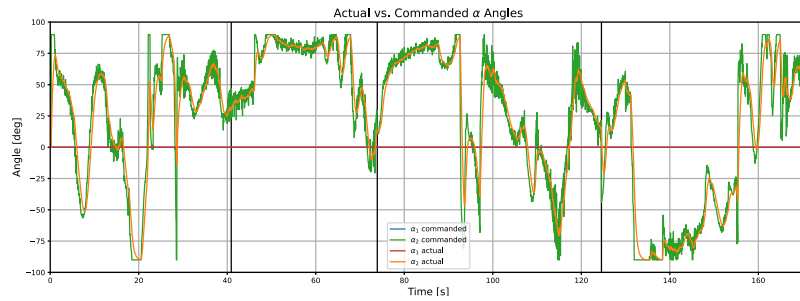


Figure 6.38: DP system 4 with pod failure ($n_1 = 0$): Actual vs. commanded azimuth angles.

6.3 Points of consideration

The following considerations are made with respect to the path-following and four-square test results.

Comparing Allocation methods for Path Following Although there were some differences in the commanded propeller speeds and azimuth angles among the dif-

ferent allocation strategies for PF, the response of the vessel under nominal conditions was practically the same for all methods. This is a good reason not to use azimuths in PF. However, system 1's pseudo-inverse-based allocation, which does not use azimuths, did not handle pod failures well, in contrast to the optimization-based methods, which used the azimuths more actively to compensate. In all cases, PF system 2 and 3 behaved almost identically in every aspect, indicating that the additional computation required by the MPC allocation method in system 3 is unnecessary. In addition, none of the optimization-based methods experienced solver issues during any test. Of the developed allocation methods, the optimization-based method described by **Algorithm 4**, used in PF system 2, appears to be the best for path-following applications, as it can handle pod failures effectively without requiring a considerable amount of computation.

No direct current compensation for path following systems None of the control systems in **Table 6.1** include an explicit current-compensation term anywhere in their control loops. As a result, when facing a constant 0.3 m/s current, the vessel inevitably drifts off the desired path, which the Nomoto-heading PID counteracts by adjusting the vessels heading. However, this is not perfect, and leads to a constant cross-track error. To contract ocean currents, it would be natural to use a guidance method capable of doing so, instead of the simple LOS guidance used here.

Limitations of the MIMO PID motion controller The DP and berthing capabilities of the USV RAN were evaluated using four different control systems in three different four-square tests. Systems 1–3 from **Table 6.4** generate a virtual control signal using a MIMO PID motion controller to find thrust magnitudes and azimuth angles. However, these virtual control signals exhibit high amplitude and frequency noise, even when navigation disturbances are removed, which degrades the performance of the allocation methods. Due to the poor performance of the motion controller, it is hard to make conclusions about how well these allocation methods *can* work for DP and berthing scenarios. System 4 in **Table 6.4** (the MPC control system) embeds the vessel's dynamics, including ocean currents, directly in its prediction horizon and cost function, eliminating the need for a separate motion controller. As a result, System 4 generates much smoother propeller speed and azimuth angle commands, leading to a more consistent vessel response in each part of the four-square tests.

Realistic currents One of the four-square tests included a mild ocean current of 0.3 m/s angled at 30°. With this current, Systems 1–3 struggles, whereas System 4 handles it effectively. More aggressive or spatially varying currents were not explored, leaving open questions about each system's robustness under stronger and more unpredictable disturbances.

Safety Margins with Dual-Pod Architecture Pod-failure results from the four-square test highlight RAN's limited redundancy. With only two azimuth pods, losing one drastically reduces control authority. Systems 1–3 failed to maintain sufficient control when pod 1 failed and only pod 2 remained operational. However, system 4 still managed to complete the test, but with a significant performance reduction, even in ideal conditions. The mix of the MIMO PID motion control and the control allocation systems provides insufficient fault tolerance, whereas the MPC capturing the whole control system (DP System 4) has better resilience. Essentially, a two-pod configuration limits redundancy and narrows safety margins for DP and berthing applications. Nevertheless, if the probability of pod failure is acceptably low given potential consequences, results from the four-square tests when using DP system 4 indicate that RAN can reliably execute these tasks when both pods are fully operational.

Slipstream modeling None of the control systems model slipstream effects in their pod-force calculations, including the vessel model. As a result, it is not possible to evaluate the effectiveness of penalizing slipstream disturbances. Incorporating wake interactions into the pod model would be necessary before drawing definitive conclusions. For this reason, slipstream disturbances were not penalized in any of the optimization-based control allocation methods. The penalty terms for loss of surge control achieves almost the same effect as slipstream disturbance penalties. However, loss of surge was not penalized to the extent of completely hindering the systems from giving $\pm 90^\circ$ azimuth angle commands.

Solver Performance and Reliability It is worth noting that the numerical solver (IPOPT) employed by the optimization-based allocation methods did not encounter any convergence issues during the tests conducted in this work. This reliable performance under the evaluated scenarios suggests that the solver is robust for the range of low-speed maneuvers considered.

Chapter 7

Conclusion

This work has examined the potential of small/mid size dual-pod catamaran USVs, for low-speed, confined-water applications, using different control systems on a model of the USV RAN. Three allocation methods were developed for both PF and DP / automatic berthing applications. Evaluations were done under ideal conditions, water currents and pod failure. One of the allocation methods is based on the pseudo inverse, while the other two methods use nonlinear-optimization. In addition, an MPC framework capturing the whole control system were developed for the challenging DP/berthing tests, showing significant improvements over the other methods.

In PF tests under ideal conditions, all three allocators tracked straight-line and Fermat's-spiral paths. The Fermat's spiral path gave reduced deviations and required fewer thrust adjustments. For the PF scenarios, none of the three allocators explicitly modeled ocean currents or slipstream effects, and it was not accounted for elsewhere in the control loop. As a result, in PF scenarios with currents, each system drifted until the Nomoto PID in the motion controller reached a heading bias that countered the current, leaving a nonzero cross-track error while following the path. In the PF pod-failure tests, the pseudo-inverse allocator could not be used, while the other two PF systems, using nonlinear-optimization based allocators, continued to follow the path using the remaining pod, but with reduced stability margins.

In the four-square tests, used for evaluation of DP/berthing potential, the first three allocators suffered from noisy virtual-control signals, making their intrinsic performance unclear. The MPC control system, described by **Algorithm 6**, produced smoother commands and tighter tracking, showing that combining motion control and control allocation can yield good low-speed maneuvering performance. In the four-square tests, the first three allocators relied on a MIMO PID motion controller to counteract currents. Even with ocean current compensation in the motion controller, the DP system using the pseudo-inverse allocation method was unable to do the test, drifting away with the currents. The other two DP systems using the MIMO PID, were able to counteract the currents, but with meaningful performance degradation. The fourth method, the MPC control

system, explicitly included the current in its model, keeping up its good performance with minimal degradation. In the four-square test with pod failure, only the MPC control system were able to perform, demonstrating superior resilience when compared to the other approaches that was unable to converge towards the goal states.

The numerical solver used by the optimization-based allocation methods did not encounter any convergence problems, but very aggressive maneuvers were not tested, so numerical stability under extreme conditions remains uncertain. To mitigate potential solver failures, it was suggested to have a fallback allocation strategy that does not rely on iterative optimization, such as the pseudo-inverse method described by **Algorithm 3**.

Future Work

- **Develop an enhanced DP and berthing motion controller.** Since the evaluation of the allocators in DP and berthing tests currently relies on the noisy MIMO PID virtual-control outputs, designing a controller that yields smoother, lower-noise commands will enable a more accurate assessment of each allocator's true DP and berthing capabilities. A better low-pass filter on the MIMO PID motion controller might also work.
- **Implement current-aware guidance for PF systems.** Replace the simple LOS guidance with a method that explicitly compensates for ocean currents to eliminate the steady cross-track error observed when a constant current is present.
- **Refine the pod-force model.** Incorporating slipstream and wake interactions into the thrust model can enable more realistic evaluations and fair comparisons between allocation methods with and without slipstream compensation.
- **Alternative allocation frameworks.** Chapter 2 identified MIP, NN, and RL MPC approaches that were not tested in this work. In particular, an RL-augmented MPC could further enhance robustness under stronger and more unpredictable currents or fault conditions.
- **Automate waypoint placement for Fermat's spiral paths.** Waypoints were chosen manually in this work. Developing an algorithm to optimize waypoint placement for the spiral geometry would streamline path planning and enable systematic comparisons across all systems.

Bibliography

- [1] T. I. Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control*, 2nd. John Wiley and Sons, 2021.
- [2] T. A. Johansen and T. I. Fossen, 'Control allocation: A survey,' *Automatica*, vol. 49, no. 5, 2013.
- [3] T. A. Johansen, 'Optimizing nonlinear control allocation,' in *IEEE Conf. Decision and Control, Nassau, Bahamas*, 2004, pp. 3435–3440.
- [4] R. Skulstad, G. Li, T. I. Fossen and H. Zhang, 'Constrained control allocation for dynamic ship positioning using deep neural network,' *Ocean Engineering*, 2023.
- [5] F. Arditti, H. Cozijn, E. V. Daalen and E. A. Tannuri, 'Robust thrust allocation algorithm considering hydrodynamic interactions and actuator physical limitations,' *Ocean Engineering*, 2019.
- [6] R. I. Julianto, T. Muttaqie, R. Adiputra, S. Hadi, R. L. L. G. Hidajat and A. R. Prabowo, 'Hydrodynamic and structural investigations of catamaran design,' *6th International Conference on Industrial, Mechanical, Electrical and Chemical Engineering*, 2020.
- [7] SNAME, 'Nomenclature for Treating the Motion of a Submerged Body Through a Fluid,' The Society of Naval Architects and Marine Engineers, Technical and Research Bulletin No. 1-5, Apr. 1950.
- [8] A. K. S. Mehdi Naderi Tor Arne Johansen, 'A fault tolerant control scheme using the feasible constrained control allocation strategy,' *International Journal of Automation and Computing*, 2019.
- [9] O. J. Sjørdalen, 'Optimal thrust allocation for marine vessels,' *Control Engineering Practice*, vol. 5, pp. 1223–1231, 1997.
- [10] S. Skjong and E. Pedersen, 'Non-angular mpc-based thrust allocation algorithm for marine vessels - a study of optimal thruster commands,' *IEEE Transactions on Transportation Electrification*, 2017.
- [11] N. Institutt for matematiske fag, *Differential Equations, Linear Algebra and Its Applications*, Second. Pearson, 2014.

- [12] B. Foss and T. A. N. Heirung, 'Merging optimization and control,' Faculty of Information Technology, Mathematics, and Electrical Engineering, Tech. Rep., 2016.
- [13] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th. Pearson, 2021.
- [14] W. C. Durham, 'Constrained control allocation,' *Journal of Guidance, Control, and Dynamics*, vol. 16, pp. 717–725, 1993.
- [15] J. Virnig and D. Bodden, 'Multivariable control allocation and control law conditioning when control effectors limit,' in *Proc. AIAA Guidance, Navigation and Control Conf., Scottsdale, AZ*, 1994, AIAA 94–3609.
- [16] J. Shi, W. Zhang, G. Li and X. Liu, 'Research on allocation efficiency of the redistributed pseudo inverse algorithm,' *Science China Information Sciences*, vol. 53, pp. 271–277, 2010.
- [17] R. J. Adams, J. M. Buffington, A. G. Sparks and S. S. Banda, *Robust Multivariable Flight Control*. London: Springer-Verlag, 1994.
- [18] J. M. Buffington and D. F. Enns, 'Lyapunov stability analysis of daisy chain control allocation,' *Journal of Guidance, Control, and Dynamics*, vol. 19, pp. 1226–1230, 1996.
- [19] M. Oppenheimer, D. Doman and M. Bolender, 'Control allocation,' in *The Control Handbook, Control System Applications, Second Edition*, W. S. Levine, Ed., 2010, ch. 8.
- [20] A. Casavola and E. Garone, 'Fault-tolerant adaptive control allocation schemes for overactuated systems,' *International Journal of Robust and Nonlinear Control*, vol. 20, pp. 1958–1980, 2010.
- [21] Y. Luo, A. Serrani, S. Yurkovich, D. B. Doman and M. W. Oppenheimer, 'Model predictive dynamic control allocation with actuator dynamics,' in *Proc. American Control Conference, Boston*, 2004, pp. 1695–1700.
- [22] C. Vermillion, J. Sun and K. Butts, 'Model predictive control allocation for overactuated systems - stability and performance,' in *IEEE Conf. Decision and Control, New Orleans, LA*, 2007, pp. 1251–1256.
- [23] M. Hanger, T. A. Johansen, G. K. Mykland and A. Skullestad, 'Dynamic model predictive control allocation using cvxgen,' in *9th IEEE International Conference on Control and Automation (ICCA11), Santiago, Chile*, 2011.
- [24] L. Zaccarian, 'Dynamic allocation for input redundant control systems,' *Automatica*, vol. 45, pp. 1431–1438, 2009.
- [25] A. Tavasoli and M. Naraghi, 'Comparison of static and dynamic control allocation techniques for integrated vehicle control,' in *Proceedings of the IFAC World Congress, Milano*, 2011, pp. 7180–7185.

- [26] F. Liao, K.-Y. Lum, J. L. Wang and M. Benosman, 'Constrained nonlinear finite-time control allocation,' in *Proc. American Control Conference, New York, NY*, 2007, pp. 3801–3806.
- [27] J. Tjønnås and T. A. Johansen, 'Optimizing non-linear adaptive control allocation,' in *Proc. IFAC World Congress, Prague*, 2005.
- [28] J. Tjønnås and T. A. Johansen, 'Adaptive control allocation,' *Automatica*, vol. 44, pp. 2754–2766, 2008.
- [29] J. Tjønnås and T. A. Johansen, 'Optimizing adaptive control allocation with actuator dynamics,' in *Proc. IEEE Conf. Decision and Control, New Orleans, LA*, 2007.
- [30] J. Tjønnås and T. A. Johansen, 'On optimizing nonlinear adaptive control allocation with actuator dynamics,' in *Proc. 7th IFAC Symposium on Non-linear Control Systems, Pretoria, South Africa*, 2007.
- [31] M. Bodson, 'Evaluation of optimization methods for control allocation,' *Journal of Guidance, Control, and Dynamics*, vol. 25, pp. 703–711, 2002.
- [32] T. A. Johansen, T. I. Fossen and P. Tøndel, 'Efficient optimal constrained control allocation via multi-parametric programming,' *Journal of Guidance, Control, and Dynamics*, vol. 28, pp. 506–515, 2005.
- [33] M. Kvasnica, P. Grieder and M. Baotić, *Multi-parametric toolbox (mpt)*, 2004.
- [34] J. Spjøtvold and T. A. Johansen, 'Fault tolerant control allocation for a thruster-controlled floating platform using parametric programming,' in *Proc. IEEE Conf. Decision and Control, Shanghai*, 2009.
- [35] R. Venkataraman, M. Oppenheimer and D. Doman, 'A new control allocation method that accounts for effector dynamics,' in *Proc. IEEE Aerospace Conference*, vol. 4, 2004, pp. 2710–2715.
- [36] M. A. Bolender and D. Doman, 'Non-linear control allocation using piecewise linear functions,' *Journal of Guidance, Control, and Dynamics*, vol. 27, pp. 1017–1027, 2004.
- [37] A. Bemporad and M. Morari, 'Control of systems integrating logic, dynamics, and constraints,' *Automatica*, vol. 35, pp. 407–427, 1998.
- [38] W. P. M. H. Heemels, B. D. Schutter and A. Bemporad, 'Equivalence of hybrid dynamical models,' *Automatica*, vol. 37, pp. 1085–1091, 2001.
- [39] T. A. Johansen, T. P. Fuglseth, P. Tøndel and T. I. Fossen, 'Optimal constrained control allocation in marine surface vessels with rudders,' *Control Engineering Practice*, vol. 16, pp. 457–464, 2008.
- [40] E. Ruth and A. J. Sørensen, 'A solution to the nonconvex linearly constrained quadratic thrust allocation problem,' in *Proceedings of 8th Conference on Manoeuvring and Control of Marine Craft, Guaruj, Brazil*, 2009, pp. 195–200.

- [41] A. Gjersdal, 'Model predictive control using genetic algorithms,' Master thesis, Norwegian University of Science and Technology, Trondheim, Norway, 2023.
- [42] M. Erdal, 'Optimal thrust allocation for dynamic positioning systems using mixed integer linear programming,' Master thesis, Norwegian University of Science and Technology, Trondheim, Norway, 2015.
- [43] V. L. Poonamallee, S. Yurkovich, A. Serrani, D. B. Doman and M. W. Oppenheimer, 'A nonlinear programming approach for control allocation,' in *Proc. American Control Conference, Boston, MA*, 2004, pp. 1689–1694.
- [44] J. Nocedal and S. J. Wright, *Numerical Optimization*. Springer, 1999.
- [45] M. W. Oppenheimer and D. B. Doman, 'Methods for compensating for control allocator and actuator interactions,' *Journal of Guidance, Control, and Dynamics*, vol. 27, 2004.
- [46] H. N. Esfahani, 'Reinforcement learning-based control and state estimation using model predictive control and moving horizon estimation,' PhD thesis, Norwegian University of Science and Technology, Trondheim, Norway, 2024.
- [47] W. Cai, 'Engineering applications of model predictive control-based reinforcement learning,' PhD thesis, Norwegian University of Science and Technology, Trondheim, Norway, 2024.
- [48] F. Liao, K.-Y. Lum, J. L. Wang and M. Benosman, 'Adaptive control allocation for non-linear systems with internal dynamics,' *IET Control Theory and Applications*, vol. 4, pp. 909–922, 2009.
- [49] F. Liao, K.-Y. Lum, J. L. Wang and M. Benosman, 'Adaptive nonlinear control allocation of non-minimum phase uncertain systems,' in *Proc. American Control Conference*, St. Louis, MO, 2009, pp. 2587–2591.
- [50] T. A. J. Mehdi Naderi Ali Khaki Sedigh, 'Guaranteed feasible control allocation using model predictive control,' *Control Theory and Technology*, 2019.
- [51] T. I. Fossen and T. Perez, *Marine systems simulator (mss)*, <https://github.com/cybergalactic/MSS>, Accessed: 2025-05-02. A Matlab/Simulink library for marine control systems design. Documentation: <https://wiley.fossen.biz>, 2004.
- [52] A. M. Lekkas, A. R. Dahl, M. Breivik and T. I. Fossen, 'Continuous-curvature path generation using fermat's spiral,' *Modeling, Identification and Control*, vol. 34, no. 4, pp. 183–198, 2013. DOI: 10.4173/mic.2013.4.3. [Online]. Available: <https://www.mic-journal.no/ABS/MIC-2013-4-3.asp>.
- [53] M. Candeloro, A. M. Lekkas and A. J. Sørensen, 'A voronoi-diagram-based dynamic path-planning system for underactuated marine vessels,' *Control Engineering Practice*, vol. 61, pp. 36–51, 2017. DOI: 10.1016/j.conengprac.2017.01.007. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0967066117300056>.

Appendix A

Simplified model parameters used by the MPC control system

Parameter	Value
m_{eff}	880.0
$m_{\text{eff,sway}}$	1200.0
$I_{zz,\text{eff}}$	1250.0
T_{surge}	5
T_{sway}	6
T_{yaw}	5
X_u	$\frac{m_{\text{eff}}}{T_{\text{surge}}}$
Y_v	$\frac{m_{\text{eff,sway}}}{T_{\text{sway}}}$
D_y	0.5
$N_r^{(\ell)}$	$\frac{I_{zz,\text{eff}}}{T_{\text{yaw}}}$
$N_r^{(\text{nl})}$	10.0

Table A.1: Simplified vessel-model parameters used by the MPC control system to predict vessel movement over a horizon, using propeller speed, azimuth angle and ocean current information.

Appendix B

Repository Structure

The structure of the Projects GitHub repository.

```
MarineVesselSimulator/  
|-- build/  
|-- data/  
    |-- DP_pictures/  
    |-- path_following_pictures/  
    |-- simdata.csv  
    |-- wpt_change_times.csv  
    |-- ...  
|-- external/  
|-- include/  
|-- literature/  
|-- src/  
    |-- Control  
        |-- control_alloc_selector.cpp  
        |-- MPC_control_alloc.cpp  
        |-- MPC_control_system.cpp  
        |-- nonlin_constrained_alloc.cpp  
        |-- PID_heading_motion_control.cpp  
        |-- PID_MIMO_motion_control.cpp  
        |-- pseudo_inverse_allocation.cpp  
    |-- Guidance  
        |-- dynamic_positioning.cpp  
        |-- guidance_selector.cpp  
        |-- LOS_observer.cpp  
        |-- LOS.cpp  
    |-- Models  
        |-- model_utilities.cpp  
        |-- ran.cpp  
        |-- ref_model.cpp
```

```
|-- Planning
    |-- fermat_spiral_planning.cpp
    |-- plan_selector.cpp
    |-- straight_line_planning.cpp
|-- Utilities
    |-- calculations.cpp
    |-- plotting.cpp
|-- main.cpp
|-- .gitignore
|-- .gitmodules
|-- CMakeLists.txt
|-- config.yaml
|-- DependancyGuide.md
|-- environment.yml
|-- LICENSE
|-- README.md
```

Appendix C

Project Description

The original project description is included here for reference.



PROJECT DESCRIPTION SHEET

Name: Sigurd von Brandis
Department: Engineering Cybernetics
Thesis title (Norwegian): Optimal pådragsallokering med begrensninger for føreløse overflatefartøy (USV-er)
Thesis title (English): Optimal Constraint Control Allocation for Uncrewed Surface Vessels (USVs)

Project Description: The project aims to develop and analyze optimal constraint control allocation methods for uncrewed surface vessels (USVs), specifically for a catamaran equipped with dual azimuth pods. The research will involve a comprehensive study of control allocation strategies, physical and operational constraints, and fault tolerance mechanisms to enhance the vessel's performance, precision, and safety in constrained environments.

Key areas of focus and development include:

1. **Comprehensive Literature Review: Literature Review:** Conduct a detailed review of control allocation methods for surface vessels with azimuth thrusters. Analyze different approaches for optimizing control allocation under physical and operational constraints. Examine fault tolerance strategies for marine vessels, including redundancy and failure handling.
2. **Specification of Requirements:** Define the scope of the thesis and clarify key contributions. Identify the physical and operational constraints relevant to control allocation in USVs. Select a suitable catamaran model and azimuth pod configuration for analysis.
3. **Control Allocation Method Design:** Develop a control allocation framework tailored for low-speed operation in confined waters. Incorporate actuator limits, environmental disturbances, and performance constraints. Explore optimization techniques for achieving precise position and heading control.
4. **Fault-Tolerant Control Mechanisms:** Design a control strategy that ensures continued operation in the presence of actuator failures. Implement fault detection and reallocation mechanisms to maintain maneuverability and safety. Analyze system robustness under different failure scenarios.
5. **Simulation and Validation:** Implement the designed control allocation method in a simulation environment. Evaluate the effectiveness of the method under various operational constraints and fault conditions. Compare performance with existing control allocation strategies.
6. **Reporting and Conclusions:** Summarize key findings from the literature review, simulations, and performance analysis. Present conclusions on the feasibility and effectiveness of the proposed control allocation method. Provide recommendations for further research and practical implementation.

Thesis performed at: Department of Engineering Cybernetics, NTNU
Supervisor: Professor Thor I. Fossen, Dept. of Eng. Cybernetics, NTNU

Appendix D

Standard Agreement Contract

The contract between the Department Head, the Supervisor, the external organization and the Student for the project collaboration.



Norges teknisk-naturvitenskapelige universitet

Fastsatt av prorektor for utdanning 10.12.2020

STANDARDAVTALE

om utføring av studentoppgave i samarbeid med ekstern virksomhet

Avtalen er ufravikelig for studentoppgaver (heretter oppgave) ved NTNU som utføres i samarbeid med ekstern virksomhet.

Forklaring av begrep

Opphavsrett

Er den rett som den som skaper et åndsverk har til å fremstille eksemplar av åndsverket og gjøre det tilgjengelig for allmennheten. Et åndsverk kan være et litterært, vitenskapelig eller kunstnerisk verk. En studentoppgave vil være et åndsverk.

Eiendomsrett til resultater

Betyr at den som eier resultatene bestemmer over disse. Utgangspunktet er at studenten eier resultatene fra sitt studentarbeid. Studenten kan også overføre eiendomsretten til den eksterne virksomheten.

Bruksrett til resultater

Den som eier resultatene kan gi andre en rett til å bruke resultatene, f.eks. at studenten gir NTNU og den eksterne virksomheten rett til å bruke resultatene fra studentoppgaven i deres virksomhet.

Prosjektbakgrunn

Det partene i avtalen har med seg inn i prosjektet, dvs. som vedkommende eier eller har rettigheter til fra før og som brukes i det videre arbeidet med studentoppgaven. Dette kan også være materiale som tredjepersoner (som ikke er part i avtalen) har rettigheter til.

Utsatt offentliggjøring

Betyr at oppgaven ikke blir tilgjengelig for allmennheten før etter en viss tid, f.eks. før etter tre år. Da vil det kun være veileder ved NTNU, sensorene og den eksterne virksomheten som har tilgang til studentarbeidet de tre første årene etter at studentarbeidet er innlevert.

1. Avtaleparter

Norges teknisk-naturvitenskapelige universitet (NTNU)	
Institutt:	Institutt for teknisk kybernetikk
Veileder ved NTNU:	Thor Inge Fossen
e-post og tlf.	thor.fossen@ntnu.no, 73594361
Ekstern virksomhet:	Kongsberg Discovery
Ekstern virksomhet sin kontaktperson, e-post og tlf.:	Arne Johan Hestnes, arne.hestnes@kd.kongsberg.com, 40447757
Student:	Sigurd von Brandis
Fødselsdato:	04.09.2001
Ev. flere studenter ¹	

Partene har ansvar for å klarere eventuelle immaterielle rettigheter som studenten, NTNU, den eksterne eller tredjeperson (som ikke er part i avtalen) har til prosjektbakgrunn før bruk i forbindelse med utførelse av oppgaven. Eierskap til prosjektbakgrunn skal fremgå av eget vedlegg til avtalen der dette kan ha betydning for utførelse av oppgaven.

2. Utførelse av oppgave

Studenten skal utføre: (sett kryss)

Masteroppgave	
Bacheloroppgave	
Prosjektoppgave	☒
Annen oppgave	

Startdato:	13.01.2025
Sluttdato:	19.05.2025

Oppgavens arbeidstittel er:

Optimal Constraint Control Allocation for USVs

¹ Dersom flere studenter skriver oppgave i fellesskap, kan alle føres opp her. Rettigheter ligger da i fellesskap mellom studentene. Dersom ekstern virksomhet i stedet ønsker at det skal inngås egen avtale med hver enkelt student, gjøres dette.

Ansvarlig veileder ved NTNU har det overordnede faglige ansvaret for utforming og godkjenning av prosjektbeskrivelse og studentens læring.

3. Ekstern virksomhet sine plikter

Ekstern virksomhet skal stille med en kontaktperson som har nødvendig faglig kompetanse til å gi studenten tilstrekkelig veiledning i samarbeid med veileder ved NTNU. Ekstern kontaktperson fremgår i punkt 1.

Formålet med oppgaven er studentarbeid. Oppgaven utføres som ledd i studiet. Studenten skal ikke motta lønn eller lignende godtgjørelse fra den eksterne for studentarbeidet. Utgifter knyttet til gjennomføring av oppgaven skal dekkes av den eksterne. Aktuelle utgifter kan for eksempel være reiser, materialer for bygging av prototyp, innkjøp av prøver, tester på lab, kjemikalier. Studenten skal klarere dekning av utgifter med ekstern virksomhet på forhånd.

Ekstern virksomhet skal dekke følgende utgifter til utførelse av oppgaven:

Reiser til og fra avtalt sted, tokt-tid og støttefartøy.

Dekning av utgifter til annet enn det som er oppført her avgjøres av den eksterne underveis i arbeidet.

4. Studentens rettigheter

Studenten har opphavsrett til oppgaven². Alle resultater av oppgaven, skapt av studenten alene gjennom arbeidet med oppgaven, eies av studenten med de begrensninger som følger av punkt 5, 6 og 7 nedenfor. Eiendomsretten til resultatene overføres til ekstern virksomhet hvis punkt 5 b er avkrysset eller for tilfelle som i punkt 6 (overføring ved patenterbare oppfinnelser).

I henhold til lov om opphavsrett til åndsverk beholder alltid studenten de ideelle rettigheter til eget åndsverk, dvs. retten til navngivelse og vern mot krenkende bruk.

Studenten har rett til å inngå egen avtale med NTNU om publisering av sin oppgave i NTNUs institusjonelle arkiv på Internett (NTNU Open). Studenten har også rett til å publisere oppgaven eller deler av den i andre sammenhenger dersom det ikke i denne avtalen er avtalt begrensninger i adgangen til å publisere, jf. punkt 8.

5. Den eksterne virksomheten sine rettigheter

Der oppgaven bygger på, eller videreutvikler materiale og/eller metoder (prosjektbakgrunn) som eies av den eksterne, eies prosjektbakgrunnen fortsatt av den eksterne. Hvis studenten

² Jf. Lov om opphavsrett til åndsverk mv. av 15.06.2018 § 1

skal utnytte resultater som inkluderer den eksterne sin prosjektbakgrunn, forutsetter dette at det er inngått egen avtale om dette mellom studenten og den eksterne virksomheten.

Alternativ a) (sett kryss) Hovedregel

☒	Ekstern virksomhet skal ha bruksrett til resultatene av oppgaven
-------------------------------------	--

Dette innebærer at ekstern virksomhet skal ha rett til å benytte resultatene av oppgaven i egen virksomhet. Retten er ikke-eksklusiv.

Alternativ b) (sett kryss) Unntak

☐	Ekstern virksomhet skal ha eiendomsretten til resultatene av oppgaven og studentens bidrag i ekstern virksomhet sitt prosjekt
--------------------------	---

Begrunnelse for at ekstern virksomhet har behov for å få overført eiendomsrett til resultatene:

6. Godtgjøring ved patenterbare oppfinnelser

Dersom studenten i forbindelse med utførelsen av oppgaven har nådd frem til en patenterbar oppfinnelse, enten alene eller sammen med andre, kan den eksterne kreve retten til oppfinnelsen overført til seg. Dette forutsetter at utnyttelsen av oppfinnelsen faller inn under den eksterne sitt virksomhetsområde. I så fall har studenten krav på rimelig godtgjøring. Godtgjøringen skal fastsettes i samsvar med arbeidstakeroppfinnelsesloven § 7. Fristbestemmelsene i § 7 gis tilsvarende anvendelse.

7. NTNU sine rettigheter

De innleverte filer av oppgaven med vedlegg, som er nødvendig for sensur og arkivering ved NTNU, tilhører NTNU. NTNU får en vederlagsfri bruksrett til resultatene av oppgaven, inkludert vedlegg til denne, og kan benytte dette til undervisnings- og forskningsformål med de eventuelle begrensninger som fremgår i punkt 8.

8. Utsatt offentliggjøring

Hovedregelen er at studentoppgaver skal være offentlige.

Sett kryss

☒	Oppgaven skal være offentlig
-------------------------------------	------------------------------

I særlige tilfeller kan partene bli enige om at hele eller deler av oppgaven skal være undergitt utsatt offentliggjøring i maksimalt tre år. Hvis oppgaven unntas fra offentliggjøring, vil den kun være tilgjengelig for student, ekstern virksomhet og veileder i denne perioden. Sensurkomiteen vil ha tilgang til oppgaven i forbindelse med sensur. Student, veileder og sensorer har taushetsplikt om innhold som er unntatt offentliggjøring.

Oppgaven skal være underlagt utsatt offentliggjøring i (sett kryss hvis dette er aktuelt):

Sett kryss	Sett dato
☐	ett år
☐	to år
☐	tre år

Behovet for utsatt offentliggjøring er begrunnet ut fra følgende:

Dersom partene, etter at oppgaven er ferdig, blir enig om at det ikke er behov for utsatt offentliggjøring, kan dette endres. I så fall skal dette avtales skriftlig.

Vedlegg til oppgaven kan unntas ut over tre år etter forespørsel fra ekstern virksomhet. NTNU (ved instituttet) og student skal godta dette hvis den eksterne har saklig grunn for å be om at et eller flere vedlegg unntas. Ekstern virksomhet må sende forespørsel før oppgaven leveres.

De delene av oppgaven som ikke er undergitt utsatt offentliggjøring, kan publiseres i NTNUs institusjonelle arkiv, jf. punkt 4, siste avsnitt. Selv om oppgaven er undergitt utsatt offentliggjøring, skal ekstern virksomhet legge til rette for at studenten kan benytte hele eller deler av oppgaven i forbindelse med jobbsøknader samt videreføring i et master- eller doktorgradsarbeid.

9. Generelt

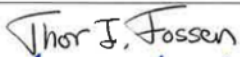
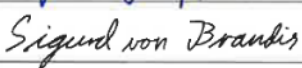
Denne avtalen skal ha gyldighet foran andre avtaler som er eller blir opprettet mellom to av partene som er nevnt ovenfor. Dersom student og ekstern virksomhet skal inngå avtale om konfidensialitet om det som studenten får kjennskap til i eller gjennom den eksterne virksomheten, kan NTNUs standardmal for konfidensialitetsavtale benyttes.

Den eksterne sin egen konfidensialitetsavtale, eventuell konfidensialitetsavtale den eksterne har inngått i samarbeidprosjekter, kan også brukes forutsatt at den ikke inneholder punkter i motstrid med denne avtalen (om rettigheter, offentliggjøring mm). Dersom det likevel viser seg at det er motstrid, skal NTNUs standardavtale om utføring av studentoppgave gå foran. Eventuell avtale om konfidensialitet skal vedlegges denne avtalen.

Eventuell uenighet som følge av denne avtalen skal søkes løst ved forhandlinger. Hvis dette ikke fører frem, er partene enige om at tvisten avgjøres ved voldgift i henhold til norsk lov. Tvisten avgjøres av sorenskriveren ved Sør-Trøndelag tingrett eller den han/hun oppnevner.

Denne avtale er signert i fire eksemplarer hvor partene skal ha hvert sitt eksemplar. Avtalen er gyldig når den er underskrevet av NTNU v/instituttleder.

Signaturer:

Instituttleder:		
Dato:		
Veileder ved NTNU:		2025-01-09
Dato:		
Ekstern virksomhet:		1.13.2025
Dato:		
Student:		1.8.2025
Dato:		
Ev. flere studenter		